全国大学生嵌入式芯片与系统设计竞赛

FPGA 创新设计赛道　中科亿海微　选题一：基于以太网 MAC 总线实现 AFDX

AFDX 端系统

FPGA 设计文档

方案与具体设计

队伍名称：[待填写]

参赛成员：[待填写]

指导教师：[待填写]

目标器件：EQ6HL130，CSG484_H 封装

开发环境：eLinx 3.0.7，ModelSim SE-64 2020.4

2026 年 10 月

摘要

本设计针对全国大学生嵌入式芯片与系统设计竞赛“基于以太网 MAC 总线实现 AFDX”赛题要求，基于中科亿海微 EQ6HL130 FPGA 研制了高完整性、双网冗余的 ARINC 664 Part 7 航电端系统。端系统逻辑由单一 125 MHz（8 ns 周期）主时钟驱动，以纯 Verilog HDL 实现了 128 条发送虚拟链路（VL）与 4096 条接收虚拟链路的线速硬件处理，并集成 ARINC 615A 软件数据加载（Data Loading）、SNMPv1/v2c 网络管理与高精度 SNTP/RTC 时钟同步服务。

面对大规模虚拟链路状态维护与 125 MHz 物理时序收敛的双重挑战，系统确立了“片上 BRAM 表格化管理与单主控状态机”的核心架构，全芯片划分为 24 个 RTL 模块：接收映射表采用 32-Bank 分块存储与双级读回流水，切断大扇出长连线延迟；管理面协议引擎（ASN.1/BER 与 TFTP）引入 2 分频局部时钟使能（CE_DIV $=2$），将复杂状态转移约束在 16 ns 多周期路径中；报文线速收发、RAM 访问与配置总线维持 125 MHz 单周期全速运行，兼顾复杂协议处理与周期级实时响应。

经系统级全功能仿真验证与官方工具链物理签核，设计达成以下核心指标：

1. 吞吐率与确定性时延：定长满载（77 VL）有效载荷吞吐率达 906.136 Mbps（81 VL 达 953.208 Mbps），多 VL 混合变长高负载（`var_high`）两方向单向吞吐率均达 904.752 Mbps（双向合计 1809.504 Mbps）；单帧端到端硬件流水时延典型值为 24.424 µs，满载稳态最大时延为 24.432 µs（其中发送段 L_{tx} 典型为 0.312 µs、至首前导码 0.256 µs，接收段 L_{rx} 首字节为 12.360 µs、末字节为 24.120 µs），显著优于赛题 1–2 ms 的指标要求。
2. 传输抖动控制：稳态发送间隔抖动实测为 0 µs；严格按官方 1022 答疑 R97 口径，定长满载同 VL 端到端时延极差抖动仅为 0.008 µs（1 拍，8 ns），多 VL 变长循环工况下为 9.992 µs，实测极差源于串行线路上前序在途帧排队深度的动态变化。
3. 硬件资源与物理时序收敛：逻辑综合消耗查找表（LUT）34294 个（占比 25.19%）、触发器（FF）17454 个（占比 12.82%）、嵌入式存储块 366 个 EBR（占比 30.50%）；在 eLinx 官方工具链（`S2d + PRES1.50`）下实现 125 MHz 布局布线后静态时序完全收敛，单周期路径最差建立裕量为 $+0.013$ ns、保持裕量为 $+0.229$ ns，多周期路径建立裕量为 $+0.940$ ns，全设计 45,745 个时序端点 0 违例，最高等效工作频率达 125.203 MHz。
4. 全流程验证覆盖：构建包含 39 项自动化回归用例的验证平台，累计通过 2585512887 次硬件断言自检与 3879565 项独立参考模型（Golden Model）逐字节比对；FSM 状态覆盖率达 100.00%、语句覆盖率达 98.55%、分支覆盖率达 96.46%；综合后 VQM 门级网表后仿真完全通过五功能联合测试。

关键词：AFDX 端系统；ARINC 664；FPGA；高完整性通信；静态时序收敛；布局布线

目录

摘要 **I**

第 **1** 章 概述 **1**
1.1 设计目标 . . . 1
1.2 主要成果 . . . 2
1.3 文档结构 . . . 3

第 **2** 章 需求分析与规格分解 **4**
2.1 赛题需求 . . . 4
2.2 官方答疑澄清与设计实现对照 . . . 4
2.3 设计规格 . . . 5
2.4 协议依据 . . . 6
2.5 规格与设计章节对应 . . . 6

第 **3** 章 总体架构 **8**
3.1 系统框图 . . . 8
3.2 模块层次与 24 个 RTL 模块划分 . . . 9
3.3 关键设计决策 . . . 10
3.4 时钟与复位 . . . 11
3.5 时间基准 . . . 12
3.6 端到端数据通路 . . . 12
3.7 设计预算 . . . 13

第 **4** 章 接口设计 **18**
4.1 顶层端口 . . . 18
4.2 GMII 接口 . . . 19
4.3 以太网帧格式 . . . 20
4.4 应用接口 . . . 21
4.5 配置总线 . . . 22
4.6 事件接口 . . . 23
4.7 内部接口 . . . 23

第 **5** 章 发送通路 **25**
5.1 发送核心与配置表 . . . 25
5.2 空闲页池 . . . 26
5.3 分页缓存 . . . 27
5.4 调度引擎 . . . 29
5.5 组帧器 . . . 31
5.6 双网发送 MAC . . . 34

第 **6** 章 接收通路 **36**
6.1 接收核心与配置表 . . . 36
6.2 接收 MAC . . . 38
6.3 帧头解析与预写入缓存 . . . 38
6.4 完整性检查与冗余管理 . . . 40
6.5 交付 . . . 43
6.6 统计分类 . . . 44

第 **7** 章 服务功能 **45**
7.1 服务通道 . . . 45
7.2 RTC 时钟同步 . . . 46
7.3 SNMP 网络管理 . . . 48
7.4 ARINC 615A 软件数据加载 . . . 51

第 **8** 章 配置、复位与统计 **55**
8.1 地址映射 . . . 55
8.2 配置表格式 . . . 56
8.3 配置读写与仲裁 . . . 57
8.4 配置写与查表的冲突仲裁 . . . 57
8.5 复位与初始化 . . . 57
8.6 统计计数器 . . . 58
8.7 异常处理汇总 . . . 58
8.8 主机编程流程 . . . 59
8.9 配置约束 . . . 60

第 **9** 章 时序与存储设计 **61**
9.1 时延预算与流水账本 . . . 61
9.2 片上 RAM 分配与物理存储规划 . . . 62
9.3 静态时序收敛与关键路径工程优化 . . . 64
9.4 时序设计规则与流水划分 . . . 65

第 **10** 章 可验证性设计与实测摘要 **67**
10.1 验证体系与自动化回归策略 . . . 67
10.2 性能与多虚拟链路变长压力实测 . . . 67
10.3 代码覆盖率签核 . . . 68
10.4 综合后门级网表功能对照仿真 . . . 68
10.5 硬件可观察性与可控性机制 . . . 69

第 **11** 章 工程交付与复现指南 **70**
11.1 统一的设计方法与架构收敛 . . . 70
11.2 工程交付清单与 24 个 RTL 模块 . . . 70
11.3 仿真测试复现指南 . . . 71
11.4 eLinx 物理实现与时序签核四步闭环 . . . 72

附录 **A** 源文件与默认参数 **73**
A.1 源文件 . . . 73
A.2 默认参数 . . . 74

附录 **B** 服务寄存器 **75**

附录 **C** 内部接口信号 **76**
C.1 消息接口 . . . 76
C.2 入队与授权 . . . 76
C.3 组帧器与 MAC . . . 77
C.4 帧描述与交付队列 . . . 77
C.5 服务发送总线 . . . 78

附录 **D** 术语与参考文献 **79**
D.1 术语 . . . 79
D.2 参考文献 . . . 79

第 1 章　概述

1.1　设计目标

AFDX（ARINC 664 Part 7）端系统是连接机载航空电子主机与确定性交换网络的核心枢纽。在发送方向，端系统将上层应用报文按 UDP/IP/以太网协议栈线速封装，严格依据各虚拟链路（Virtual Link, VL）设定的带宽分配间隔（Bandwidth Allocation Gap, BAG）执行整形调度，并向 A、B 双冗余物理网络锁步并发；在接收方向，端系统对双网到达帧流执行线速协议头解析与完整性检查（Integrity Checking, IC），并在冗余管理（Redundancy Management, RM）窗口内完成去重，确保向上层应用单次交付有效数据。赛题明确要求实现 ARINC 664 P7 基础通信、高完整性通信、SNMP 网络管理、ARINC 615A 软件数据加载（Data Loading）及 RTC 时钟同步五大功能，同时满足线速吞吐 $\geq$ 100 Mbps、端到端时延 $<$ 1–2 ms、抖动数十微秒级以及 128 发送 / 4096 接收 VL 的性能指标。

在 FPGA 上实现全硬件化的 AFDX 端系统，核心矛盾在于逐字节线速流水处理与大规模链路状态存储之间的架构平衡。4096 条接收 VL 与 128 条发送 VL 的配置与状态需要较大的片上存储容量；采用分布式寄存器阵列将导致逻辑资源随链路规模线性膨胀，而采用片上块 RAM（BRAM）虽可大幅节约逻辑，但同步读延迟要求引擎显式安排查表、判定和回写流水。此外，大容量映射表还会产生严重的地址扇出与读出选择延迟。为此，设计确立了“32-Bank 分块存储与读出流水、管理面隔拍时钟使能解耦、业务面 125 MHz 单周期全速运行”的系统微架构，在保证高存储密度的同时实现了 125 MHz 严格时序闭环。

本设计采用 Verilog-2001 HDL 构建全硬件化 AFDX 端系统顶层模块 `AFDX_top`，以单一 125 MHz（8 ns 周期）主时钟直接驱动八位 GMII 物理接口，实现双网双向 1 Gbps 物理线速（满载净有效载荷吞吐达 953.208 Mbps）。架构包含以下四项关键技术：

1. 基于片上 **BRAM** 的全表格化状态维护架构：128 条发送 VL 与 4096 条接收 VL 的配置参数及动态状态全部收敛入片上块 RAM，由专用硬件引擎按定拍流水执行读写更新；字节流水通路与深层控制逻辑由全部链路时分复用，实现每拍处理一字节的高效线速处理。
2. 前瞻预取与准点放行调度机制（**Lookahead Prefetch & Scheduled Release**）：发送引擎在 BAG 允许时刻到达前 512 拍提前检索候选 VL 并完成帧头组装。当报文提前排队且无线路冲突时，MAC 在允许发送时刻准点输出首前导码字节，实现稳态零发送抖动。
3. 双网锁步自研线速 **MAC**（**Lockstep Dual-MAC**）：采用单主控状态机同步驱动 A、B 双网 CRC-32 累加器与物理输出寄存器，保证双网副本在时钟周期级完全同拍起始；GMII 发送接口直接由 MAC 级寄存器引出，空闲启动延迟严格为 0 拍。
4. 服务通道状态折叠与流水时延精密补偿：SNMP 与 615A 协议引擎通过公共状态序列复用字节解析、字段拼装与响应发送逻辑，大幅收敛 FSM 逻辑面积；RTC 模块

采用逐位移位加减流水实现纳秒/NTP 小数定点换算，并对片内 5 拍固定写回流水（40 ns）执行硬件确定性补偿。

根据帧长、时钟周期和各级流水拍数，可计算端系统的载荷容量与端到端时延。表 1-2 汇总相关指标，第 3.7 节给出预算推导。

1.2 主要成果

表 1-1 列出赛题五项功能的实现要点，表 1-2 给出技术指标要求、设计目标与实测结果。

表 1-1 赛题功能与实现要点

赛题功能	实现要点	详见
ARINC 664 P7 通信	128 条发送 VL 按 BAG 调度，UDP/IP 封装，A、B 双网同时发送；4096 条接收 VL 按 VLID 直接查表	第 5、6 章
高完整性通信	FCS、帧长、IP/UDP 头逐帧检查；每网独立的序号完整性检查；冗余管理按处理顺序交付首个有效副本	第 6 章
SNMP 网络管理	v1/v2c 的 Get、GetNext、Set 和 v2 Trap，36 个管理对象，BER 逐字节编解码，多周期解耦	第 7.3 节
615A 软件数据加载	基于 TFTP 的单文件上传（LUI、LUS、LUR、LUH、数据文件）与介质定义下载（Media-Defined Download, MDD），8 KiB 加载区，CRC-16 硬件校验	第 7.4 节
RTC 时钟同步	SNTPv4 时钟同步，32 位秒加 32 位纳秒，8 ns 硬件时钟分辨率与 40 ns 延迟精确补偿（5 拍流水）	第 7.2 节

表 1-2 技术指标与实测映射

指标	赛题要求	设计目标	实测结果	依据与说明
传输速率	100 Mbps	载荷容量 956.44 Mbps	定长满载 906.136 Mbps；81 VL 极限 953.208 Mbps；变长高负载 904.752 Mbps	按接收端去重有效载荷计数，千兆 GMII 每拍一字节（第 3.7 节、表 10-1）
端到端时延	<1–2 ms	无争用时延 24.424 µs；预算上界 26.360 µs	满载最大时延 24.432 µs（L_{tx} 最大 0.312 µs / 首前导码 0.256 µs；L_{rx} 首字节 12.360 µs / 末字节 24.120 µs）	1022 答疑 R95/R96 口径；无排队立即发送从提交至首前导码 32 拍（0.256 µs）
传输抖动	≤ 几十微秒	稳态无争用为 0；争用上界 ≤50 µs	发送间隔抖动 0 µs；R97 同 VL 极差定长满载 0.008 µs、变长循环 9.992 µs	1022 答疑 R97 口径，同一 VL 多帧端到端时延最大减最小极差（第 3.7.3 节）
工作时序	125 MHz 稳定工作	综合建立裕量 ≥2 ns；布线后零违例	布局布线后（`S2d + PRES1.50`）建立裕量 +0.013 ns、保持裕量 +0.229 ns，最高频率 125.203 MHz	单周期路径 153,342 条，多周期路径 50,296 条（建立裕量 +0.940 ns），45,745 端点 0 违例
帧格式	VLID、SN 1–255，载荷 ≤1471 B	支持以太网帧、IPv4、UDP、SN 及 FCS	帧长 64–1518 字节，复位首帧 SN 为 0，之后 1–255 循环	符合 ARINC 664 P7 与 IEEE 802.3 标准
VL 容量	128 发送 / 4096 接收	128 发送 / 4096 接收，VLID 任意直接映射	128 发送与 4096 接收全配置抽测 100% 交付	块 RAM 状态表管理，接收映射表 65536 项全覆盖

1.3 文档结构

第 2–4 章说明规格、架构与接口，第 5–8 章说明收发、服务及配置，第 9–10 章说明时序、存储与可验证性，第 11 章总结设计。附录提供源文件、参数、寄存器、内部接口、术语及参考文献。

第 2 章　需求分析与规格分解

本章依据选题指南、官方《FPGA 创新大赛问题汇总》（1022 版，涵盖 R95–R97、R110–R112 等最新澄清）及表 2-3 所列协议规范，将赛题需求细化为功能、性能和接口规格。

2.1　赛题需求

选题指南（正文第 2–4 页）对 AFDX 端系统提出三类要求：

- 功能：ARINC 664 Part 7 通信、高完整性数据通信、SNMP 网络管理、ARINC 615A 软件数据加载、RTC 纳秒时钟同步。
- 技术指标：传输速率 100 Mbps；帧格式采用 IEEE 802.3 格式及 ARINC 664 扩展，含 VLID 和 1–255 的序列号，有效载荷不超过 1471 字节，总帧长不超过 1518 字节；端到端时延小于 1–2 ms，抖动不超过几十微秒；最多 128 条发送 VL、4096 条接收 VL。
- 接口与编码：顶层模块统一采用 `AFDX_top` 模板的 8 个端口；端口、信号和寄存器用小写，参数用大写，重要代码段须有简洁注释。

2.2　官方答疑澄清与设计实现对照

端到端时延从发送端应用接口成功提交消息末字节时（m_{last} 握手完成）开始计算，涵盖发送排队、组帧、MAC 处理、物理链路串行传输及接收处理的全流程时间。严格依据官方 1022 答疑 R97 口径，分别定义首字节端到端交付时延 L_{app} 与末字节端到端交付时延 $L_{\text{app,last}}$：

$$L_{\text{app}} = L_{\text{tx}} + L_{\text{gmii}} + L_{\text{rx}}, \qquad L_{\text{app,last}} = L_{\text{tx}} + L_{\text{gmii}} + L_{\text{rx,last}} \tag{2-1}$$

其中 L_{tx} 为应用提交末字节至 GMII 发送端输出以太网帧起始定界符（SFD，`8'hD5`）的时间；L_{gmii} 为发送端 SFD 至接收端 SFD 的网络链路传输时延（直连模型下为 0 或固定仿真延迟）；L_{rx} 为接收端 GMII 收到 SFD 至应用接口输出首字节交付的时间，$L_{\text{rx,last}}$ 为交付末字节的时间。各阶段拍数与流水预算推导见第 3.7 节。

表 2-1 官方 1022 答疑澄清与设计实现对照

项目与答疑序号	官方答疑要求与澄清	本设计的实现与架构边界
顶层端口映射 (R61, R95-2, R110)	R110 说明模板 8 端口是仿真模型对外的通用接口；R61 明确 GMII 为 PHY 接口，R95-2 要求两次例化 MAC	遵循 R61/R95-2 的物理分层边界：将模板 8 端口映射为 A 网 GMII 物理接口，扩展 B 网物理接口、独立的 `app_*` 应用数据流接口及 `cfg_*` 配置总线接口
MAC 实现要求 (R46, R57, R111)	R46/R57 允许调用厂商 IP；R111 明确澄清 MAC IP 无强制使用要求，允许参赛队自行设计	采用纯 RTL 自研双网一体化 MAC（第 5.6 节），单状态机双网同拍起始，空闲发送延迟为 0 拍，消除了官方 IP 的 59 拍起发延迟与额外 FIFO 资源
有效载荷吞吐 (R95-1)	接收端去重后的有效载荷吞吐率须达到 100 Mbps，冗余副本只计一次，需千兆 GMII 接口	采用八位 125 MHz 千兆 GMII，双网线速 1 Gbps；定长满载有效载荷吞吐率达 906.136 Mbps，变长高负载工况达 904.752 Mbps（表 10-1）
时钟与复位 (R3, R59)	频率与接口速率匹配，单时钟统一驱动；同步复位	全设计采用单一 125 MHz 全局时钟（周期 8 ns），GMII 每拍传输一字节；管理面复杂状态机采用 CE_DIV = 2 多周期使能树解耦
端到端分段时延 (R96, R97-1)	单帧 GMII 时延不超过 2 ms；明确定义分段时延：L_{tx}（提交末字节至 SFD）、L_{gmii}（网络传输）与 L_{rx}（SFD 至交付）	严格按 R97-1 边界测量：满载最大端到端时延为 24.432 µs（L_{tx} 最大 0.312 µs / 首前导码 0.256 µs；L_{rx} 首字节 12.360 µs / 末字节 24.120 µs），远低于 2 ms 门限
传输抖动口径 (R97-2)	明确定义抖动为同一条 VL 多帧端到端时延的最大值减最小值（极差），要求计算各 VL 极差并取最大值	稳态发送间隔抖动测量为 0 µs；严格按 R97-2 口径，定长满载同 VL 时延极差仅为 0.008 µs，变长循环多 VL 工况下极差为 9.992 µs
目标器件与考核 (R30, R112)	器件自主选择；允许以仿真结果截图和仿真演示视频作为核心考核依据	目标器件选用中科亿海微 EQ6HL130（CSG484_H）；建立包含 39 项回归与纯 ModelSim 评委入口的完备仿真验证闭环
协议版本依据 (R12, R47, R48)	协议版本未被强制限定，功能参考国际主流航空电子规范	严格依据 ARINC 664 P7（2005 版）、ARINC 615A-2（2002 版）、RFC 1157/3416（SNMPv1/v2c）与 RFC 4330（SNTPv4）

2.3 设计规格

表 2-2 列出细化后的设计规格，并按功能（F）、性能（P）、接口（I）和实现约束（C）编号。

表 2-2 设计规格

编号	规格	内容
F1	用户数据发送	每个端系统支持 128 条发送 VL，每条均可独立配置 BAG（以时钟周期计，不超过 2^{30}）、VLID、源/目的 IP 与端口、最大载荷和队列深度；消息长度为 1–1471 字节；按 VL 配置选择 A 网、B 网或双网发送

接下页

表 2-2 （续）

编号	规格	内容
F2	用户数据接收	支持 4096 条接收 VL，可映射任意 16 位 VLID；检查前导码、FCS、帧长、IP 头校验和、UDP 长度与校验和；两网分别进行完整性检查；启用冗余管理且副本到达间隔在 `skew_max` 内时，同一消息只交付一次
F3	SNMP 网络管理	SNMPv1/v2c 的 Get、GetNext、Set 和 SNMPv2-Trap；36 个管理对象；Community（`community`）校验
F4	615A 软件数据加载	基于 TFTP 的单文件上传与介质定义下载（Media-Defined Download, MDD）；LUH 头文件与数据文件 CRC-16 硬件校验；超时重传机制
F5	RTC 时钟同步	SNTPv4 纳秒级时钟同步；32 位秒加 32 位纳秒时间戳；应答报文回显客户端 Transmit 时间戳
P1	吞吐	每方向有效载荷不低于 100 Mbps（定长满载实测 906.136 Mbps，变长高负载实测 904.752 Mbps）
P2	端到端时延	不超过 2000 µs（满载稳态最大端到端时延实测为 24.432 µs）
P3	传输抖动	发送间隔抖动实测为 0 µs；同 VL 端到端时延极差定长满载实测仅 0.008 µs、变长循环为 9.992 µs
P4	帧格式	帧长 64–1518 字节（含 FCS）；复位后首帧 SN 为 0，之后按 1–255 循环
P5	容量	128 条发送 VL、4096 条接收 VL 全部可配置并可同时启用
I1	官方端口	`AFDX_top` 的 8 个端口名称、方向和位宽保持完全兼容，承载 A 网 GMII 物理接口
I2	线路接口	A、B 两路八位 GMII，每拍一字节，帧间隔不少于 12 拍
I3	扩展接口	独立的应用发送/接收流接口、32 位字寻址配置总线、统计与事件脉冲端口
C1	时钟与物理时序	单一 125 MHz 全局时钟，周期 8 ns；管理面协议状态机采用 CE_DIV = 2 多周期解耦；eLinx 布局布线后静态时序收敛（建立裕量 +0.013 ns、保持裕量 +0.229 ns，最高频率 125.203 MHz，0 违例端点）
C2	编码规范	信号小写、参数大写；每个模块的文件开头说明其作用、接口和关键时序

2.4 协议依据

表 2-3 协议依据与版本

功能	依据	实现范围
AFDX 网络	ARINC 664 Part 7（2005-06-27）	帧格式、VL 与 BAG、完整性检查、冗余管理
数据加载	ARINC 615A-2（2002-05-10）、ARINC 665-1、RFC 1350	单文件上传与介质定义下载（MDD）；协议文件版本 A4，向下兼容 A3
网络管理	RFC 1157、RFC 3416、RFC 3418	SNMPv1/v2c，BER 编码
时钟同步	RFC 4330	SNTPv4 48 字节报文

2.5 规格与设计章节对应

表 2-4　规格与设计章节对应

规格	设计章节	规格	设计章节
F1	5.3–5.6	P4	4.3
F2	6.2–6.5	P5	5.1、6.1
F3	7.3	I1–I3	第 4 章
F4	7.4	C1	第 3.4 节、第 9 章
F5	7.2	C2	第 9 章、附录 A.1
P1–P3	3.7、5.4		

第 3 章　总体架构

3.1　系统框图

图 3-1 是 AFDX_top 的系统框图。端系统由八个一级模块组成：发送核心和接收核心构成线速数据通路；应用口与 RTC、SNMP、615A 三个服务通过服务通道共用一对收发核心；配置模块提供寄存器、统计计数器和三张配置表的访问接口；时间基准为整个设计提供自由运行的周期计数。各模块的分工见表 3-1。

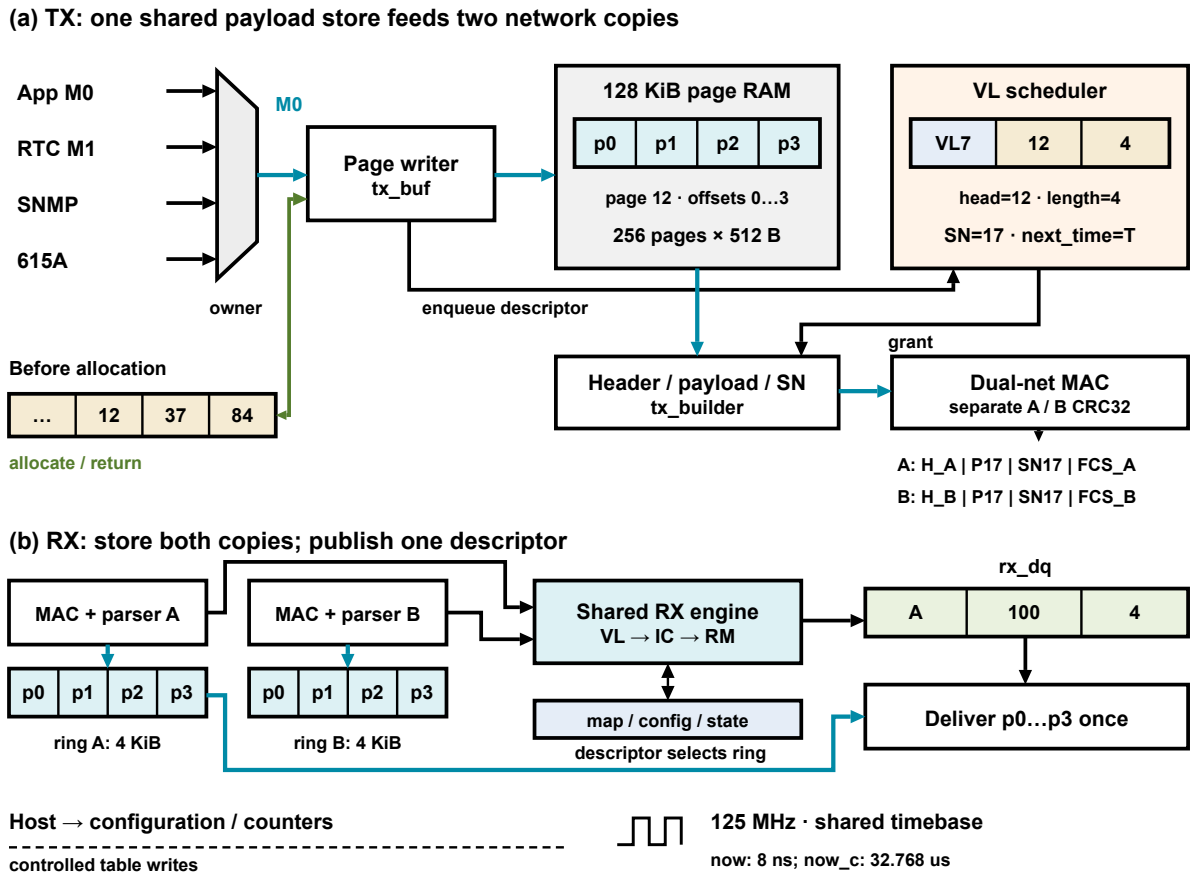


图 3-1　AFDX_top 的共享存储与双网数据通路。示例消息的载荷仅存一份，由描述符授权组帧并发往已启用的两网；接收侧载荷预写在本网环形缓存中，共享引擎只发布获准交付的描述符。

表 3-1　一级模块分工

模块	职责
afdx_tx_core	发送核心：消息分页缓存、128 条 VL 的 BAG 调度、组帧、双网 MAC（第 5 章）
afdx_rx_core	接收核心：双网 MAC、帧头解析、完整性检查、冗余管理、交付（第 6 章）
afdx_svc	服务通道：应用与三个服务按整条消息轮询发送；按 UDP 目的端口将接收消息送至对应服务或应用
afdx_rtc	SNTPv4 纳秒级时钟同步（第 7.2 节）
afdx_snmp	SNMP 代理（第 7.3 节）

接下页

表 3-1 （续）

模块	职责
afdx_615a	615A 软件数据加载目标端（第 7.4 节）
afdx_cfg	地址译码、寄存器、27 个统计计数器、配置表写入握手（第 8 章）
afdx_timebase	32 位周期计数 now 与粗粒度计时 now_c

3.2 模块层次与 24 个 RTL 模块划分

AFDX_top 顶层下共划分为 24 个 Verilog RTL 模块（源码位于 rtl/ 目录），模块归属、层次关系与片上 RAM 实例数量见表 3-2（各源文件的具体职责见附录 A.1）。全设计共包含 21 张逻辑存储表，在物理实现上封装为 52 个 afdx_ram_sdp 简单双端口 RAM 模块实例（经综合工具映射为 366 个 EBR 物理原语；其中发送核心 7 个、接收核心 37 个、SNMP 2 个、615A 6 个）。

为兼顾管理面协议状态机复杂度与 125 MHz 时钟约束，设计对 SNMP 与 615A 实施了引擎分离：

1. **afdx_snmp.v** 与 **afdx_snmp_engine.v**：afdx_snmp.v 作为顶层封装，负责 UDP 报文捕获、配置总线路由、MIB/Trap RAM 访问控制以及 2 分频局部时钟使能（CE_DIV=2）复制树驱动；ASN.1/BER 解码、VarBind 查找与应答组装核心状态机位于 afdx_snmp_engine.v，运行于 16 ns 时序预算的 2 周期多周期使能域。
2. **afdx_615a.v** 与 **afdx_615a_engine.v**：afdx_615a.v 负责 TFTP 端口分发、超时计数器与 6 块协议 RAM（含第 6 块 denial_ram）的物理接口；数据加载协议主状态机、LUH 头文件与数据文件 CRC-16 硬件校验位于 afdx_615a_engine.v，约束在 16 ns 多周期路径中。
3. **afdx_mgmt_tx.v**：管理面流式预取发送适配器，由 SNMP 与 615A 独立实例化复用。该模块运行于 125 MHz 主时钟下，在引擎提交报文后直接从协议 RAM 中流式读出数据并适配为底层的发送握手，避免额外复制大容量帧缓冲，解耦管理面多周期引擎与底层发送仲裁器的时序关联。

表 3-2 24 个 RTL 模块分工与下级层次

所属域与模块	子模块及下级关系	**RAM** 物理数
afdx_timebase	周期计数 now[31:0] 与粗粒度计时 now_c[15:0]	0
afdx_cfg	寄存器译码、27 个统计计数器两级流水更新与配置写握手	0
afdx_svc	应用与服务 4 路轮询发送仲裁（寄存输出）与端口分发	0
afdx_tx_core	afdx_tx_buf（写口寄存）、afdx_tx_engine、afdx_tx_builder afdx_tx_free（↪ afdx_ram_sdp: tx_free） afdx_mac_tx（↪ afdx_crc32_d8 ×2）、1 级流缓冲结构 afdx_ram_sdp ×6: tx_cfg、tx_state、tx_desc、 tx_mnext、tx_pnext、tx_payload	7

接下页

表 3-2 （续）

所属域与模块	子模块及下级关系	**RAM** 物理数
afdx_rx_core	afdx_mac_rx ×2（每个含 1 个 afdx_crc32_d8） afdx_rx_parse ×2、afdx_rx_engine、afdx_rx_deliver rx_map（划分为 32 个 2048 × 13 子库，afdx_ram_sdp ×32） rx_dq（迁入块 RAM，afdx_ram_sdp ×1，32×68 bit） afdx_ram_sdp ×4：rx_cfg、rx_state、rx_ring_a、rx_ring_b	37
afdx_rtc	afdx_ntp_fraction ×3（NTP 小数换算单元）	0
afdx_snmp	afdx_snmp_engine（ASN.1/BER 核心，CE_DIV=2 多周期域） afdx_mgmt_tx（125 MHz 流式预取发送适配器） afdx_ram_sdp ×2：request_ram、response_ram	2
afdx_615a	afdx_615a_engine（数据加载状态机与 CRC16，CE_DIV=2 域） afdx_ram_sdp ×6：packet_rx、packet_tx、lur_ram、 header_ram、load_ram、denial_ram（拒绝原因表）	6
全设计合计：**24** 个 **RTL** 模块，**21** 张逻辑表		**52**

3.3 关键设计决策

表 3-3 关键设计决策

设计问题	候选方案	选择	理由
时钟结构	双网各自时钟域；统一 125 MHz 单时钟	单时钟	GMII 接口线速为 1 Gbit/s，载荷容量为 956.44 Mbps，满足赛题 100 Mbps 的要求；核心内部收发与控制逻辑在同一时钟域协同运行
线路接口	模板端口传输 GMII 帧（含前导码、SFD、FCS）；传输 MAC 用户侧数据	GMII 帧	与 GMII 的逐字节定义一致
MAC 实现	eLinx 官方 MAC IP；自研	自研	直接控制前导码、FCS 和帧间隔；输出寄存器直接驱动 GMII，空闲发送延迟为 0 拍
双网 MAC	双 MAC 独立发送；单 MAC 同时驱动两网	单 MAC 驱动两网	两网副本同拍起始，节省一套 MAC；单网 VL 也按整帧占用双网发送时隙（第 5.6 节）
VL 状态存储	寄存器阵列；块 RAM 状态表	块 RAM 状态表	寄存器用量随 VL 数量线性增长；采用块 RAM 后，由对应引擎按固定流水读写，配置写入须等待访问许可
VLID 映射	全地址直接查表；哈希加 CAM；限制 VLID 范围	直接查表	查表固定耗时 2 拍，无地址冲突；需约 0.85 Mbit 存储（65536×13 bit），可由片上存储提供

接下页

表 3-3 （续）

设计问题	候选方案	选择	理由
发送调度	轮询扫描；最早到期优先；预取窗口扫描	预取窗口扫描	每拍至多一级比较；512 拍窗口为扫描回访与组帧留出时间，无争用时可在允许时刻启动发送
接收写入	整帧检查后写入；边接收边预写入	先预写入后提交	载荷边接收边写入环形缓冲，检查失败时回退预写入指针，释放本帧占用的空间
服务发送	服务独立发送通路；按消息轮询共享发送核心	共享发送核心	服务报文与用户报文遵守同一 BAG 调度；无服务报文时应用直通，零附加延迟
服务实现	软核处理器；硬件状态机	硬件状态机	协议解析与报文组装均由硬件状态机完成；RTC 时间戳在报文握手当拍采样
服务状态序列	各分支独立展开；公共状态序列加返回状态寄存器	公共状态序列	SNMP 和 615A 复用取字节、整数拼接与报文输出过程，统一同步 RAM 的访问时序

3.4 时钟与复位

端系统逻辑采用单一主时钟 `clk`，工作频率 125 MHz，周期 8 ns，所有内部触发器均在 `clk` 上升沿同步更新。`reset` 为高有效的同步复位，外部须保证其释放与 `clk` 上升沿严格同步；板级实现时由物理复位同步器提供。

为满足航电高层管理协议（SNMP 与 ARINC 615A）在 125 MHz 主频下的静态时序收敛要求，系统在统一主时钟树下设置了管理面局部时钟使能结构：

1. 多周期引擎结构（**CE_DIV** $= 2$）：在 `afdx_snmp` 与 `afdx_615a` 顶层生成每两拍有效一次的时钟使能信号 `ce`，并通过多级低扇出寄存器复制树（`CE_GROUPS`）驱动核心引擎内部逻辑。全工程共有 2,755 个触发器约束为 2 周期建立路径（16 ns 建立时间预算），大幅缓解深层协议分支的组合时序压力。
2. 单周期外围与接口隔离：以太网报文接收捕获、双端口协议 RAM 写口、32 位配置总线、RTC 纳秒时钟基准以及用于流式预取的 `afdx_mgmt_tx` 适配模块均保持在 125 MHz 单周期全速运行。这种结构既避免了异步跨时钟域（CDC）带来的亚稳态风险与同步 FIFO 开销，又使关键控制与存储接口维持周期级确定性。

核心顶层采用与 `clk` 同步的 GMII 字节流接口。连接外部物理 PHY 时，接收数据随 PHY 恢复的 RX_CLK 变化，发送侧需向外输出 GTX_CLK；因此板级外围需在接收引脚与核心之间设置帧级弹性缓冲，将 RX_CLK 域字节流同步至 `clk` 域，发送侧则由 `clk` 经输出寄存器驱动 GTX_CLK。

复位结束后，发送核心清零 `tx_state`、`tx_cfg`，并将 256 个页号写入空闲页池；接收核心清零 `rx_map`、`rx_cfg` 和 `rx_state`。深度最大的 `rx_map` 物理划分为 32 个子库并行清零，配合初始化定时器计数需 65536 拍。两个核心的 `init_done` 经两级流水打拍并在顶层相与锁存输出为 `tx_ready`，从解除复位到 `tx_ready` 为 1 共 65539 拍，即 524.312 µs。

表初始化完成之前，`cfg_ready` 保持 0、配置写不被接受，凡在此期间开始接收的以太网帧均整帧忽略。

3.5 时间基准

`afdx_timebase` 为全设计产生三个信号：

- `now[31:0]`：自由运行的周期计数，复位为 0，每拍加 1，约 34.36 s 回绕。帧起点以 1 拍（8 ns）为粒度，所有时间戳的先后比较均通过作差并检查符号位实现，因此 BAG 可配置为不超过 2^{30}（约 8.59 s）的任意整数拍（例如 100003 拍）。
- `now_c[15:0]`：粗粒度计时，等于 `now[27:12]`，每 4096 拍（32.768 µs）加 1，仅用于接收侧的冗余管理。
- `byte_en`：字节使能，复位期间为 0，此后恒为 1，即每拍都是字节边界。

3.6 端到端数据通路

图 3-2 给出一条用户消息从发送端应用到接收端应用的完整路径。发送侧依次经过分页缓存、调度引擎、组帧器和双网 MAC；两份帧经 A、B 网到达接收端后，各自经 MAC 和解析器检查，由接收引擎做完整性检查和冗余管理，最后由交付模块逐字节送给应用。

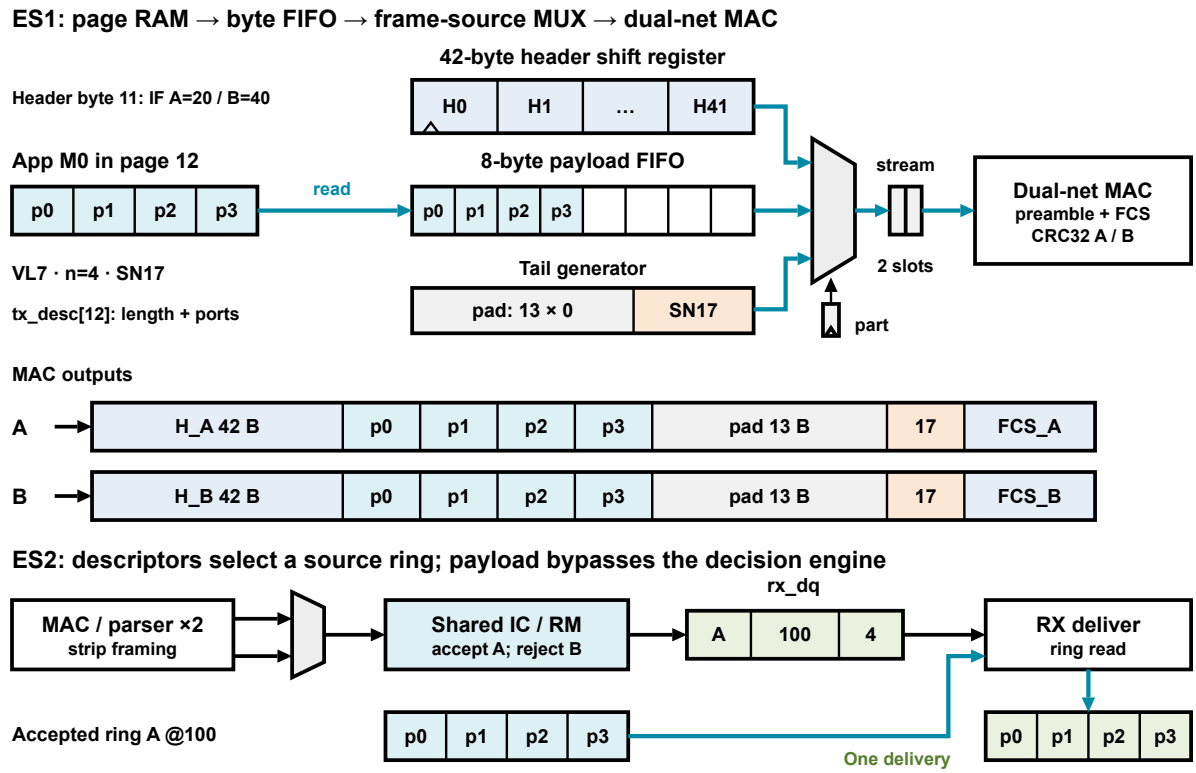


图 3-2 一条四字节消息的端到端表示。ES1 将同一组 p0–p3 加上 42 字节帧头、13 字节补零、SN 与 FCS 后发往两网；ES2 去掉帧封装并交付一次。示例采用 VL 7、SN 17，两帧均正确、IC/RM 启用、处于冗余窗口内且交付队列有空位，A 描述先被处理

帧内数据通路每拍处理一个字节，对应 1 Gbit/s。发送侧先把应用消息写入分页缓存，满足调度条件后，一边读取载荷，一边生成以太网帧；接收侧将到达的载荷预写入环形缓冲，帧检查通过后再提交给交付模块。各段的拍数和上界在第 3.7 节推导。

3.7 设计预算

本节由各模块的状态机拍数推导吞吐、时延、抖动、缓存和握手等待的设计值，作为第 5–8 章模块设计的约束。拍数均以 125 MHz 时钟计，1 拍 = 8 ns；仿真实测结果见测试报告。

本节及第 5–8 章以相邻两级输出有效时刻之间的周期差作为阶段延迟。状态机从 IDLE 态响应请求并启动操作需要 1 个周期，因此阶段延迟比操作本身的执行时间多 1 拍。例如，调度引擎的入队阶段共耗时 6 拍，其中响应请求需 1 拍，执行 ENQ 操作需 5 拍。

3.7.1 吞吐

载荷为 n 字节的帧连同 8 字节前导码、4 字节 FCS 和 12 拍帧间隔，在线路上占用的时长为

$$C(n) = 8 + 47 + \max(n, 17) + 12 = 67 + \max(n, 17) \text{ 拍}. \tag{3-1}$$

满载帧 $n = 1471$ 占 $C = 1538$ 拍，最短帧 $n \le 17$ 占 84 拍。每网每方向的载荷容量为

$$R_{\max} = \frac{1471 \times 8 \text{ bit}}{1538 \times 8 \text{ ns}} = 956.44 \text{ Mbps}. \tag{3-2}$$

设 BAG 为 B 拍，N 条 VL 各按该间隔发送满载帧时，总载荷率为 $N \times 1471 \times 8 \text{ bit}/(B \times 8 \text{ ns})$，可调度条件为 $N \times C \le B$。BAG 为 1 ms 时 $B = 125000$，$N \le \lfloor 125000/1538 \rfloor = 81$，此时载荷率 953.208 Mbps；77 条 VL 为 906.136 Mbps，线路占用率为 94.7%。

线速通路上各环节的吞吐预算如下：

- 应用消息接收：`afdx_tx_buf` 正常每拍接收一字节，每次申请新缓存页暂停 6 拍；消息接收结束后，写入描述符及等待入队应答共暂停 10 拍。一条占 3 页的 1471 字节消息在应用口共耗时 $1471 + 3 \times 6 + 10 = 1499$ 拍，小于该帧占用线路的 1538 拍。
- 授权与准备：引擎完成一帧授权需要 9 拍，组帧器准备帧头需要 11 拍，合计 20 拍，在当前帧发送期间完成，小于最短帧的 84 拍时隙。
- 接收交付：交付口每拍一字节；单网到达的载荷率至多 $1471/1538$ 字节每拍，低于交付速率。

3.7.2 端到端时延

端到端时延的起止点定义见第 2 章。下文按收发通路无其他消息争用、当前 VL 已满足 BAG 发送间隔的基准条件，给出各区段的典型拍数和设计上界（表 3-4），发送侧和接收侧的逐拍时序见图 3-3。

图 3-3(a) 中，消息入队时直接成为候选。若末字节在 RECV 状态下正常接收，且无需在消息尾部申请新页，由于载荷写口增加了 1 拍流水寄存、核心输出端插入了 1 级流

缓冲（双缓冲结构），从接收末字节到输出首个前导码字节固定为 32 拍（0.256 µs）；按 1022 答疑 R95 规则计至以太网帧 SFD 结束（8 字节前导码尾），发送处理时延 L_{tx} 典型为 39 拍（0.312 µs）。单字节消息或末字节跨页时多约 6 拍。图 3-3(b) 给出单网单帧的接收过程，由于 `rx_map` 采用 32 子库多 Bank 分块存储增加 2 拍读回流水，从 GMII 最后一个 FCS 字节到首个交付字节由 24 拍更新为 26 拍（0.208 µs）。

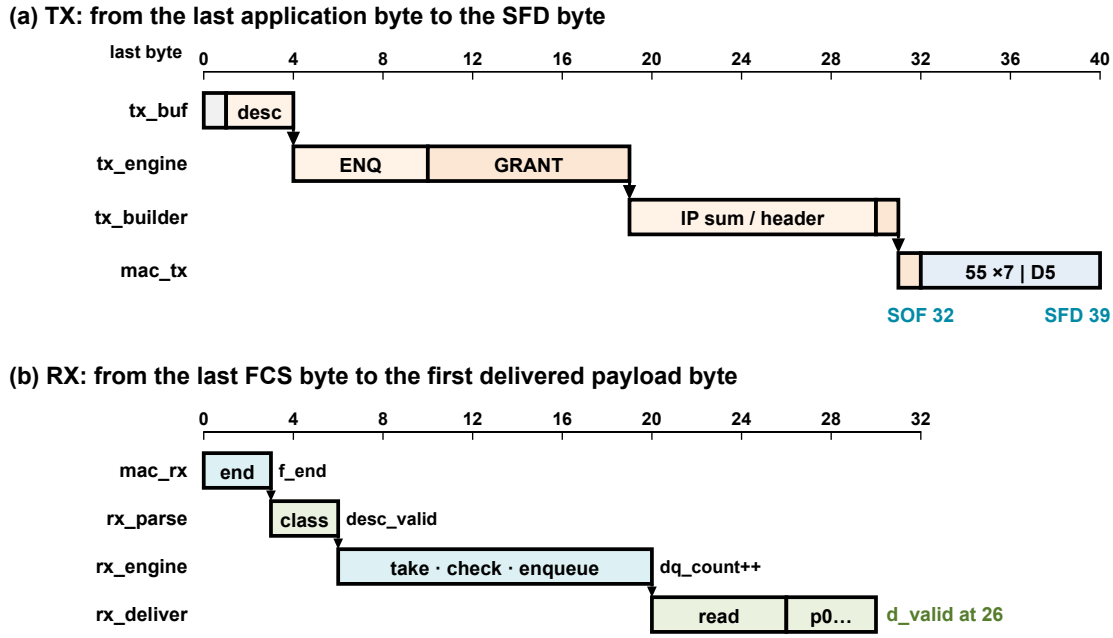


图 3-3 无其他消息争用、VL 已到期时的流水时序：(a) 发送准备与起始；(b) 接收检查与交付

表 3-4 端到端时延预算（拍）

区段	典型	上界	构成
发送侧至前导码	32	100	分页缓存写载荷寄存 1 拍、写描述符并请求入队 3 拍，引擎入队 6 拍，授权 9 拍，组帧准备 11 拍，输出流缓冲与 MAC 起始 2 拍；计至 SFD 结束（L_{tx}）为 39 拍
线路	1525	1525	前导码首字节至 FCS 末字节，共 $8+1518$ 字节（$n=1471$）
接收至首字节	26	200	MAC 检测帧尾 3 拍，解析器分类 3 拍，多 Bank 分块映射与引擎判定入队 14 拍，交付模块读块 RAM 队头并读取环形缓冲 6 拍；上界另计另一网描述与过期清理让出
逐字节交付	$n-1$	$n-1$	首字节到末字节，每拍一字节，$n=1471$ 时为 1470 拍
合计	3053	3295	24.424 µs（遇 AGE 扫描 1 拍为 3054 拍 / 24.432 µs）/ 26.360 µs

存在其他消息时，还须计入发送排队、提前授权占槽、线路占用与接收排队交付的等待；端到端时延的上界由各 VL 的 BAG、帧长和流量配置共同决定。

3.7.3 发送抖动

记 VL v 的帧允许时刻为 T_v（上一帧实际起始时刻加 BAG），实际起始时刻为 S_v，发送抖动为 $J_v = S_v - T_v$，即同一 VL 相邻两帧起始间隔与 BAG 之差。

与第 3.7.2 节考察消息到达时 VL 已到期的直通延迟不同，本节考察消息在允许时刻之前已提前排队时的准点发送精度。当线路和待发槽位均未被其他帧占用时，发送抖动为 0。调度器能否在 T 时刻准点启动发送，取决于候选能否在 T 之前完成扫描、授权和准备。一条 VL 在 $T-512$ 进入提前窗口，此后：

- 扫描回访：无候选时，引擎每 2 拍检查一条 VL，轮询 128 条共需 256 拍。每扫描 16 条 VL 至多插入一次入队操作（6 拍），每发送一帧还需插入一次发送起始确认（TX Start Confirm）（4 拍）；每次插入操作都会清空 5 级扫描流水并重新发出（约 6 拍，见第 5.4 节），折合每次入队约 12 拍、每次发送起始确认（TX Start Confirm）约 10 拍。计入这些开销后，任一 VL 相邻两次被检查的间隔约为 400 拍。
- 授权与准备：授权 9 拍、组帧器准备 11 拍、起始判断 2 拍，共 22 拍。

$400+22=422<512$，留有 90 拍裕量。因此在上述条件下，MAC 在 T 时刻准点输出前导码首字节。

线路争用时，物理链路同一时刻只能发送一帧，组帧器也仅设一个待发槽位。v 到期且消息已排队时，可能因以下三种原因推迟发送：

- 线路占用：排在 v 前面的帧逐个发完，每帧线路占用 $C(n_u)=67+\max(n_u,17)$ 拍。
- 提前授权占槽等待：扫描按 VL 索引轮转，选中首条已到期或将在 512 拍内到期的候选 VL。若扫描先遇到一条尚未到期的 VL u 并发出授权，组帧器的待发槽位将被 u 占用，直到 T_u 时刻才启动发送，期间 v 无法获得授权。每次此类提前授权引入的占槽等待为 $w_u<512$ 拍。
- 轮询扫描延迟：扫描从上次选中位置继续向后轮询，在遍历到 v 之前所选中的 VL 均先于 v 发送；扫描与这些帧的发送重叠进行，扫描范围不超过一圈，连同 v 自身的授权和准备不超过约 422 拍。

记 Q_v 为在 v 之前发送的其他 VL 帧的集合，则

$$J_v \le \Big(\sum_{u\in Q_v} \big(C(n_u)+w_u\big) + 422 \Big) \times 8 \text{ ns}, \tag{3-3}$$

其中 w_u 只对在自身允许时刻之前被授权的帧非零。服务报文各走自己的 VL，同样按 BAG 调度，已包含在 Q_v 中。

一次争用只造成一次相位后移。下一次允许时刻按实际起始时刻加 BAG 计算（第 5.4 节）。多条 VL 发生发送争用后，后发 VL 的相位后移，此后各自按 BAG 更新允许时刻。要持续保持错开的相位，需满足两个条件：

- 各 VL 的 BAG 互为整数倍。ARINC 664 P7 规定 BAG 取 2^k ms，满足这一点；任取整数拍的 BAG 会使相位相互漂移，周期性地再次发生争用。
- VL 在允许时刻之前已有消息排队。计时到期时若队列为空，`timer_active` 被清除，下一条消息到达即发，相位改由应用的提交时刻决定。

两个条件都满足时，争用只在启动时发生，稳定后同一 VL 相邻两帧的起始间隔恰为 BAG。

以 P3 的 50 µs 作为发送抖动的检查取值，按式 (3-3) 对应 $\sum_{u\in Q_v}(C(n_u)+w_u) \le 6250-422=5828$ 拍。例如：

- 同时到期的其他 VL 至多 3 条最长帧（$w_u = 0$）：$3 \times 1538 = 4614$ 拍，满足；
- 前面有 2 条最长帧且都被提前授权：$2 \times (1538 + 512) = 4100$ 拍，满足；
- 77 条最短帧同时到期：$76 \times 84 = 6384$ 拍，不满足。

同时参与争用的 VL 数量超出上述示例时，主机须错开各 VL 首条消息的提交时刻，使任一 VL 之前排队发送的帧满足该条件；在上述两个条件下，错开只需在启动时做一次（第 8.9 节）。

与 ARINC 664 P7 的抖动公式 $40\ \mu s + \sum(20 + L_{\max}) \times 8$ bit/1 Gbit/s 相比，$C(n) \times 8$ ns 恰等于公式中对应帧的线路占用时间；本设计的扫描开销 422 拍（3.4 µs）与至多 8 次提前授权等待（共 4096 拍）之和小于公式中的 40 µs 项。因此对满足 P7 公式、且排在前面的提前授权帧不超过 8 个的配置，式 (3-3) 给出的发送抖动上界不超过按 P7 抖动公式计算的值，也不超过 P7 规定的 500 µs 抖动上限。

3.7.4 存储预算与缓存规划

在存储架构规划中，设计严格解耦了“全量静态链路配置”与“动态在途流量缓冲深度”：

- 静态配置容量：硬件配置表与映射表完全支持 128 条发送 VL 与 4096 条接收 VL 的并发静态配置，支持任意 16 位 VLID 查表映射。
- 动态发送载荷缓存：发送侧片上配置 128 KiB 共享载荷存储池（划分为 256 页 × 512 字节），采用基于单向链表的动态页分配机制。单个最长报文（1471 字节）占用 $\lceil 1471/512 \rceil = 3$ 页，存储池最多可支持 85 帧满载大包同时在途排队。在 77 条与 81 条 VL 双向满载的基准测试工况下，各 VL 同时排队一条最长报文分别占用 231 页（占比 90.2%）与 243 页（占比 94.9%），完全覆盖满载无损调度需求；当突发流量导致空闲页暂时耗尽时，硬件等待固定上限拍数后执行安全丢弃并置位统计脉冲，确保系统免于死锁。
- 接收端环形缓冲与描述队列：A、B 网各设 4096 字节接收环形缓冲，解析器基于已提交指针实时计算剩余容量，可安全容纳连续两个最长报文（2×1471 字节有效载荷）。交付描述队列 `rx_dq` 设为 32 项深度；在最长消息交付持续 1471 拍期间，单网线速到达的最短帧至多为 $\lceil 1471/84 \rceil = 18$ 个，双网冗余副本经 RM 去重后只交付一份，32 项深度具备充裕的突发吸收裕量。

3.7.5 握手等待

表 3-5 握手等待与初始化预算

接口或操作	等待上界与计算依据
入队应答	引擎一次只执行一种操作，其中最长的授权操作需 9 拍。每扫描满 16 次或已有候选时，可插入入队操作；入队应答等待不超过 64 拍。
`m_ready` 反压	取页暂停 6 拍；消息末尾写描述符、入队应答、页面释放（至多 3 页，每页 3 拍）和事件脉冲共暂停 $2 + 64 + 9 + 1 = 76$ 拍，连续反压上界取 96 拍。

接下页

表 3-5　（续）

接口或操作	等待上界与计算依据
应用口等待	`app_tx_ready` 受服务通道轮询仲裁控制（第 7.1 节）。三个服务均有报文待发时，应用至多等待其各发送一条：RTC 报文 48 字节、SNMP 响应不超过 256 字节、615A 报文不超过 516 字节；经管理发送适配器流式预取后，字节发送以线速每拍 1 字节推进，连同预取启动与入队等待，握手等待上界约为 1100 拍（约 8.8 μs）。
配置写	向配置表写入时，需等待对应收发核心通过 `cfg_ok` 给出许可。发送引擎最坏连续执行发送起始确认、入队和授权约 25 拍，接收引擎处理两网各一份描述需至多 30 拍，若中断 AGE 清理则另加 1 拍，等待上界取 64 拍（第 8.4 节）。
初始化	最大的 `rx_map` 逻辑容量 65536 项，硬件划分为 32 个子库并行清零（2048 拍），配合全局清表定时器计数及 `tx_ready` 跨级输出寄存，从解除复位至 `tx_ready` 有效共耗时 65539 拍（524.312 μs）。

服务通道按整条消息轮询，限制了服务连续占用消息通路的时间，表中的约 8.8 μs 是提交口的握手等待预算。入队后，用户与服务报文按各自 VL 的 BAG 参与调度，共享组帧器和发送 MAC；服务 VL 的线路占用与提前授权等待统一计入式 (3-3) 及第 8.9 节的配置预算。

第 4 章　接口设计

4.1　顶层端口

AFDX_top 保留选题模板的 8 个端口（名称、方向和位宽不变），并按 AFDX 双网冗余与应用接口需求扩展 B 网、应用、配置和事件端口。顶层共声明 40 个端口，总位宽为 259 位，少于 EQ6HL130 可提供的 339 个用户 I/O。端口分组见表 4-1。

表 4-1　顶层端口

端口	方向	含义
选题模板端口		
clk	入	125 MHz 时钟
reset	入	同步复位，高有效
tx_data[7:0]、tx_valid	出	A 网 GMII 发送，相当于 TXD、TX_EN
rx_data[7:0]、rx_valid	入	A 网 GMII 接收，相当于 RXD、RX_DV
tx_ready	出	收发核心初始化完成
rx_ready	入	不使用
冗余网络		
tx_data_b[7:0]、tx_valid_b、rx_data_b[7:0]、rx_valid_b	出/入	B 网，含义同 A 网
应用发送		
app_tx_valid、app_tx_ready	入/出	字节握手
app_tx_data[7:0]、app_tx_last	入	消息字节；last 标记末字节
app_tx_vl[6:0]	入	发送 VL 索引，在首字节锁存
app_tx_port_en、app_tx_sport[15:0]、app_tx_dport[15:0]	入	app_tx_port_en 为 1 时使用消息自带的 UDP 端口，否则使用 VL 的默认端口；在首字节锁存
应用接收		
app_rx_valid、app_rx_data[7:0]、app_rx_last	出	交付字节，不带反压
app_rx_vl[11:0]、app_rx_sport[15:0]、app_rx_port[15:0]、app_rx_net	出	接收 VL 索引、UDP 源端口、UDP 目的端口（app_rx_port）及来源网络；整条消息期间保持不变
配置与统计		
cfg_wr、cfg_rd、cfg_addr[17:0]、cfg_wdata[31:0]	入	配置总线请求
cfg_rdata[31:0]、cfg_rvalid、cfg_ready	出	读数据、读有效、写就绪

接下页

表 4-1 （续）

端口	方向	含义
ev_tx[6:0]	出	发送统计脉冲：accept、drop_dis、drop_full、drop_long、drop_nopage、tx_a、tx_b
ev_a[7:0]、ev_b[7:0]	出	各网接收统计脉冲，见表 6-3
ev_rm_drop、ev_deliver	出	冗余丢弃、交付
ev_gdrop[1:0]	出	保留端口，固定为 0

顶层参数 TX_PAGE_AW、TX_OFF_AW、LOOKAHEAD、RX_RING_AW、RX_DQ_AW 的默认值分别为 8、9、512、12、5，直接传递给收发核心。

4.1.1 顶层端口映射与网络分层定义

按照 ARINC 664 Part 7 标准物理分层与官方 1022 答疑澄清（R61、R95-2、R110）：

1. **A** 网物理接口：顶层模板的 8 个端口（clk、reset、tx_data、tx_valid、rx_data、rx_valid、tx_ready、rx_ready）对应 A 网 GMII 物理接口，传输包含前导码、SFD 与 FCS 的线速以太网字节流。
2. 双网冗余扩展：按标准双网冗余要求对称扩展 B 网 GMII 物理端口（tx/rx_data_b_*），支持单网独立收发与双网并行收发。
3. 应用层与配置接口：主机应用数据通过独立的流式消息接口 app_tx_*/app_rx_* 收发，端系统配置表与统计计数器通过 32 位字寻址总线 cfg_* 访问，硬件状态变化由 ev_* 脉冲输出。
4. **MAC** 实现与评测依据（**R111**、**R112**）：MAC 模块采用纯 RTL 自研实现（R111 允许参赛队自研 MAC）；功能仿真波形、日志与覆盖率记录作为设计验证的依据（R112）。

4.2 GMII 接口

A、B 两网各有一组八位 GMII，在 valid 为 1 的每拍传送一个字节。

- 发送：tx_valid 从前导码首字节至 FCS 末字节连续为 1，帧间至少空闲 12 拍，见图 4-1。GMII 输出直接来自 MAC 的输出寄存器，发送适配延迟为 0。若某条 VL 未使能其中一路网络，则该帧发送期间对应网络的 tx_valid 与 tx_data 全程保持为 0。
- 接收：rx_valid 的上升沿表示一帧开始，下降沿表示该帧结束。相邻两帧的低电平间隔最短为 1 拍。MAC 先寄存输入，再逐拍查找帧起始定界符（SFD）、累加 CRC 并剥离 FCS。
- 复位：复位期间输出为 0；对于表初始化完成前开始接收的帧，接收核心将整帧忽略（第 3.4 节）。
- 时钟与错误信号：接口采用全双工模式，收发字节均与 clk 同步，不含 TX_CLK、RX_CLK、TX_ER、RX_ER、CRS 和 COL。接收核心根据字节流检查前导码、帧长和 FCS；连接外部 PHY 时，外围适配逻辑须处理 PHY 错误标记并丢弃带错帧。收发时钟适配见第 3.4 节。

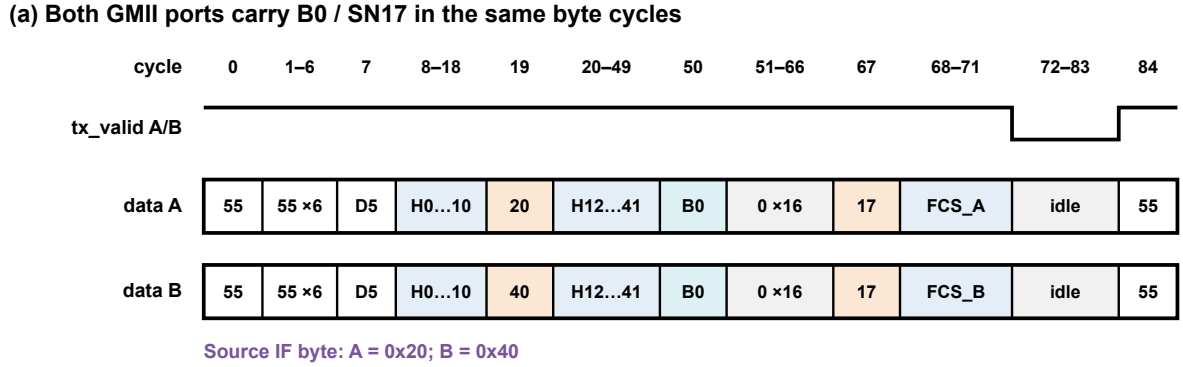


图 4-1 GMII 发送一帧的接口时序（1 字节消息，最短帧）。第 0 拍为前导码首字节，图中按标注的周期段压缩展示；示例载荷为 B0、SN 为 17，两网同拍起始，仅第 11 字节（接口号，A 为 `0x20`、B 为 `0x40`）和 FCS 不同；某网未启用时该网整帧保持为 0

4.3 以太网帧格式

图 4-2 给出了本设计发送的 AFDX 帧格式。目的 MAC 和源 MAC 按 ARINC 664 Part 7 的固定格式编码：目的 MAC 为 `03 00 00 00` 加 16 位 VLID，源 MAC 为 `02 00 00` 加 16 位端系统用户 ID 和 1 字节接口号（A 网 `0x20`、B 网 `0x40`）。其后依次为 IPv4 头、UDP 头和应用数据；应用数据不足 17 字节时补零至该长度，再附加 1 字节序列号 SN，最后由 MAC 添加 4 字节 FCS。

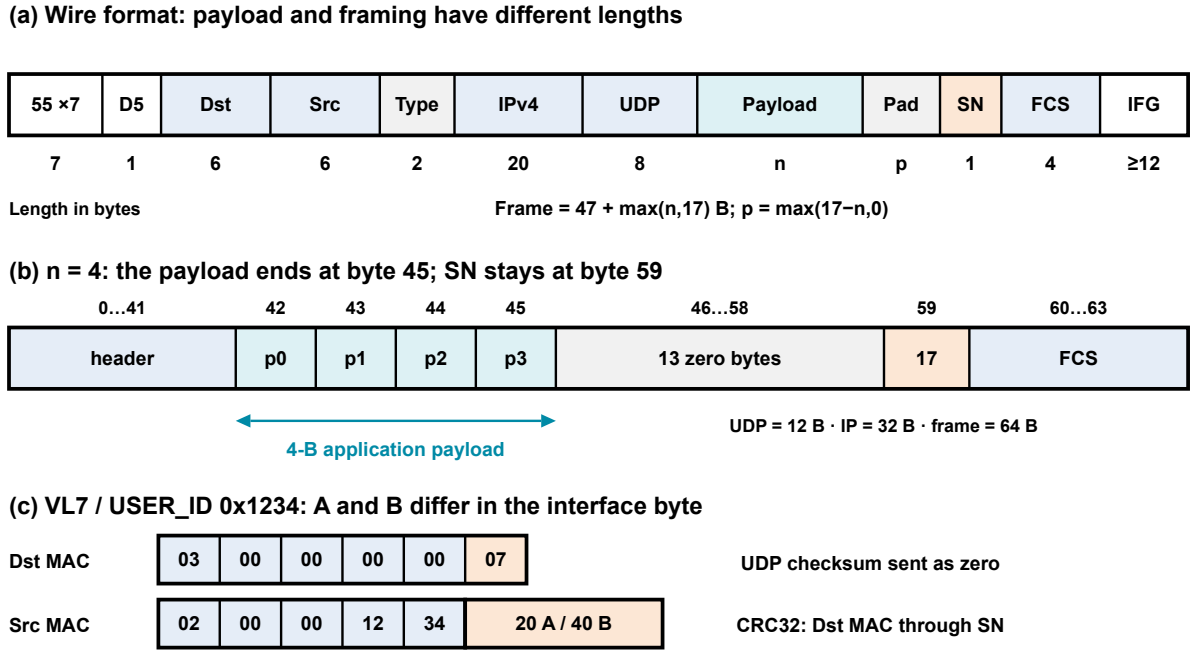


图 4-2 AFDX 帧格式与具体字节位置。应用数据长度 $1 \le n \le 1471$ 字节；示例 $n = 4$ 时，p0–p3 位于第 42–45 字节，补零 13 字节，SN 位于第 59 字节，含 FCS 的帧长为 64 字节。地址编码采用 VL 7 与 USER_ID=`0x1234`

表 4-2 帧字段（字节序列以十六进制表示）

字节位置	长度	内容
0–3	4	`03 00 00 00`，Part 7 规定的目的 MAC 固定部分
4–5	2	VLID，高字节在前
6–8	3	`02 00 00`，源 MAC 固定部分
9–10	2	端系统用户 ID（寄存器 `USER_ID`）
11	1	接口号：A 网 `0x20`，B 网 `0x40`
12–13	2	`08 00`（IPv4）
14–33	20	IPv4 头：`45 00`，总长度 $n+28$，标识（每帧加 1），`00 00`，TTL `0x01`，协议号 `0x11`（17），头校验和，源 IP，目的 IP
34–41	8	UDP 头：源端口，目的端口，长度 $n+8$，校验和（填 0）
42 起	n	应用数据
其后	$17-n$	$n<17$ 时补 0 至 17 字节
再后	1	序列号 SN
最后	4	FCS（CRC32，由 MAC 添加）

由此，含 FCS 的帧长为 $47+\max(n,17)$，范围 64–1518 字节；A、B 两份帧只有第 11 字节和 FCS 不同。复位后，各 VL 首帧的 SN 均为 0，之后按 1、2、…、255、1、… 循环。组播目的 IP 按 Part 7 图 3-38 配置为 `224.224.VLID[15:8].VLID[7:0]`，发送核心直接使用配置表中的目的 IP，因此单播服务（如 615A）也可共用该通路。

端系统以 VL 为独立调度与查表粒度，承载不分片的单包 UDP 消息：发送侧将 UDP 校验和置 0；接收侧直接以目的 MAC 低 16 位 VLID 查表定界并校验协议头字段，通过校验后将载荷与 UDP 端口一并交付上层应用或内部服务。

CRC32 采用反射多项式 `0xEDB88320`，初值为全 1，计算范围覆盖从目的 MAC 到 SN 的全部字节。结果取反后按低字节在前的顺序发送。收发通路均采用组合模块 `afdx_crc32_d8`，以 32 位当前 CRC 值和 8 位数据为输入，输出更新后的 CRC 值。

4.4 应用接口

4.4.1 发送握手与缓存等待

应用以字节流提交消息：`app_tx_valid` 与 `app_tx_ready` 同拍为 1 时传送一个字节，`app_tx_last` 标记末字节；VL 索引和 UDP 端口在首字节锁存。

没有服务报文争用时，`app_tx_ready` 直接连接发送核心的 `m_ready`。发送核心每拍接收一个字节，仅在分配缓存页或等待入队结果时暂停，连续暂停不超过 96 拍。服务报文传输期间，应用等待当前服务消息完成；三个服务同时有报文时，等待预算约为 3850 拍（第 3.7 节）。

缓存页用尽或 VL 队列已满时，发送核心丢弃该消息并计数，回收已分配的缓存页，随后继续接收并丢弃该消息的剩余字节。

图 4-3(a) 以无服务争用、长度为 $2\le n\le 512$ 字节且不跨页的消息为例：接收首字节后，分配缓存页需 6 拍；接收末字节后，写入消息描述并完成入队需 10 拍。`app_tx_ready`

为 0 时，应用保持当前字节及其有效标记，直至完成握手；图中的 C0 是下一条消息的首字节。单字节消息或末字节跨页时，末尾处理需额外增加约 6 拍。

4.4.2 接收消息的连续交付

接收时，一条消息的 n 个字节在连续 n 拍内送出，期间 `app_rx_valid` 保持为 1，`app_rx_last` 与末字节同拍。VL 索引、端口和来源网络在整条消息期间保持固定。接口时序见图 4-3(b)。

接收接口不提供反压，按固定字节节拍连续交付消息，因此交付通路无需处理下游停顿。外部应用可用缓存写口或 DMA/FIFO 接口接收数据，并应保证每个有效周期都能接收一个字节。

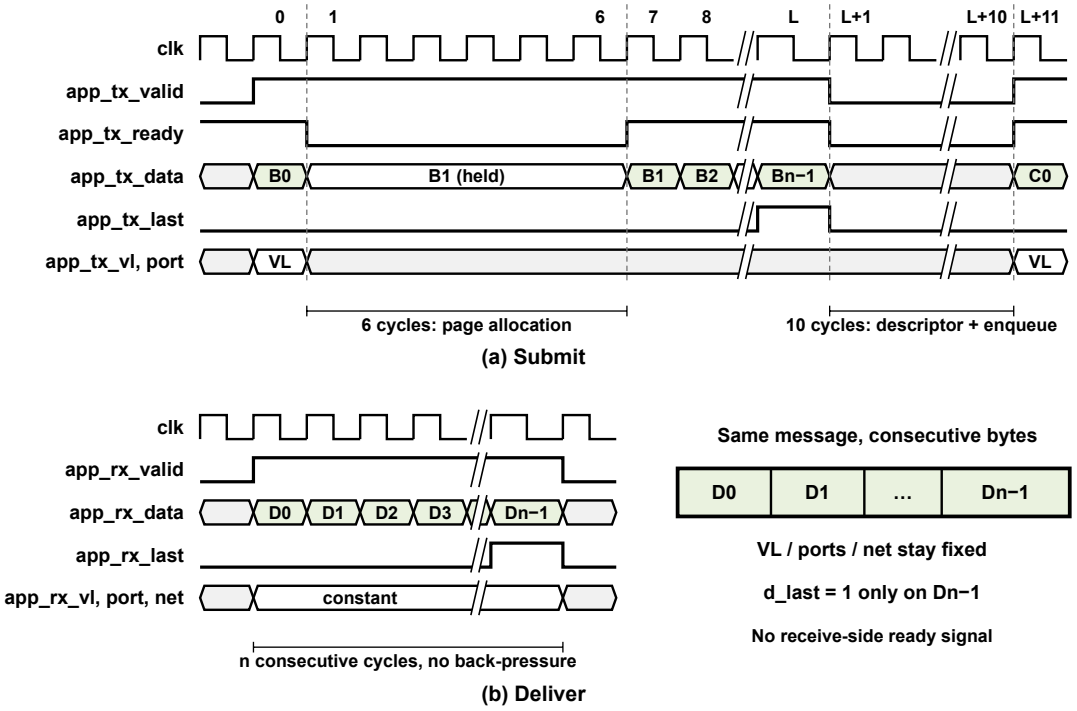


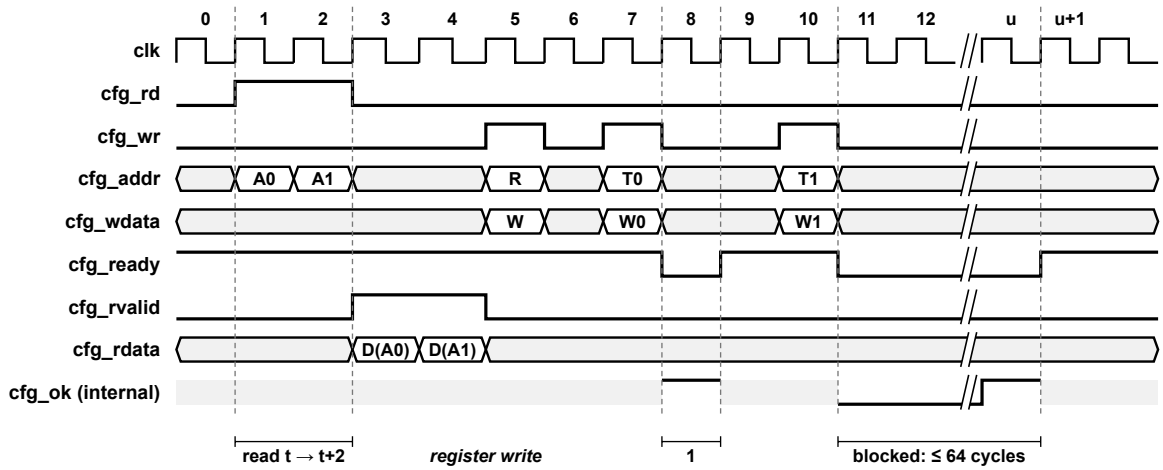
图 4-3 应用接口时序：(a) 无争用、空闲引擎及时接纳时的消息提交与握手示例；(b) 接收消息连续交付。示例中的 6 拍取页和 10 拍提交间隔不代表所有争用条件下的等待上界

4.5 配置总线

配置总线支持寄存器和三张配置表的写入，以及寄存器和计数器的读取，时序如下（t 为周期号，图 4-4）：

- 读：t 拍 `cfg_rd` 为 1，则 $t+2$ 拍 `cfg_rvalid` 为 1、`cfg_rdata` 有效。复位结束后可以每拍连续读。同拍读写同一寄存器时，读操作返回更新前的值。
- 写寄存器：`cfg_wr` 与 `cfg_ready` 同拍为 1 时，完成写操作。
- 写配置表：$t+1$ 拍起 `cfg_ready` 变 0，待写寄存器保持地址和数据，直到核心在某拍 u 给出 `cfg_ok` 时写入，$u+1$ 拍 `cfg_ready` 恢复；不受阻时 $u=t+1$，最长等待 64 拍（第 8.4 节）。

图 4-4 中，A、R、T 分别对应连续寄存器读、寄存器写和配置表写；寄存器写期间 cfg_ready 保持为 1。



RX table write: the same T1 / W1 survives the blocked interval

accepted at 10	pending while cfg_ok = 0	permit at u	ready at u+1
{T1, W1}	{T1, W1, enable=1}	write T1 ← W1	enable=0

图 4-4 配置总线的连续读、寄存器写与接收配置表写时序

4.6 事件接口

ev_* 端口在统计事件发生时输出单周期脉冲，与第 8 章的计数器一一对应，供外部逻辑监视各类事件。对于每条发送消息，发送核心均产生且仅产生 accept 或四类丢弃脉冲中的一种。对于表初始化完成后开始接收的每一帧，接收核心在对应网络上产生一个帧分类脉冲；若判定为正确帧，还会输出一个处理结果脉冲（第 6.6 节）。

4.7 内部接口

一级模块之间及核心内部子模块之间的接口以寄存器输出为主。ready 握手与 cfg_ok 采用组合输出；afdx_svc 的数据选择和接收分发也采用组合直通，仅包含选择与门控逻辑。内部接口主要采用以下四种约定：

- 字节流握手（valid/ready/last）：valid 与 ready 同拍为 1 时传送一个字节；附带字段在首字节锁存并保持到末字节。
- 请求应答（req/ack）：请求持续有效，直到接收到应答脉冲；应答与结果在同一拍输出。
- 单深缓冲握手（**Single-Slot Handshake**）（valid/take 或 valid/ready）：生产者置 valid 并保持字段不变，消费者接收后，生产者才能送入下一项。
- 写允许授权（cfg_ok）：表的读写控制模块通过组合信号给出写允许授权。许可有效时，核心寄存外部写请求，并在下一拍写入 RAM；该拍不再发出冲突的表读请求（第 8.4 节）。

表 4-3 汇总各内部接口，各信号的定义见附录 C。

表 4-3 内部接口汇总

接口	两端	约定	关键时序
消息 m_*	afdx_svc → tx_buf	字节流	每次分配缓存页时，m_ready 为低 6 拍；消息末尾为低约 10 拍；连续为低不超过 96 拍
入队 enq_*	tx_buf → tx_engine	请求应答	应答不晚于请求后 64 拍
授权 grant_*	tx_engine → tx_builder	单深缓冲握手	从当前帧 tx_sof 有效的当拍起，可接受下一帧授权
发送起始指示	tx_builder → tx_engine	单拍脉冲	回报首个前导字节输出当拍的 now
帧字节 s_*	tx_builder ↔ mac_tx	字节流	MAC 控制字节接收时刻，在前导码的最后一拍接收第 0 字节
帧描述 desc_*	rx_parse → rx_engine	单深缓冲握手	f_end 后第 3 拍给出；单网判定耗时 8–15 拍（含取描述与写回）
交付队列 rx_dq	rx_engine → rx_deliver	32 项队列	队列非空后的第 6 拍输出首个 d_valid
交付 d_*	rx_deliver → afdx_svc	字节流，无反压	服务在首字节当拍完成端口匹配
服务发送 svc_tx_*	三个服务 → afdx_svc	字节流	按整条消息轮询
配置表写	afdx_cfg → 收发核心	写允许授权	等待不超过 64 拍（第 8.4 节）

第 5 章　发送通路

本章沿数据流向介绍发送核心 `afdx_tx_core` 的五个部分：分页缓存 `afdx_tx_buf` 与空闲页池 `afdx_tx_free`、调度引擎 `afdx_tx_engine`、组帧器 `afdx_tx_builder` 和双网 MAC `afdx_mac_tx`。

5.1　发送核心与配置表

图 5-1 是发送核心的框图。应用和服务提交的消息先由 `afdx_tx_buf` 分页写入载荷 RAM，再加入发送队列。`afdx_tx_engine` 按 BAG 选出下一帧，向 `afdx_tx_builder` 发出授权。组帧器随后预读载荷、拼接帧头，并逐字节送入 `afdx_mac_tx`，由 MAC 发往两网。

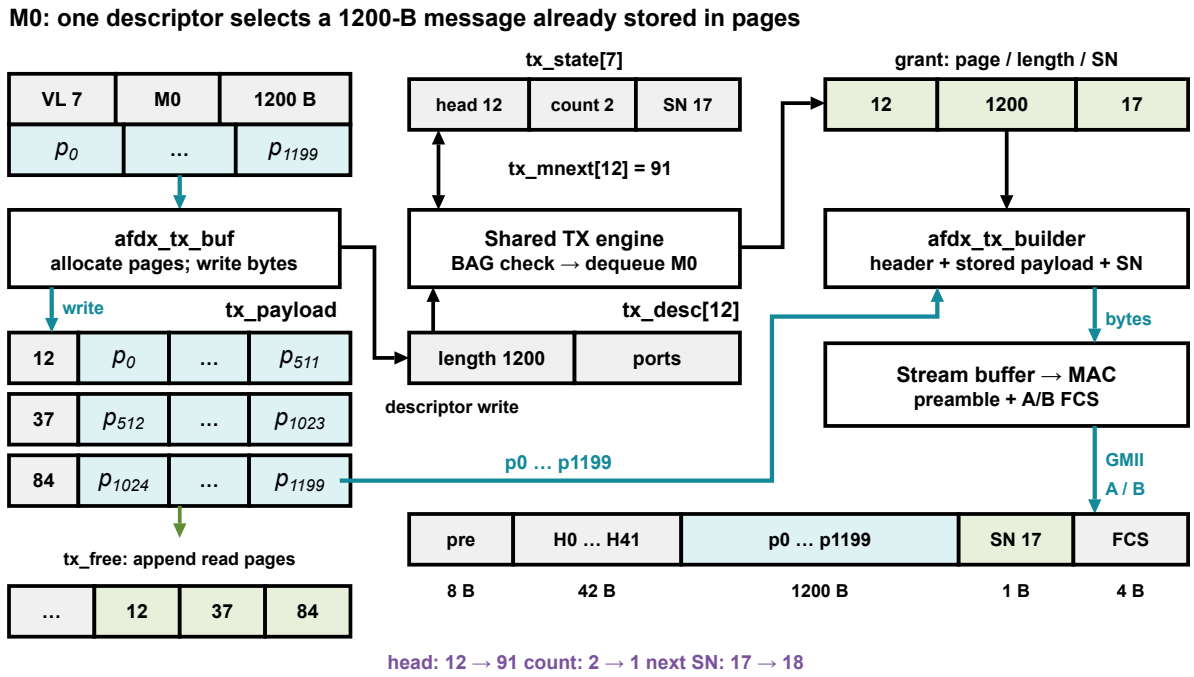


图 5-1　以 VL 7 的消息 M0 为例，1200 字节载荷存入页 12、37、84，描述符由首页号 12 索引。调度引擎只读取描述符与队列状态，授权组帧器读取载荷；消息出队后，队首沿 `tx_mnext[12]` 前移到 91。下方展示相同载荷在双网帧中的位置，以及读完后归还的页号

发送侧共设 7 张表，均采用 `afdx_ram_sdp`（单写单读、同步读出）实现。每张表的写口和读口分别由一个模块独占，见表 5-1。

`tx_cfg` 中每条 VL 占 8 个字，其中前 6 个配置字有效（表 8-2）。将配置字合并为一张 1024×32 的单表并由引擎在授权过程中逐拍顺序读取（表 5-5），可减少拆分为六张浅表时的块 RAM 容量浪费。`afdx_tx_core` 例化 7 张表和 5 个子模块，统一管控 `tx_cfg` 的写口与复位初始化：复位后完成 `tx_cfg`（1024 拍）、`tx_state`（128 拍）清零与空闲页池装满（257 拍）后置位 `init_done`，启动引擎和分页缓存；运行期间仅在 `cfg_we` 与引擎的 `cfg_ok` 同拍为 1 时寄存写地址与数据，并于下一拍写入配置表。

表 5-1 发送侧配置与状态表

表	深度 × 位宽	写口	读口	内容
tx_cfg	1024×32	配置总线	tx_engine	每条 VL 8 个字，地址 {VL[6:0], 字号[2:0]}
tx_state	128×63	tx_engine	tx_engine	timer_active、next_time、sn、q_count、q_head、q_tail
tx_mnext	256×8	tx_engine	tx_engine	同一 VL 队列中的下一条消息（首页号）
tx_desc	256×44	tx_buf	tx_engine	消息描述：长度、port_en、源/目的端口
tx_pnext	256×8	tx_buf	tx_builder	消息内的下一页
tx_payload	131072×8	tx_buf	tx_builder	128 KiB 载荷，地址 {页号 [7:0], 偏移 [8:0]}
tx_free	256×8	tx_free	tx_free	空闲页号 FIFO

5.2 空闲页池

afdx_tx_free 管理载荷缓存的 256 个页号；每页从分配到归还期间均由一条消息独占。空闲页池采用 FIFO 结构，由一块 256×8 的 RAM、读写指针和计数器组成；复位后用 257 拍依次写入页号 0–255，装满后允许取页。

页池提供一个取页口和两个页面回收口，均带握手：

- 取页口 pop_req/pop_ready/pop_ack/pop_ok/pop_page 连接 tx_buf，采用请求应答握手；
- 页面回收口 1 ret1_* 连接 tx_builder，每读完一页便归还该页；
- 页面回收口 2 ret2_* 连接 tx_buf，用于回收被丢弃消息占用的缓存页。

取页由四步状态机完成，见表 5-2。从请求被接受到 pop_ack 共 4 拍，其中 2 拍用于同步 RAM 读出和寄存。

表 5-2 空闲页池取页状态

状态	动作	转移
POP_IDLE	pop_ready= 1；接受请求时若 count= 0 立即应答 pop_ok= 0，否则按 rd_ptr 发起 RAM 读取	有页 → POP_READ
POP_READ	等待同步 RAM 读出	→ POP_CAPTURE
POP_CAPTURE	寄存读出的页号	→ POP_REPLY
POP_REPLY	给出 pop_ack、pop_ok= 1 和页号	→ POP_IDLE

每个页面回收口先把页号锁存至各自的暂存寄存器（ret*_ready 表示该寄存器为空），再经共用写口逐个写入 FIFO 尾部；两个寄存器均满时页面回收口 1 优先。页面申请与释放同拍发生时 count 不变。

FIFO 深度等于页数，任一页号在池中至多出现一次。写口每拍可写入一个页号，两个页面回收口各自最快每 2 拍还一页。

5.3 分页缓存

afdx_tx_buf 从消息接口逐字节接收消息，按页写入 tx_payload，在 tx_pnext 中把同一消息的页串成链，最后写 tx_desc 并向引擎发出入队请求。载荷缓存共 256 页、每页 512 字节，地址为 {页号 [7:0], 偏移 [8:0]}。

载荷写口采用一级流水：写使能和地址先保存至 payload_we_d、payload_wa_d，数据根据 payload_wait_d 从暂存首字节/跨页字节或输入数据寄存器中选择，再统一锁存至 payload_we、payload_wa、payload_wd。常规末字节在 ENQUEUE 拍与描述符一同写入同步 RAM。发送核心启用 EARLY_ENQUEUE=1，在 DESC 拍提前给出入队请求；引擎仍须采样请求并读表，随后才可授权发送。

分页缓存的接口包括消息接口 m_*（表 C-1）；取页口和页面回收口 2；tx_payload、tx_pnext、tx_desc 的写口；入队请求应答口（表 C-2）；五个统计脉冲 ev_accept、ev_drop_dis、ev_drop_full、ev_drop_long、ev_drop_nopage。

5.3.1 消息接收与页分配

分页缓存的页链与 VL 消息队列见图 5-2，接收状态机各状态的动作见表 5-3。m_ready 由当前状态组合译码产生，初始化完成后仅在 IDLE、RECV、DRAIN 三个状态为 1。当 m_valid 与 m_ready 同时为 1 时，接口完成一次字节传输。

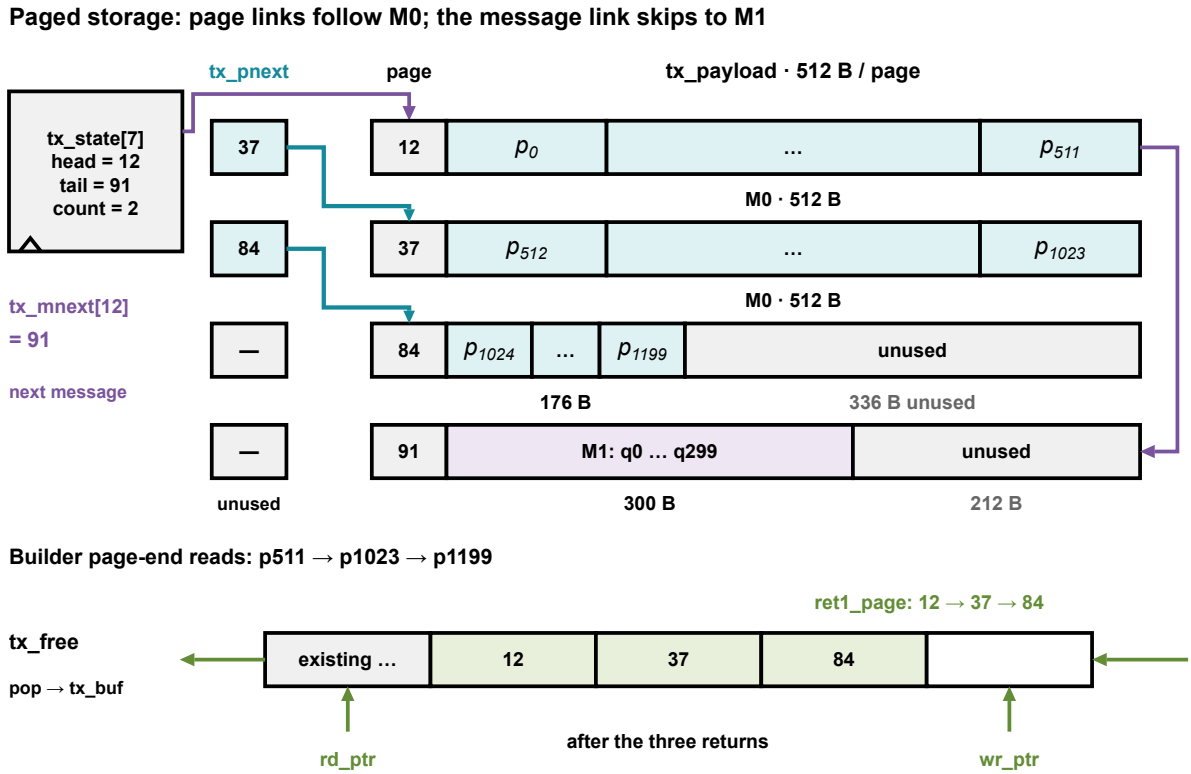


图 5-2 载荷 RAM 的稀疏页视图。M0 的 1200 字节分布于页 12、37、84，末页的 176 字节已用空间与 336 字节未用空间分开显示；青色连线按同索引的 tx_pnext 值访问下一页。紫色消息链从首页 12 跳到 M1 的首页 91，后者保存另一条 300 字节消息。下方为无并发取页时三次回收后的空闲 FIFO，读指针指向原有空闲页，写指针已越过新增的页号 12、37、84；终止页链的依据是消息长度

表 5-3 分页缓存各状态

状态	动作	转移
IDLE	接收首字节并锁存至 held_data，锁存 VL 与端口，长度记 1	→ ALLOC
ALLOC	置 pop_req	→ WAIT_PAGE
WAIT_PAGE	等 pop_ack；得页后记入 6 个页寄存器之一，首页记为 enq_head，否则写 tx_pnext[上一页] = 新页；把暂存字节写到新页偏移 0	无页 → RETURN；暂存字节是末字节 → DESC；否则 → RECV
RECV	每拍收一字节写入当前页，偏移加 1；偏移到 511 时置 boundary	已满 1471 字节仍有数据 → RETURN；跨页字节暂存 → ALLOC；末字节 → DESC
DESC	写入描述符表 tx_desc[enq_head] = {长度, port_en, 端口}	→ ENQUEUE
ENQUEUE	置 enq_valid	→ REPLY
REPLY	等 enq_ack，记下 enq_result	接纳 → EVENT；拒绝 → RETURN
RETURN	经页面回收口 2 逐页归还已占用的缓存页	消息已结束 → EVENT；否则 → DRAIN
DRAIN	继续接收剩余字节并丢弃（不写入缓存）	末字节 → EVENT
EVENT	按原因产生一个统计脉冲	→ IDLE

图 5-2 中，消息 M0 的前两页各保存 512 字节，末页保存 176 字节；读取以消息长度为界，末页剩余空间不参与发送。tx_pnext[12]= 37 指向本消息的下一页，而 tx_mnext[12]= 91 指向同一 VL 的下一条消息；M0 出队后，q_head 前移到 91。

写满一页后，将下一字节暂存于 held_data，取到新页后写入其偏移 0。跨页字节到达时才发起取页请求。最长消息占 3 页，6 个页寄存器为更小的页长参数留有余量。

5.3.2 时序与异常处理

以首字节被接收为第 0 拍：第 1 拍发取页请求，第 2 拍被空闲页池接受，第 6 拍 pop_ack 到达并写入首字节，第 7 拍回到 RECV。每次取页期间，m_ready 保持为 0 的时长共 6 拍。接收消息末字节后，DESC 和 ENQUEUE 各占 1 拍，引擎在 6 拍后返回入队应答，EVENT 再占 1 拍；在这一阶段，m_ready 约有 10 拍保持为 0。

超长（第 1472 个字节到来）、取页失败与入队被拒三种情况均转入 RETURN 态：先归还本消息已占用的全部缓存页，再继续接收并丢弃剩余字节。模块在输出该消息的统计脉冲前完成全部缓存页的回收。reason 记录首次出现的原因，每条消息在 EVENT 状态产生一个统计脉冲。

每条消息的统计脉冲出现在末字节被接收后 96 拍以内、下一条消息末字节之前，因此第 k 个脉冲对应第 k 条消息。

消息结束阶段决定了 m_ready 连续为 0 的最大时长：$2 + 64 + 9 + 1 = 76$ 拍，取设计约束 96 拍（第 3.7 节）。一条 1471 字节消息在应用口共占用 1499 拍，短于其在线路上占用的 1538 拍。

5.4 调度引擎

afdx_tx_engine 是 tx_state 和 tx_mnext 的唯一读写控制模块，负责判定消息能否入队、按 BAG 选择待发帧、向组帧器授权，并根据实际起始时刻更新下一次允许发送的时刻。

5.4.1 队列状态与操作仲裁

tx_state 为每条 VL 保存一项 63 位状态，见表 5-4。每条 VL 的消息队列是一条单向链表：q_head、q_tail 存首尾消息的首页号，tx_mnext[首页号] 指向下一条消息，因此 128 条 VL 共用 256 项链表。

表 5-4 tx_state 字段

位	字段	含义
62	timer_active	next_time 有效；为 0 表示可立即发送
61:30	next_time	下一帧最早起始的 now 值
29:22	sn	下一帧的序号，复位为 0，之后 1–255 循环
21:16	q_count	排队消息数，不超过配置的 qmax（至多 63）
15:8	q_head	队首消息首页号
7:0	q_tail	队尾消息首页号

引擎一次只执行一种操作，图 5-3 展示优先选择、共享计数器和状态表写口的具体示例。在 IDLE 状态，引擎按“发送起始确认（START）> 入队（ENQ）> 授权（GRANT）> 扫描”的优先级选择操作，选中后由 phase 控制各步骤逐拍推进，完成后返回 IDLE（表 5-5，表中拍数不含 IDLE 响应的 1 拍）。

入队操作在入队请求有效且满足“已有候选”或“连续扫描满 16 条 VL”时触发（want_enq）；授权操作在已有候选、组帧器待发槽位就绪且无待确认起始帧时触发（want_grant）。无上述操作时，IDLE 每 2 拍扫描检查一条 VL。

表 5-5 调度引擎的操作

操作	触发	逐拍动作	拍数
START	组帧器报告 t_{start}（先锁存到 ack_pending）	0：读 tx_state[pend_vl]；1：保存；2：写 timer_active= 1、next_time= t_{start}+bag，清 pend_valid	3
ENQ	enq_valid，且已有候选或扫描配额已满	0：读 tx_cfg 字 0 和 tx_state；1：保存；2：d =now−next_time；3：$d+512$；4：判定，接纳则写回 q_count+1、新队尾，非空队列另写 tx_mnext[旧队尾]，并在条件满足时直接定为候选	5
GRANT	有候选且 grant_ready	0–4：依次读配置字 1–5；1：保存字 0 和 tx_state；2：读 tx_desc、tx_mnext[队首]；4：保存描述；7：给出授权，写回 SN 前进、q_count−1、队首前移，记 pend_vl	8
扫描	无上述请求且无候选	每 2 拍发起一次 VL 状态读取，经 5 级流水判定	2/VL

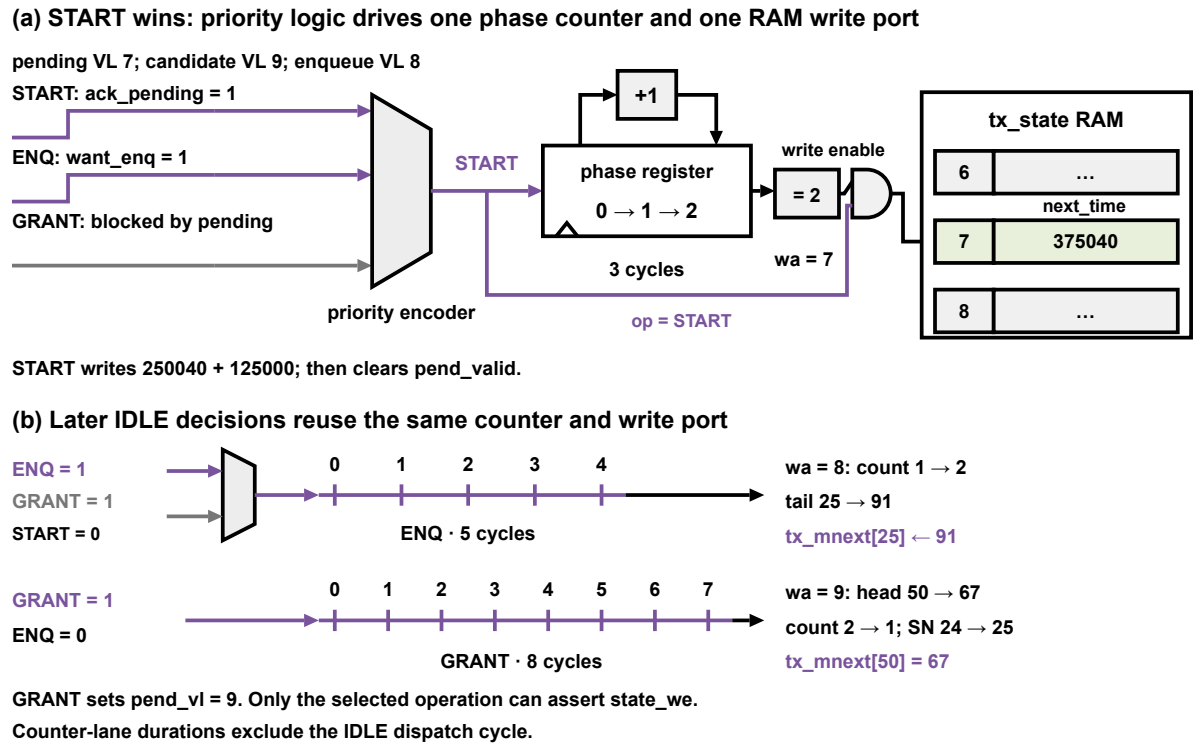


图 5-3 优先选择、带加一反馈的 phase 寄存器与单一状态表写口。(a) START 和 ENQ 同时请求，待确认起始帧使 GRANT 暂不可用；选中 START 后在 phase 2 写回 VL 7 的新期限并清除 pending。(b) 后续 ENQ 与 GRANT 继续复用计数器和写口，分别在末阶段写入 VL 8、9 的状态；刻度表示时间上的 phase 值，拍数不含 IDLE 响应拍，入队示例假定检查通过

5.4.2 入队与扫描

入队判定按顺序检查：VL 未启用或两网都未启用时返回结果码 1；消息长度 n 大于 `max_payload` 时返回 3；`q_count` 不小于 `qmax` 时返回 2；其余情况接受入队并返回 0。接纳入队时若当前没有候选、该 VL 不是 `pend_vl` 且已在 512 拍窗口内，直接定为候选，省去一次扫描。

扫描时每 2 拍发起一次 `tx_state` 读取，依次检查各条 VL，再通过 5 级流水作出判定。图 5-4 展示两条 VL 的判定与选中消息的发送示例。记 $d = \texttt{now} - \texttt{next_time}$，判据为：

- `q_count` $\neq 0$、（`timer_active` $= 0$ 或 $d + \texttt{LOOKAHEAD} \geq 0$）且不是 `pend_vl`：定为候选，扫描停在下一条；
- `q_count` $= 0$、`timer_active` $= 1$ 且 $d \geq 0$：写回 `timer_active` $= 0$，防止 `now` 回绕约 34 s 后把旧时刻误判为将来；
- 判定完成时 `scan_commit` 指针前移至下一条 VL；任一操作插入时流水线作废，自 `scan_commit` 重新发出，并按更新后的状态重新判定。

扫描从上次选中 VL 的下一条继续，轮流检查全部 128 条 VL。入队操作仅在已有候选或扫描满 16 条（`SCAN_QUOTA`）时插入，使持续入队期间仍能推进扫描；无候选时，任一 VL 相邻两次被检查的间隔约 400 拍（第 3.7 节）。

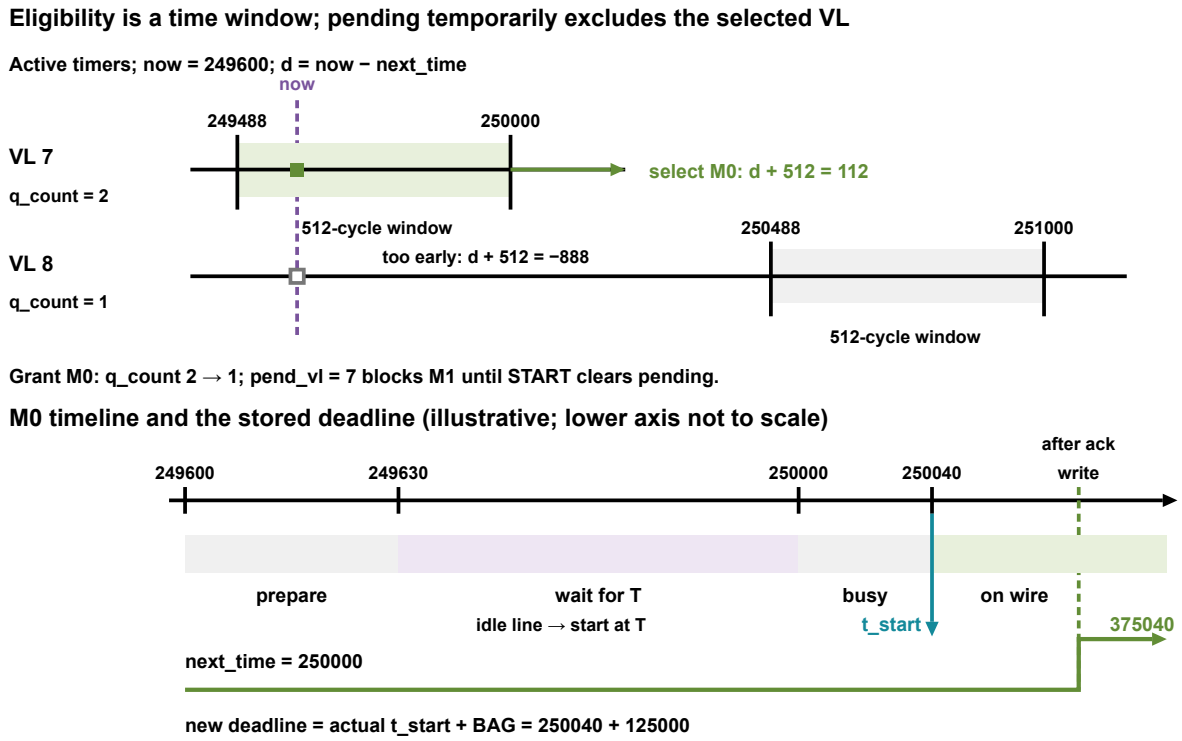


图 5-4 两条 VL 的资格窗口与 M0 的期限状态。定时器有效、now= 249600 时，VL 7 已进入其 512 拍提前准备窗口，VL 8 尚未进入；VL 7 授权后以 pending 阻止队内下一条消息重复入选。下方时间轴为不按比例的机制算例：M0 在 $T = 250000$ 前准备就绪，但因线路争用在 250040 起始；新期限按实际起始加 125000 拍 BAG 得到 375040，并在收到发送起始确认、执行 START 写回后才更新，阶梯不与起始拍重合。无定时约束时跳过 T 检查

5.4.3 授权状态与发送时刻

一帧从授权到真正起始之间，该 VL 的 `next_time` 仍是旧值。`pend_vl` 保存该帧的 VL，扫描和入队操作均不会再次将其选为候选；发送起始确认（TX Start Confirm）写回新的 `next_time` 时清除该标记。组帧器的单帧待发槽位对应一个 `pend_vl` 寄存器。

128 条 VL 轮询一遍需 256 拍，长于最短帧的 84 拍时隙；若到期后再启动扫描，发送时刻将随轮询位置浮动。引擎在到期前 `LOOKAHEAD`= 512 拍内选出候选并完成授权，组帧器提前完成长度和校验和计算，使 MAC 在 `now` 到达允许时刻 T 的当拍输出第一个前导字节。扫描回访约需 400 拍，再加上授权和准备所需的 22 拍，合计小于 512 拍，见第 3.7 节和图 3-3。

`next_time` 以实际起始时刻 t_{start} 加 `bag` 计算。线路争用使某帧晚发时，下一帧的允许时刻随之顺延，同一 VL 相邻两帧的起始间隔始终不小于 BAG。

授权阶段将当前 `sn` 锁存至组帧器，并将下一序列号写回状态表：复位后各 VL 首帧序列号为 0，随后按 1–255 循环递增。

5.5 组帧器

`afdx_tx_builder` 根据授权信息组装完整帧内容（目的 MAC 到 SN），在允许时刻送入 MAC，并向引擎报告实际起始时刻。

组帧器内部最多同时处理两帧：一帧在 `stream_active` 状态向 MAC 送字节，另一帧在 `pending` 指示的待发槽位中准备并等待。`grant_ready` $=\neg$`pending`，`pending` 在本帧的 `tx_sof` 有效时清除。因此，当前帧起始后即可接受下一帧授权，并在当前帧发送期间完成下一帧的准备。

5.5.1 帧头准备与载荷发送

组帧器的结构见图 5-5：帧头通过 336 位移位寄存器输出，载荷通过预读流水和 8 项字节 FIFO 输出，补零和 SN 则由计数器控制输出，三路数据经同一个选择器送入 MAC。

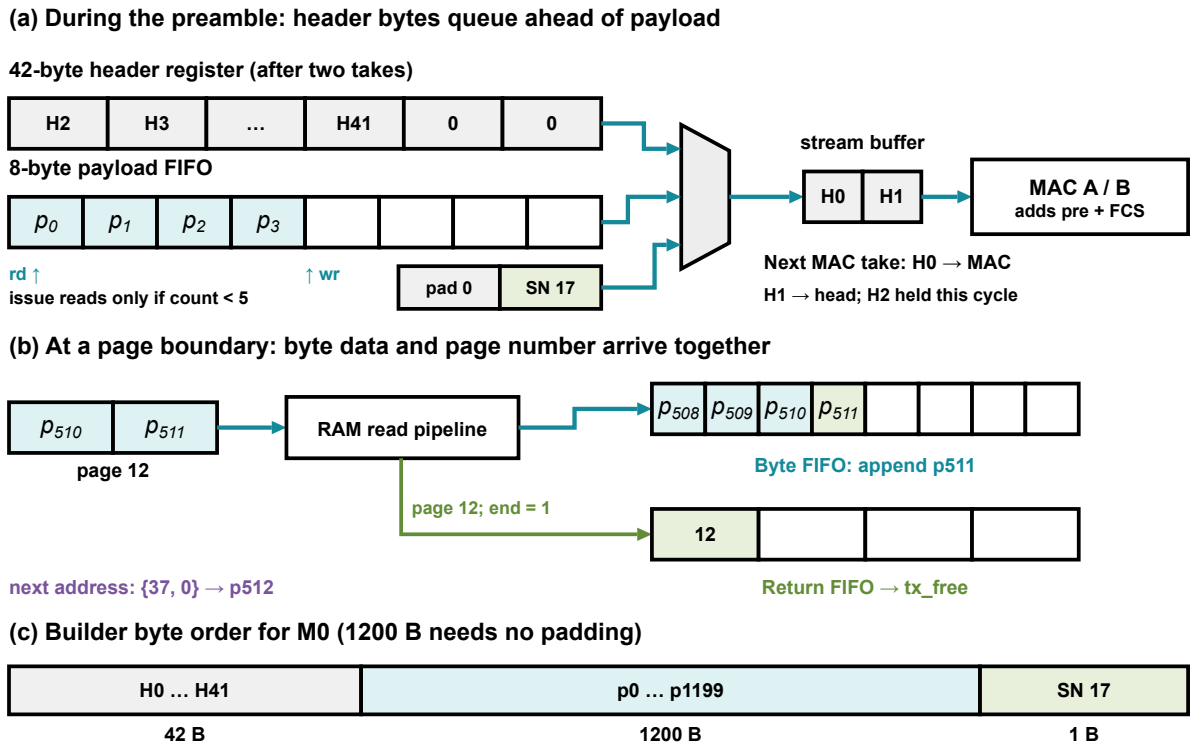


图 5-5 以 M0 的具体字节展示组帧器机制。(a) 前两个帧头字节已进入流缓冲，帧头寄存器已移位两次，载荷保留在 8 项 FIFO 中。下次 MAC 握手取走队首 H0，H1 前移；由于上游 ready 已寄存，该拍 H2 继续保持。(b) 页 12 的末字节 p_{511} 进入字节 FIFO 时，页号 12 同时进入独立的 4 项页面回收 FIFO；下一个页地址为 {37,0}，对应 p_{512}。FIFO 内容为逻辑队列顺序示例。(c) 1200 字节消息无需补零，帧头、载荷、SN 依次输出

组帧器在接受授权的同拍锁存全部字段和 `USER_ID`，以 16 位计数器的当前值作为 IP 标识，并将计数器加 1。随后经表 5-6 所示的 10 个准备步（连同授权锁存拍共 11 拍）计算 IP 头校验和，每拍只做一次加法。

表 5-6 组帧器准备步骤

prep	动作
0	IP 总长 $= n + 28$，UDP 长 $= n + 8$；累加器置初值 `0x4500+0x0111=0x04611`
1–6	依次累加 IP 总长、标识、源 IP 高/低 16 位、目的 IP 高/低 16 位
7–8	两次把高 4 位进位加回低 16 位
9	取反得校验和，置 `prepared`

启动发送时，必须已置位 prepared 且上一帧已送完，同时满足以下任一条件：本帧无定时约束（grant_timed= 0，即该 VL 此前队列为空、允许立即发送）、允许时刻已过，或距离允许时刻至多 6 拍。

为隔离组帧状态机与双网 MAC 发送器之间的跨模块路径，afdx_tx_core 在 afdx_tx_builder 与双路 afdx_mac_tx 之间插入了一级流缓冲（stream buffer，含 buf_valid/buf_ready 与 out_valid/out_ready 的双缓冲解耦结构）。为精确补偿载荷读流水与输出流缓冲引入的寄存拍数，afdx_tx_builder 的提前启动偏移配置为 6 拍（参数 START_LEAD_REGISTERED = -6，对应 now + 6 >= T 时在 S_IDLE 锁存槽位）。由此，在线路空闲时，MAC 输出首个前导码字节的时刻与理论到期时刻对齐在 now= T 当拍。

起始拍将 42 字节帧头并行装入移位寄存器，A、B 两路共用，仅将 B 网的第 11 字节（网络接口标识）替换为 0x40。每当 MAC 取走一个字节，组帧器便按 part 推进输出：先发送 42 字节帧头，再发送 n 字节载荷；若 $n < 17$，则补零至 17 字节。最后发送带 s_last 的 SN。tx_sof 所在拍的 now 作为 t_{start} 回报引擎。

从起始拍开始，只要 FIFO 中不足 5 字节，组帧器便每拍发起一次 tx_payload 读取；读请求寄存、同步 RAM 和输出寄存共 3 拍后写入 FIFO。当前页与下一页号分开保存：起始时读 tx_pnext[首页]，此后每次跨页时，读取下一页的后继页号，使跨页前后的读请求保持连续。读到页末字节或消息末字节时，该页号随数据一起进入 4 项页面回收 FIFO，经页面回收口 1 归还至空闲页池。

5.5.2 时序与异常处理

- **FIFO 容量**：发起读取时，FIFO 中至多有 4 字节，另有至多 3 个尚未返回的读请求，最多 $4+3+1=8$ 字节，等于 FIFO 深度。
- 载荷预读与供给时序：MAC 在起始后第 8 拍才取首字节（字节 0），帧头又占 42 拍，第一个载荷字节在起始后第 50 拍才被取走，而 FIFO 在起始后第 4 拍就开始填充；此后每拍补入一字节、取出一字节。下一页号在跨页前 500 拍以上已读出。
- 准备时间：下一帧准备需 11 拍，可与当前帧的发送重叠；最短帧时隙为 84 拍。

在无发送争用且当前 VL 已满足 BAG 间隔时，从应用消息末字节提交完成到 GMII 首前导码字节 tx_sof 固定为 32 拍（0.256 µs，原为 31 拍 / 0.248 µs）；若按 1022 答疑 R95 规则计至以太网 SFD（8'hD5，前导码第 8 字节）结束，则发送处理时延 L_{tx} 典型为 39 拍（0.312 µs），上界为 100 拍；基准条件与逐拍分解见第 3.7 节图 3-3。

若 MAC 输出 tx_underrun，本帧记为错误帧，不产生 ev_tx_a/ev_tx_b；其余帧则在 tx_eof 有效时，分别为本帧启用的网络产生一个统计脉冲。

5.6 双网发送 MAC

`afdx_mac_tx` 用一个状态机驱动 A、B 两套 CRC 和输出寄存器，两网同拍输出。发送节拍由 MAC 控制，上游按其握手信号提供字节。图 5-6(a) 展示两网字节分别进入 CRC 累加器、形成 FCS 及相关状态转换，(b) 展示第 6.2 节接收 MAC 的四字节尾部寄存与 FCS 剥离机制。

相比每网独立例化一套带用户侧 FIFO 的通用三速以太网 MAC IP（相同器件下的资源对照见测试报告），共用单状态机和字节位置控制既能减少重复控制逻辑，也从结构上保证 A、B 两网副本在 MAC 输出寄存器端同拍起始。相应地，两网共用发送时隙，单网 VL 也按整帧占用该时隙，带宽按全部发送 VL 合并预算（第 8.9 节）。

发送状态机仅在 `byte_en` 有效时推进，帧间隔计数按 `clk` 递减；当前设计中 `byte_en` 在复位后恒为 1，发送时序按每拍一字节组织。各状态的动作见表 5-7，发送一帧的字节时间见表 5-8，L 为取走的字节数。MAC 在起始拍锁存 `s_en_a`、`s_en_b`；未启用的网络在整帧期间 `tx_valid` 与 `tx_data` 保持为 0；两网均未启用时状态机仍按相同节拍推进，使各帧时序与网络使能无关。上游持续有帧待发时，帧间隔恰为 12 拍。

数据阶段若上游未能及时提供字节（下溢 Underrun），MAC 输出 `tx_underrun` 脉冲，发出 4 字节不取反的错误 FCS，然后继续接收并丢弃上游这一帧的剩余字节。

复位期间线路输出全为 0，复位结束后先等待 12 拍帧间隔，再允许发送首帧。

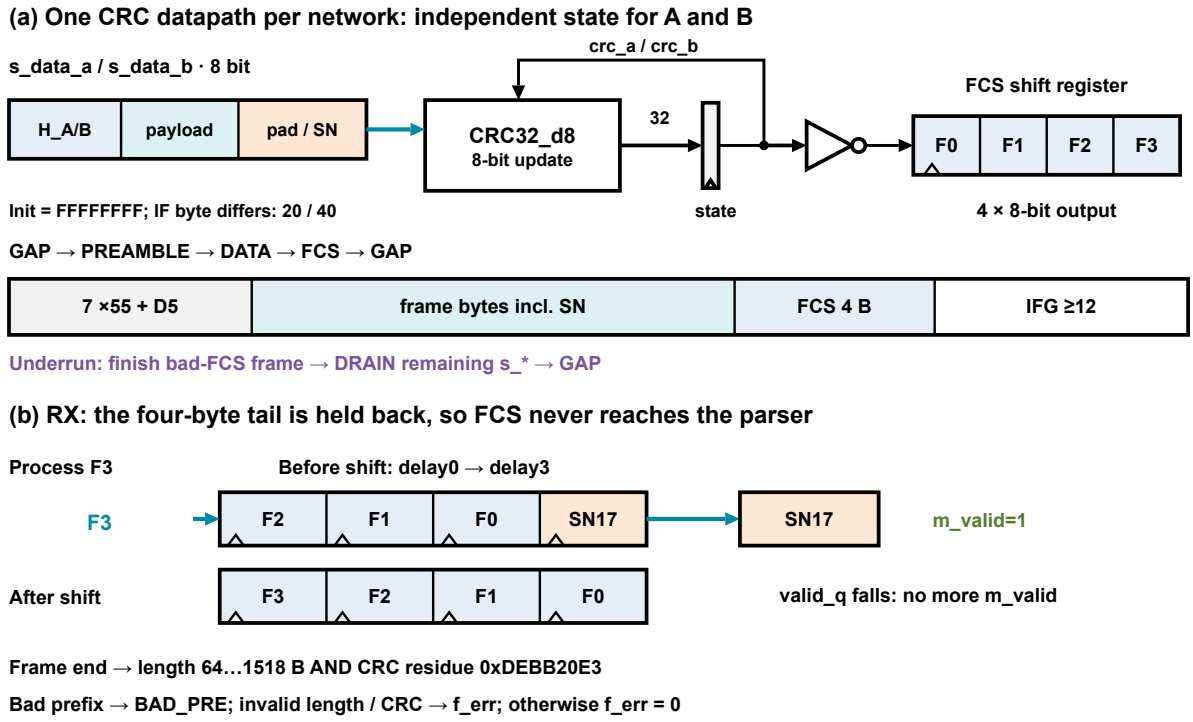


图 5-6 双网 MAC 的字节处理及状态转换机制。(a) A、B 两网帧头中的接口标识不同，分别更新独立 CRC 并生成 FCS；下方给出正常发送与下溢（Underrun）后的转换路径。(b) 接收侧保留最近四个字节：处理最后一个 FCS 字节时交付此前保存的 SN，帧结束后保留的四个 FCS 字节不再输出到解析器

表 5-7 发送 MAC 各状态

状态	动作	转移
GAP	输出 0；`gap_left` 递减	`gap_left`$=0$ 且 `s_valid` → PREAMBLE，同拍锁存两网使能、输出 `tx_sof`
PREAMBLE	输出 7 个 `0x55` 和 `0xD5`；末拍 `s_ready`$=1$ 取走首字节（字节 0）	取到字节 → DATA；未备好 → FCS（下溢 Underrun）
DATA	每拍输出一字节并更新两网 CRC	末字节已出 → FCS；未备好 → FCS（下溢 Underrun）
FCS	输出 4 字节 FCS，低字节先发；第 4 拍后撤 `valid`、输出 `tx_eof`，`gap_left`$=11$	正常 → GAP；下溢（Underrun）→ DRAIN
DRAIN	`s_ready`$=1$，继续接收并丢弃上游剩余字节	收到 `s_last` → GAP

表 5-8 MAC 发送的字节时间

字节时间	线上内容	本拍动作
起始前一拍	空闲	判断起始，锁存两网使能
0–6	`0x55`	
7	`0xD5`	取走首字节（字节 0）
8 至 $8+L-1$	字节 0 至 $L-1$	取走下一字节，取走带 `s_last` 的字节后停止
其后 4 拍	FCS	A、B 两网各自更新 CRC32
再后至少 12 拍	空闲	第 12 拍可判断下一帧起始

第 6 章　接收通路

接收核心 `afdx_rx_core` 为 A、B 两路网络各配置一套接收 MAC 和帧头解析器，两网共用同一接收引擎与交付模块。MAC 在接收过程中逐字节校验 CRC，解析器同步检查帧头并将载荷预写入本网的环形缓冲。对于通过解析器检查的帧，接收引擎根据帧描述依次执行 VL 查表、完整性检查和冗余管理，再通知解析器提交数据或回退写指针；已提交的消息最终由交付模块逐字节送往应用口。

6.1　接收核心与配置表

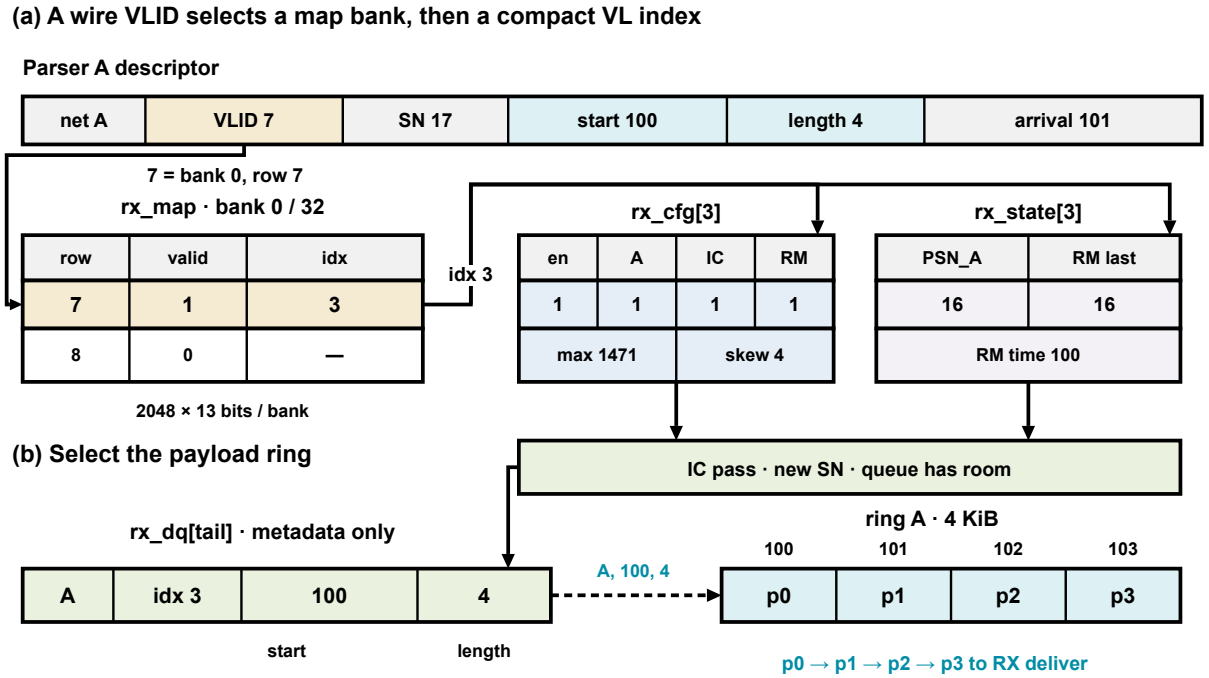


图 6-1　接收查表与载荷定位示例。A 网的 VLID 7 选择映射表 bank 0 的第 7 行，得到索引 3，随后读取该项配置和状态。接纳后，交付队列只保存元数据，通过网络、起点和长度定位 ring A 中的四个载荷字节；图中仅列出本例使用的字段

接收侧的片上表见表 6-1。

为了降低大容量存储表在大扇出布线上的时序延迟，`rx_map`（逻辑规模 65536×13 bit）在物理实现上划分为 **32** 个 2048 × 13 **bit** 的 **`afdx_ram_sdp`** 子库（**bank**）。写侧与接收描述表共享 16 路局部配置总线副本（参数 `RX_CONFIG_BANKS=16`），读侧采用两级寄存器多路复用流水；查表时由高 5 位地址选中目标子库，读出延迟由原单级 1 拍调整为 2 拍读回流水。

`rx_cfg` 中的 `skew_max` 以 `now_c` 的计时单位（32.768 µs）计。`afdx_rx_core` 例化 A、B 两路接收 MAC 与解析器、共用接收引擎、交付模块及 5 张表，其中 `rx_map`、`rx_cfg` 的写口由核心统一管控。复位后，核心用 65536 拍逐项清零 `rx_map`（前 4096 拍同步清零 `rx_cfg`），引擎另行清零 `rx_state`；全部完成后置位 `init_done`。运行期间仅在 `cfg_ok`

表 6-1　接收侧配置与状态表

表	深度 × 位宽	写口	读口	内容
rx_map	65536×13	配置总线	rx_engine	valid[12]、idx[11:0]，物理划分为 32 个 2048×13 子库
rx_cfg	4096×32	配置总线	rx_engine	en、net_a、net_b、ic_en、rm_en、skew_max[16:5]、max_payload[27:17]
rx_state	4096×44	rx_engine	rx_engine	每网 PSN 及有效位；rm_last、rm_time 及有效位
rx_ring_a/b	4096×8	本网 rx_parse	rx_deliver	每网一个环形缓冲，按到达顺序存载荷
rx_dq	32×68	rx_engine	rx_deliver	afdx_ram_sdp 块 RAM：来源网、起点、长度、idx、UDP 端口

为 1 时锁存配置写请求并于下一拍写入配置表。核心将各模块的统计脉冲按位相或，寄存一拍后输出。

6.2 接收 MAC

`afdx_mac_rx` 把 GMII 输入寄存一拍后，在 `rx_valid` 为 1 的每一拍处理一个字节，FCS 剥离机制见图 5-6(b)。

- 帧起始定界（**SFD** 检测）：前导字节 `0x55` 的数量不限，检测到 `0xD5` 后以紧随的下一字节作为帧首字节。若收到其他值，或在检测到 `0xD5` 之前 `rx_valid` 已回到 0，则判为前导码错误。复位后若 `rx_valid` 已为 1，MAC 先等待其回到 0，再从下一帧开始接收。
- 剥离 **FCS**：每接收一字节均更新 CRC 和帧长；输出比输入晚 4 字节，收到第 $k+4$ 字节时输出第 k 字节，最后 4 字节（FCS）不送后级。
- 帧结束：寄存后的 `rx_valid` 首次为 0 时，MAC 判定帧接收结束，输出 `f_end` 和 `f_err`。错误判定依次检查前导码错误、帧长不在 64–1518 范围内、CRC 余式不等于 `0xDEBB20E3`，均无异常则判为正确帧。MAC 向后级最多输出 1514 字节；若帧长超限，则按帧长错误结束本帧。
- 帧间：`f_end` 之后立即回到空闲，可分辨间隔为 1 个低电平周期的两帧。

以 GMII 上 FCS 末字节到达时刻为第 0 拍：第 1 拍该字节锁存至输入寄存器，第 2 拍寄存后的 `rx_valid` 拉低，第 3 拍输出 `f_end`、`f_err` 和 `f_len`。剥离 FCS 后的最后一个有效字节在第 2 拍从 `m_data` 输出，比第 3 拍的 `f_end` 早 1 拍，解析器据此完成校验和累加。

6.3 帧头解析与预写入缓存

`afdx_rx_parse` 由字节位置计数器 `position` 驱动，随输入字节流式检查帧头；另设两个辅助状态机分别负责长度计算与帧尾分类，结构见图 6-2。

6.3.1 流式检查与长度计算

帧头必须满足以下条件，任一条件不满足均判为“帧头错”：

- 第 0–3 字节为 `03 00 00 00`；第 12–13 字节为 `08 00`；第 14 字节为 `0x45`；第 23 字节为 `0x11`；
- IP 不分片（MF 为 0、片偏移为 0）；IP 头 10 个 16 位字的反码和为 `0xFFFF`；
- 记 n 为 UDP 长度减 8，要求 $1 \le n \le 1471$，IP 总长等于 $n+28$，且含 FCS 的帧长等于 $47 + \max(n, 17)$；
- UDP 校验和非 0 时，伪头、UDP 头与 n 字节数据的反码和为 `0xFFFF`。

补零字节的值不检查。解析器同时记录 VLID、源/目的端口、n 和 SN。

在第 39 字节收到 UDP 长度低字节后，`length_phase` 控制长度计算分两拍完成：先在 LEN_SIZES 步计算 n、检查长度是否合法，并求出期望 IP 总长；再在 LEN_LIMITS 步计算载荷末位置、SN 位置、期望帧长和环形缓冲所需空间。第 42 字节到来时，上述结果均已就绪。

IP 校验和与 UDP 校验和分别使用 20 位和 26 位累加器，每两字节累加一次：前者累加 IP 头的 10 个 16 位字，后者累加 UDP 伪头、UDP 头及载荷。进位折叠采用两级寄

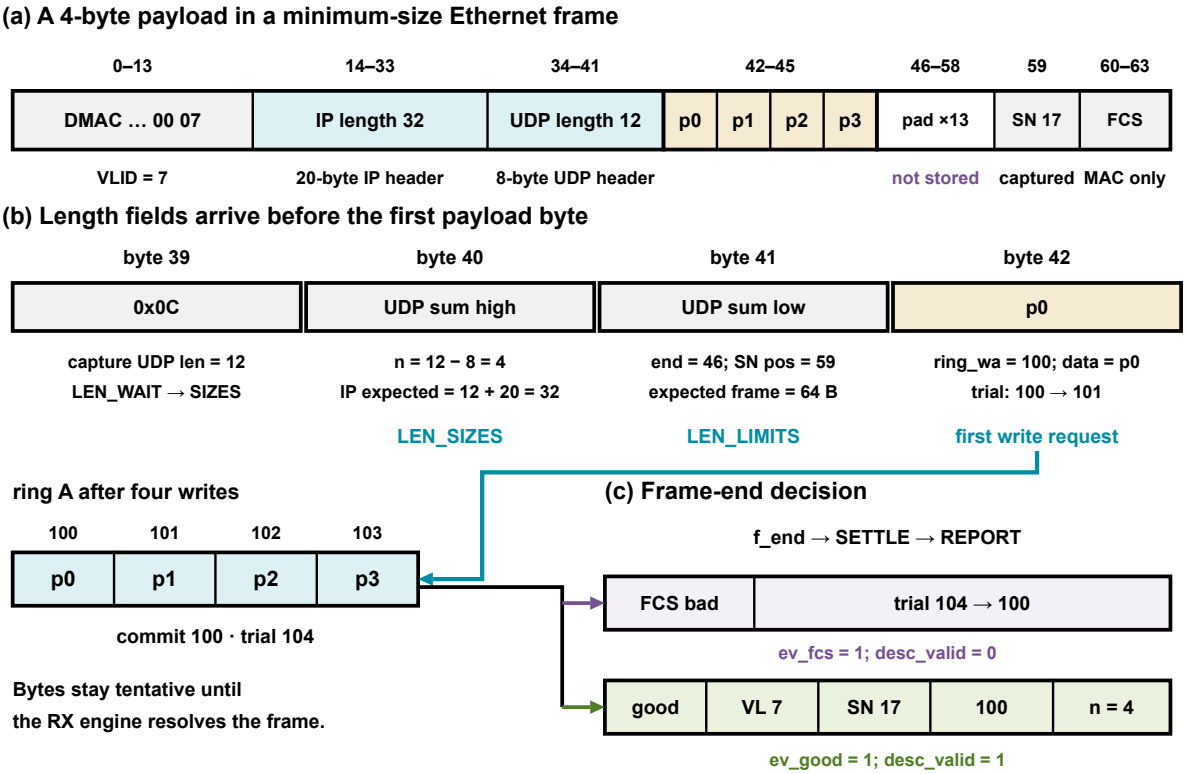


图 6-2 四字节载荷的解析示例：(a) UDP 长度 12、IP 总长 32，对应四个载荷字节、13 个补零字节和位置 59 的 SN；(b) 第 39 字节捕获长度，第 40、41 字节所在拍计算边界，第 42 字节发起首次载荷写请求；(c) 同一预写结果在帧尾按校验结果发布描述或回退。正确分支假定其余帧头检查均通过

存器流水，与累加过程并行；到帧尾后第 1 拍（FIN_SETTLE），UDP 校验和结果最终稳定。

6.3.2 载荷预写入与提交

从第 42 字节开始，解析器将载荷边接收边写入本网环形缓冲，待最终判定后再决定是否提交。环形缓冲采用三个指针（图 6-3）：读指针 `rd` 由交付模块维护；已提交写指针 `wr_commit` 和预写入指针 `wr_trial` 由本网解析器维护。

计算剩余空间时以 `wr_commit` 为准，不计入正在接收的帧。第 42 字节到来时 n 已知，若剩余空间小于 n，解析器便不写入本帧载荷，并将其记为溢出。

6.3.3 帧尾分类与描述交接

每次 `f_end` 有效时，解析器均对该帧进行一次分类。`finish_phase` 在 `f_end` 后第 2 拍进入 FIN_REPORT 并作出判定，类别脉冲和帧描述均经寄存后输出，在第 3 拍有效。分类时，若 `f_err` 表示 FCS 错误或帧长错误（含前导码错误），则直接采用该结果；否则依次判定帧头错误、溢出或正确。

解析器在帧尾完成错误帧的分类与计数，并将预写入指针 `wr_trial` 回退至已提交写指针 `wr_commit`；正确帧则生成帧描述字交由接收引擎处理（表 C-4）。两网在同一拍结束的正确帧，其帧描述也在同一拍给出。

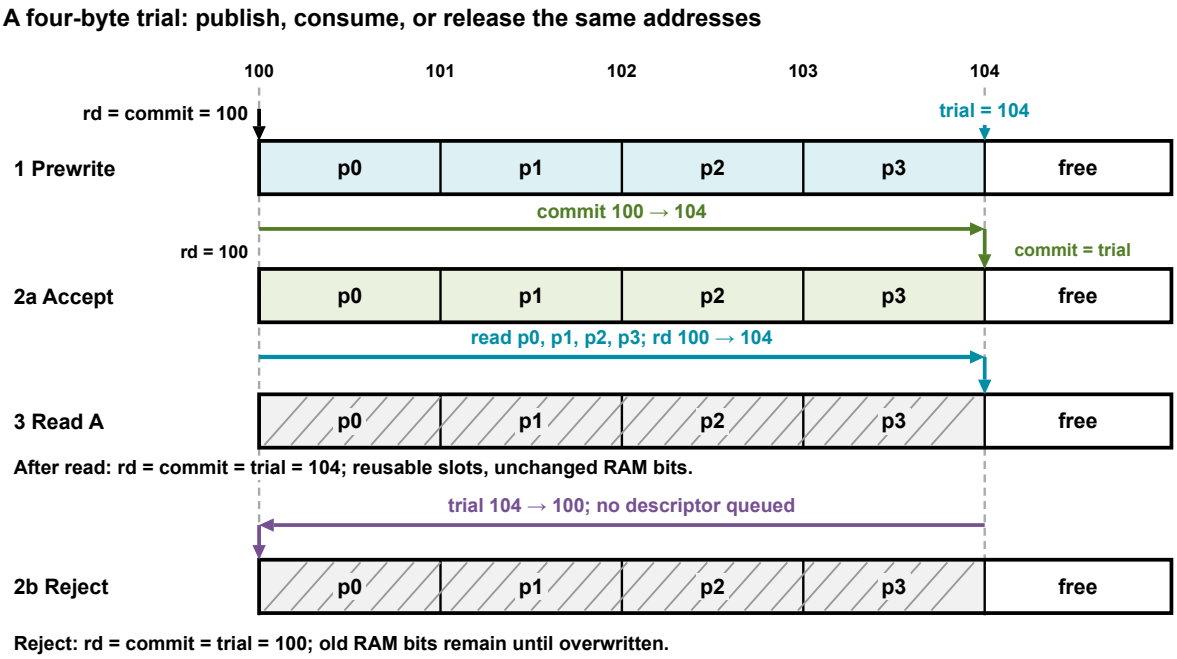


图 6-3 四字节载荷在同一地址坐标中的预写、接纳、读取与拒绝。接纳使 commit 从 100 前进至 104，逐字节读取再使 rd 前进至 104；拒绝是从预写状态出发的另一分支，使 trial 从 104 回退至 100。斜线表示空间可复用；读取和拒绝均不清除 RAM 中的旧位

同一网相邻两帧的 `f_end` 至少相隔 73 拍（最短帧 64 字节加前导码 8 字节和 1 拍间隔）。引擎处理一份描述至多需要 15 拍，两网各一份描述的服务时间合计至多 30 拍；加上中断 AGE 的至多 1 拍，仍短于同一网相邻描述的到达间隔。若数据帧的首个前导字节在表初始化完成前到达，解析器将忽略整帧，不生成帧描述，也不计入统计。

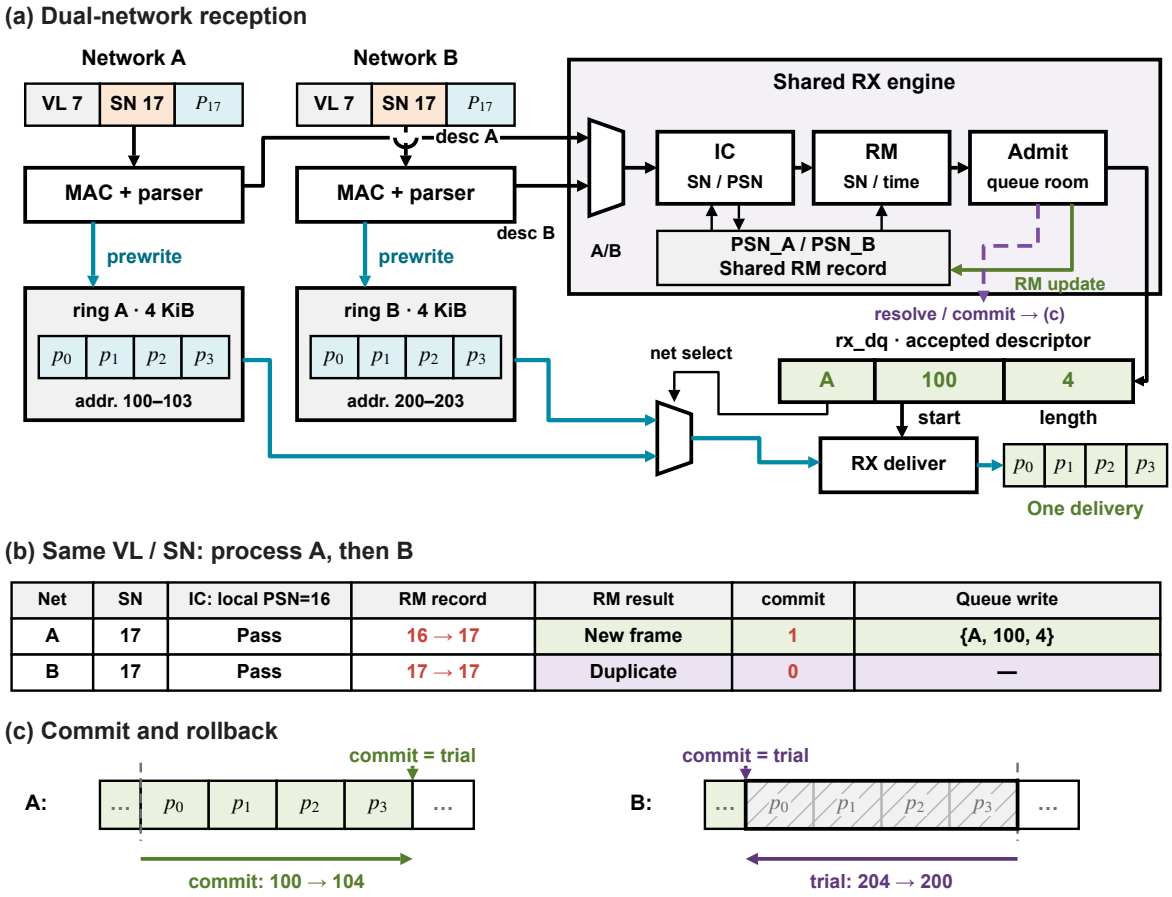
6.4 完整性检查与冗余管理

`afdx_rx_engine` 是 `rx_state` 的唯一读写控制模块，一次处理一份帧描述或执行一步冗余记录过期清理，两网描述同拍到达时先处理 A 网。

6.4.1 帧描述处理流程

双网副本从预写入到一次交付的机制见图 6-4。引擎先查询 `rx_map`，再读取 `rx_cfg` 和 `rx_state`，随后依次执行配置检查、完整性检查、冗余管理和交付入队。任一步未通过，均按对应原因计数，并通知解析器回退预写入指针。

引擎有 INIT、IDLE、FRAME、AGE 四个操作态，FRAME 按 `phase` 0–13 逐拍推进，每拍至多一级比较，见图 6-5 和表 6-2。对于未通过检查的帧，引擎提前跳转至写回或报告步骤，因此一份描述的处理时间为 8–15 拍（未知 VL 丢弃路径 8 拍，常规完整处理为 15 拍，含 IDLE 取描述的 1 拍）。图中的 `desc` 为 `desc_a_valid` 或 `desc_b_valid`；AGE 的任一步收到新描述时均返回 IDLE。



(b) Same VL / SN: process A, then B

Net	SN	IC: local PSN=16	RM record	RM result	commit	Queue write
A	17	Pass	16 → 17	New frame	1	{A, 100, 4}
B	17	Pass	17 → 17	Duplicate	0	—

图 6-4 双网副本的一次交付机制。示例中 VL 7 的两份 SN 17 副本同时待处理，各网 PSN 和初始 RM 记录均为 16；配置有效、IC/RM 已启用、到达间隔在冗余窗口内，且交付队列有空位。A 网先通过并入队，B 网通过独立 IC 后被共享 RM 判为重复，回退其预写指针；地址仅为示例

表 6-2 接收引擎 FRAME 各步

phase	动作
（IDLE）	取走帧描述（take），以 VLID 读多 Bank rx_map
0–2	等待多 Bank rx_map 2 级流水线读出
3	分 bank 接口在表内锁存映射结果 selected_map
4	有效映射以 selected_map[11:0] 触发 rx_cfg 与 rx_state 读取；无效映射撤销读使能
5–6	等待同步读出，保存配置和状态（无效映射在 5 拍跳转至 13 报告未知 VL）
7	寄存网络使能与最大载荷长度的检查结果；计算 PSN 一步后继、到达时间差与序号距离
8	配置检查不符则转 13 报告配置不允许；否则到达时间差取绝对值，计算 PSN 两步后继序号，距离折算到 1–255 环上
9	寄存完整性检查（IC）结果并更新本网 PSN；并行寄存冗余窗口和序号比较结果
10	IC 不通过则转 12；否则冗余管理（RM）判定 deliver_allowed
11	检查交付队列容量；若可交付且有空位则写入 rx_dq，并更新 rm_last、rm_time
12	状态有变化则写回 rx_state
13	给出 resolve 与 commit 脉冲，按处理结果产生统计事件，返回 IDLE

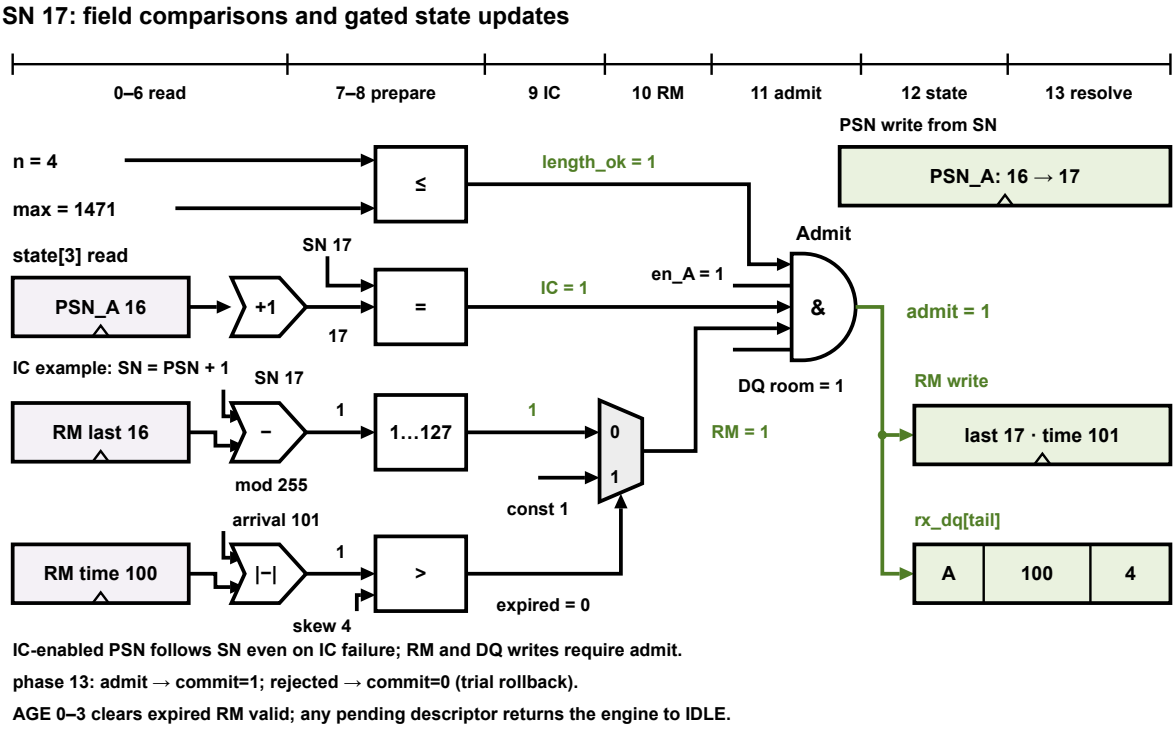


图 6-5 一份 SN 17 描述的比较与写使能通路。IC/RM 均启用且状态有效，本例 SN 恰好匹配 PSN 的一步后继；RM 以模 255 序号距离和到达时间差判新，窗口过期时由 MUX 直接放行。门控结果使能 RM 记录和交付队列写入；本网 PSN 的更新独立于最终接纳结果。上方标尺给出相关 FRAME 阶段

6.4.2 配置与完整性判定

`rx_map` 映射无效时，引擎将该帧计为“未知 VL”；VL 未启用、本网未启用或 n 大于 `max_payload` 时，计为“配置不允许”。

完整性检查（`ic_en`= 1 时）每网独立进行。记本网上一帧的序号为 PSN，满足以下任一条件即可通过检查：表初始化后本网首次收到该 VL；SN 为 0（对端复位）；SN 为 PSN 在 1–255 环上递增 1 步或 2 步的后继序号，后者允许丢一帧。PSN 为 0 时，后继序号分别为 1 和 2。

启用完整性检查时，通过映射与配置检查的帧均以当前 SN 更新本网 PSN，包括序号检查失败的帧；后续序号连续时可恢复同步。未通过序号检查的帧计为“完整性检查丢弃”。

引擎先对每份副本独立进行完整性检查，通过后再进行冗余管理，符合 ARINC 664 P7 的处理顺序。

6.4.3 冗余窗口与过期清理

冗余管理（`rm_en`= 1 时）按处理顺序交付首个有效副本；两网描述同拍待处理时先处理 A 网。副本到达间隔在 `skew_max` 内时，同一消息只交付一次，判据如下：

- 尚无记录，或到达时刻与 `rm_time` 之差超过 `skew_max`：交付；当两份副本的处理顺序与到达顺序相反时（如引擎忙期间 B 网先完成解析、A 网后完成解析，引擎转入空闲时按 A 网优先仲裁先处理了 A 网），对 16 位时间差取绝对值。

- 否则 SN 等于 rm_last：另一网的重复帧，丢弃并计“冗余管理丢弃”。
- 否则 SN 为 0 或 rm_last 为 0：交付。
- 否则在 1–255 环上，SN 领先 rm_last 1–127 步则交付，其余丢弃。

对于判定可交付的帧，若 rx_dq 已满，则计为“溢出”并回退预写入指针；否则提交环形缓冲中的载荷，将帧描述写入 rx_dq，并更新 rm_last 和 rm_time。两网描述同拍到达时，从描述有效到引擎完成判定，后处理的一份不超过 31 拍（两份描述各 15 拍，加上 AGE 被中断后返回 IDLE 的至多 1 拍；设计约束取 64 拍）。

now_c 每变化一次，引擎在空闲时通过 AGE 状态的 4 个步骤检查下一条 VL；有帧描述到来时立即返回 IDLE 处理帧，之后重试该项过期清理检查。判据为：若记录已超过 0x4000 个计时单位（约 0.537 s），清除 rm_time_valid。

帧描述处理优先于过期清理，清理被打断后会重新检查同一项。4096 项约需 0.134 s 检查一遍，记录在约 0.671 s 内失效，早于 16 位时间差的半圈时间 1.074 s。

6.5　交付

afdx_rx_deliver 按入队顺序从 rx_dq 取出帧描述，再从对应环形缓冲每拍读出并输出一个字节，同时释放已读空间，结构见图 6-6。

为降低 32 项大位宽寄存器多路复用树（原 8 选 1 + 4 选 1 结构）的组合逻辑级数，交付队列 rx_dq 由寄存器阵列重构为 afdx_ram_sdp 双端口块 RAM（32 项 × 68 bit）。

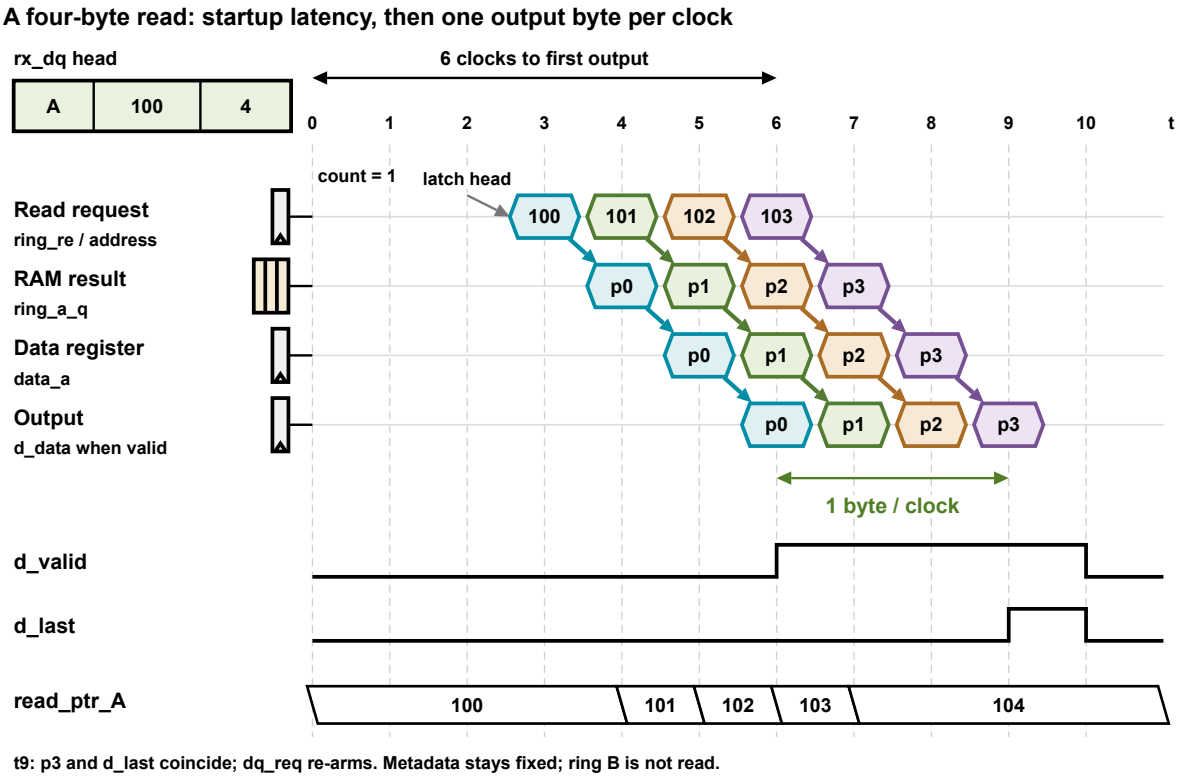


图 6-6　四字节消息的交付时空图。每条斜线跟踪同一字节从读请求经同步 RAM、数据寄存到输出的逐拍传播；总线值段标出各拍的读地址或字节，行首图元表示真实 RAM 或寄存器。队列计数在第 0 拍变为 1，第 6 拍出现首字节，随后每拍连续输出一个字节；有效、末字节和读指针波形共用同一时间轴，区分启动延迟与稳态吞吐

如图 6-6 所示，交付模块采用轻量化头部微状态机推进：队列读地址持续由 rd_ptr 给出；在 HEAD_IDLE 态，取头请求 dq_req 和非零队列计数触发状态转移。在 HEAD_SELECT 态锁存同步 RAM 输出 dq_q，给出 dq_valid 与出队脉冲，并推进 rd_ptr；HEAD_RELEASE 等待请求撤销后回到空闲。读控制接纳描述后锁存元数据，以起始地址发起第一次读取，随后由 issuing 和剩余字节计数 left 控制地址逐拍加 1，直至全部载荷读出。

读请求、同步 RAM、数据寄存流水由 valid_pipe、last_pipe 对齐，第一个 d_valid 在 dq_count 变为非零后的第 6 拍有效。d_last 有效的当拍，模块重新允许取下一个队头。

一条消息的 n 个字节在连续 n 拍内送出，d_last 与末字节同拍；VL 索引、端口和来源网络在整条消息期间不变。交付口不带反压。

线路空闲时，从帧尾 FCS 结束到首个 d_valid 的典型延迟为 26 拍（0.208 µs），设计上界为 200 拍（图 3-3）。若前一消息尚未交付完，当前消息需等待；取队头与读流水共约 6 拍，可在前一消息末字节之后的 16 拍内开始交付。32 项描述队列用于暂存长载荷交付期间到达的短帧描述（第 3.7 节）。

6.6 统计分类

对于表初始化完成后开始接收的每一帧（一段 rx_valid 连续为 1 的字节流），解析器均在帧尾离开线路后的 32 拍内，在对应网络上输出且仅输出一个类别脉冲。对于分类为正确的帧，接收引擎还输出且仅输出一个处理结果脉冲；该脉冲在引擎完成判定后的 8 拍内有效，且不晚于帧尾后的 150 拍。每网的 ev 位定义见表 6-3。

表 6-3 接收统计脉冲 ev_a、ev_b

位	名称	产生模块	含义
0	正确帧	rx_parse	通过 MAC 和帧头检查
1	FCS 错误	rx_parse	CRC 校验不符
2	帧长错误	rx_parse	帧长越界、超长或前导码错误
3	帧头错误	rx_parse	固定字段、IP/UDP 长度或校验和错误
4	未知 VL	rx_engine	rx_map 无效
5	配置不允许	rx_engine	VL 或本网未启用，或超过 max_payload
6	完整性检查丢弃	rx_engine	SN 不连续
7	溢出	两处相或	环形缓冲剩余空间不足，或 rx_dq 已满

位 7 由两处溢出脉冲相或产生：解析器报告环形缓冲剩余空间不足，引擎报告 rx_dq 已满。同一网上，相邻帧尾至少相隔 73 拍，引擎判定至多 31 拍，解析器的第 k 帧类别脉冲与引擎的第 $k-1$ 帧结果脉冲分时到达，可相或合并。

两网合计另有 ev_rm_drop（冗余管理丢弃）和 ev_deliver（交付）。这些脉冲与第 8 章的计数器一一对应，记录每类接收结果。

第 7 章　服务功能

RTC、SNMP 和 615A 均通过 UDP 报文通信，经服务通道 `afdx_svc` 与应用共享收发核心。三类服务报文均绑定独立配置的发送 VL，与用户消息统一参与 BAG 调度、完整性检查和冗余管理。

7.1　服务通道

`afdx_svc` 由组合选择逻辑和状态寄存器组成：发送侧为 `busy`、`owner`、`turn` 以及授权向量寄存器 `served[3:0]`，接收侧为 `rx_active`、`rx_claimed`，结构见图 7-1。

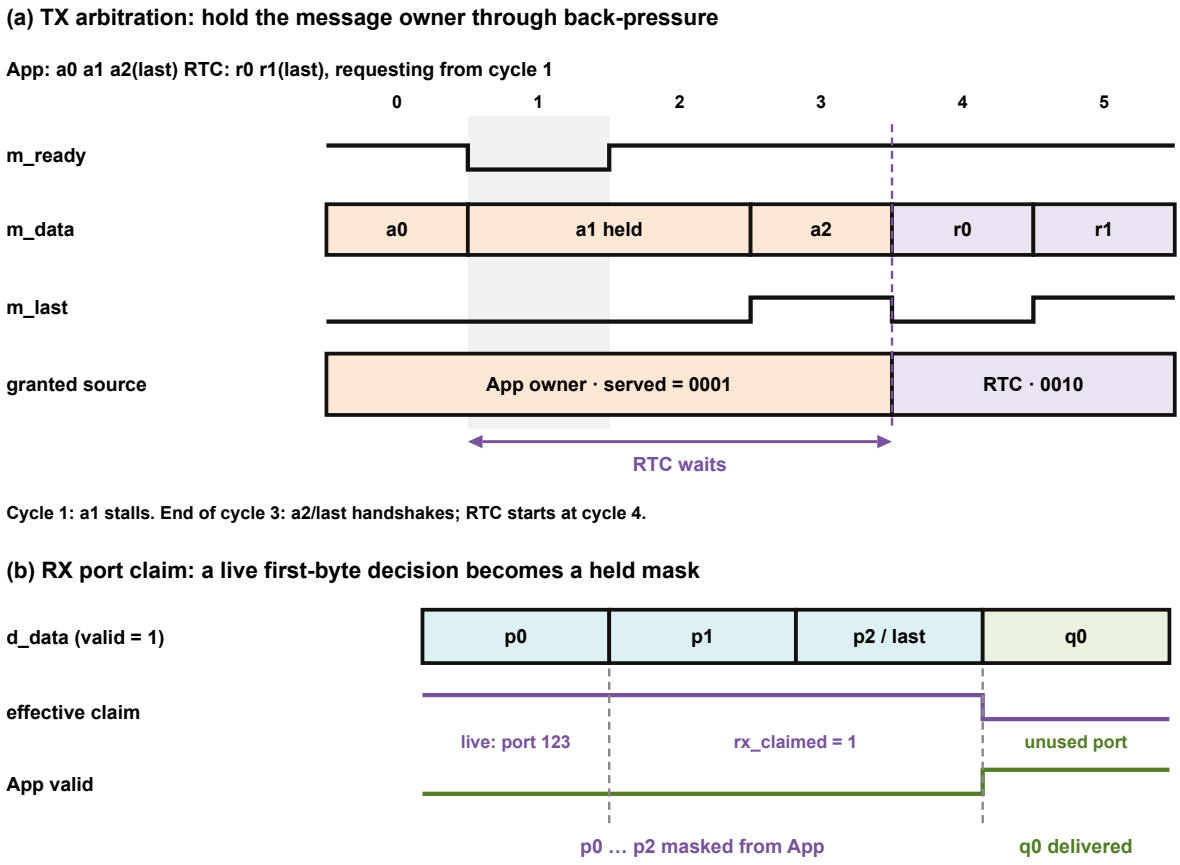


图 7-1　服务通道示例：(a) 应用消息未结束时保持授权，RTC 消息等待；(b) UDP 端口 123 在首字节认领并屏蔽应用交付

发送仲裁器以整条消息为单位，依次轮询应用、RTC、SNMP 和 615A 四个来源。选中的来源独占消息通路，直至末字节完成握手，随后从下一个来源开始轮询。服务发送口均空闲时，应用与发送核心之间的数据和 ready 信号直连。

为切断四个服务源发送状态机到发送核心 `afdx_tx_core` 注入多路器之间的跨模块组合长路径，`afdx_svc` 对仲裁授权向量进行了一级寄存（`served[3:0]`），实现仲裁决策与多路数据选通的时序解耦。发送仲裁器由状态寄存器构成：`busy` 指示当前消息传输忙，`owner` 锁存当前来源，`turn` 指示下一轮轮询起点；末字节握手时，仲裁器清零 `busy`，

并将 turn 更新为下一个来源。因此，任一来源提出请求后，至多等待其余三个来源各传完一条消息。

接收核心的交付数据同时广播给应用和三个服务（图 7-1(b)）。三个服务使用互不冲突的 UDP 端口，均不对交付口施加反压。首字节到达当拍，各服务根据 d_dport 经组合逻辑译码产生端口匹配信号，一方面直接门控首字节的应用口 valid，另一方面锁存至 rx_claimed 并在后续字节期间（rx_active= 1）持续保持。因此，匹配服务端口的消息由对应服务接收，应用口的 valid 和 last 全程保持为 0；其余消息原样交付应用。

7.2 RTC 时钟同步

afdx_rtc 维护由 32 位秒寄存器和 32 位纳秒寄存器构成的本地实时时钟，每拍递增 8 ns，纳秒值满 10^9 时向秒进位。秒计数以 2000-01-01 00:00:00 UTC 为起点；发送报文中的 NTP 秒值在此基础上加 3155673600。时钟同步遵循 RFC 4330 标准 SNTPv4 客户端/服务端协议，交互时序与四时间戳原理见图 7-2。

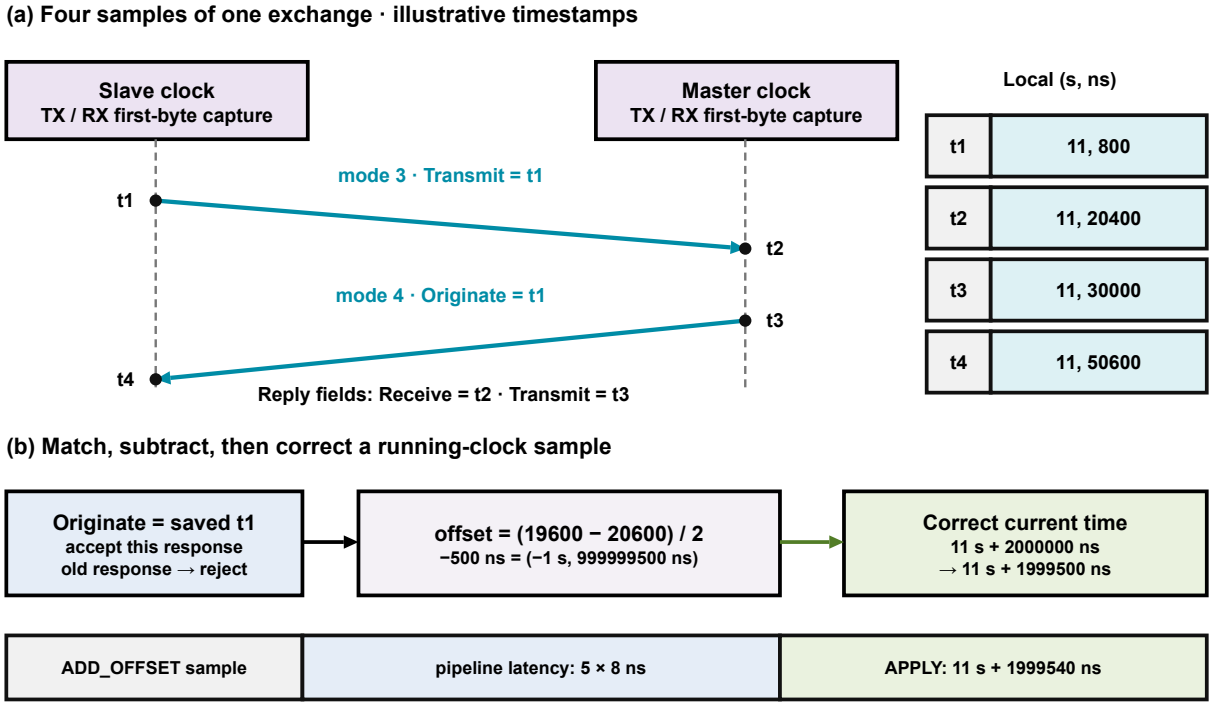


图 7-2 SNTP 四时间戳机制与 −500 ns 偏差校正；时间值用于原理说明

7.2.1 时间戳采样与收发时序

afdx_rtc 包含本地实时时钟、48 字节接收移位寄存器、发送字节状态机、时钟同步计算状态机和三个 afdx_ntp_fraction 换算器（发送时间戳编码、接收或参考时间戳编码、响应时间戳解码），算术流水与进位选择数据通路见图 7-3。

同步从端（Client）按配置周期 PERIOD 定期发起同步请求，记录发送时刻 t_1；时钟源端（Server）在收到请求时采样 t_2，发送应答时采样 t_3，并将请求的 Transmit 时间戳逐位

回显至应答的 Originate 字段；从端收到应答时记录本地时间戳 t_4。从端仅接纳 Originate 与本次请求严格相等的应答，随后计算链路双向时钟偏差：

$$\text{offset} = \frac{(t_2 - t_1) + (t_3 - t_4)}{2} \tag{7-1}$$

并将计算出的偏差累加至当前本地时钟。在一个同步周期内未收到有效应答时，下一周期以新的 Transmit 时间戳发起重试，直至同步完成。

发送时间戳在报文首字节与发送核心握手当拍由硬件精准采样，接收时间戳在核心交付首字节时瞬时锁存。时间戳在握手后通过专用算术流水换算，并在对应字段输出至物理链路前完成准备。

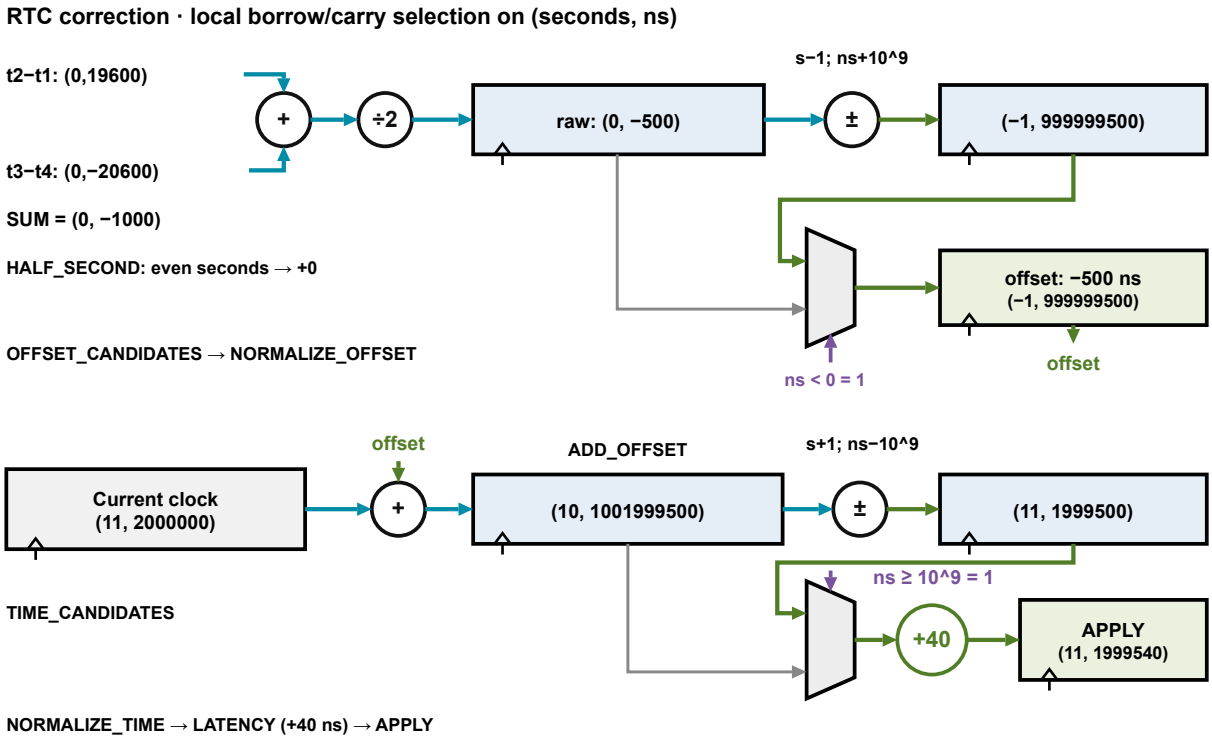


图 7-3 RTC 时钟同步算术数据通路：−500 ns 偏差的秒/纳秒借位候选、MUX 选择、时钟进位与 40 ns 流水补偿

服务端有待响应的请求或客户端同步定时器到期时，发送状态机立即启动。在无反压条件下，每个字节发送耗时 4 拍：TX_GROUPS 先在两组字段中各选一个 32 位字，TX_WORD 选定目标字，TX_BYTE 选出字节并置位 `tx_valid`，TX_HANDSHAKE 等待 `tx_ready`。发送单个 48 字节报文至少需 192 拍。Transmit 字段位于第 40–47 字节，距首字节握手至少相隔 160 拍，而硬件 NTP 小数换算仅耗时 65 拍，确保时间戳完全在飞（On-the-Fly）编码装配。服务端的 Receive 字段 t_2 在请求首字节到达时即启动换算，应答报文在整条请求校验通过后无缝启动回送。

7.2.2 时间换算与偏差校正

`afdx_ntp_fraction` 负责 32 位 NTP 二进制小数与本地纳秒整数之间的定点精确换算。纳秒编码为 NTP 小数时，采用 32 轮、每轮 2 拍的逐位非恢复余数除法计算

$\lfloor \mathrm{ns} \times 2^{32}/10^{9} \rfloor$；反向换算采用逐位相加与定点右移，并在末轮执行四舍五入。运算每拍至多执行一级 32 位加减，整数纳秒经往返换算严格实现零误差闭环。

秒差和纳秒差分别处理，每拍至多执行一级 32 位运算，步骤见图 7-3：先依次解码响应中的 t_2、t_3 小数部分（各 65 拍），再用 11 拍完成两段时间差、求和、除以 2 与写回，其中将奇数秒对应的半秒折算为纳秒，并通过 OFFSET_CANDIDATES、TIME_CANDIDATES 两个候选计算态拆开进位计算与结果选择。在 ADD_OFFSET 步中，将偏差校正量加到当前时钟值上；从该步到 APPLY 步写回共经过 5 拍，因此在 LATENCY 步由硬件精确补偿 40 ns。时钟同步计算期间，本地硬件 RTC 正常连续累加推进。

Client 正在等待响应且发送状态机空闲时，以响应的版本为 VN$=4$、模式为 mode$=4$ 且 Originate 与本次请求的 Transmit 逐位相等作为有效响应的充要判据，匹配后即启动时钟同步计算；迟到的旧响应因 Originate 不符而被自然过滤。计算期间或 `rx_done` 有效的周期不发起新请求。

7.2.3 报文字段与寄存器

请求与响应均为 48 字节，所有多字节字段高字节先发，见表 7-1。双方 UDP 源、目的端口均为 `UDP_PORT`（默认 123）。

表 7-1 SNTPv4 报文字段

字节	请求（Client）	响应（Server）
0	`0x23`：LI=0、VN=4、mode=3	`0x24`：LI=0、VN=4、mode=4
1	stratum=0	stratum=1
2–3	Poll=4、Precision=-26	同左（约 14.9 ns）
4–11	Root Delay、Root Dispersion 均为 0	同左
12–15	0	Reference ID=`LOCL`
16–23	0	最近一次配置装入的时刻
24–31	Originate=0	请求 Transmit 的逐位复制
32–39	Receive=0	t_2
40–47	Transmit=t_1	t_3

RTC 寄存器位于 `0x80`–`0x88`，见附录表 B-1。设置时间时，先写 `SEC`、`NS` 两个暂存字，再通过 `CTRL` 的装入位使两者同时生效；读 `SEC` 时同时锁存纳秒值，随后读 `NS` 得到同一时刻的值。偏差以 `OFFSET_SEC`$\times 10^9+$`OFFSET_NS` 纳秒表示。

7.3 SNMP 网络管理

SNMP 代理接收 UDP 目的端口为 161 的管理报文，响应以请求的源端口为目的端口，经配置的发送 VL 送出。系统采用模块化三层结构：顶层包装 `afdx_snmp`、协议核心引擎 `afdx_snmp_engine` 以及管理面共享发送输出缓冲 `afdx_mgmt_tx`。图 7-4 是其处理流程。

为了在 125 MHz 严苛时钟下实现深层 BER 编解码状态机的物理时序收敛，`afdx_snmp` 在顶层实现 `CE_DIV=2` 局部使能翻转树，并通过多路复制驱动将 `afdx_snmp_engine` 内部 47 个状态及其数据通路约束为 16 ns（2 周期）多周期路径；与此同时，高速 UDP

报文接收捕获、请求/响应 RAM 接口、配置总线访问以及 afdx_mgmt_tx 发送缓冲保持 125 MHz 单周期全速运行，既保障了 16 ns 宽松的协议解码建立时间，又避免了跨时钟域亚稳态风险与报文漏收。

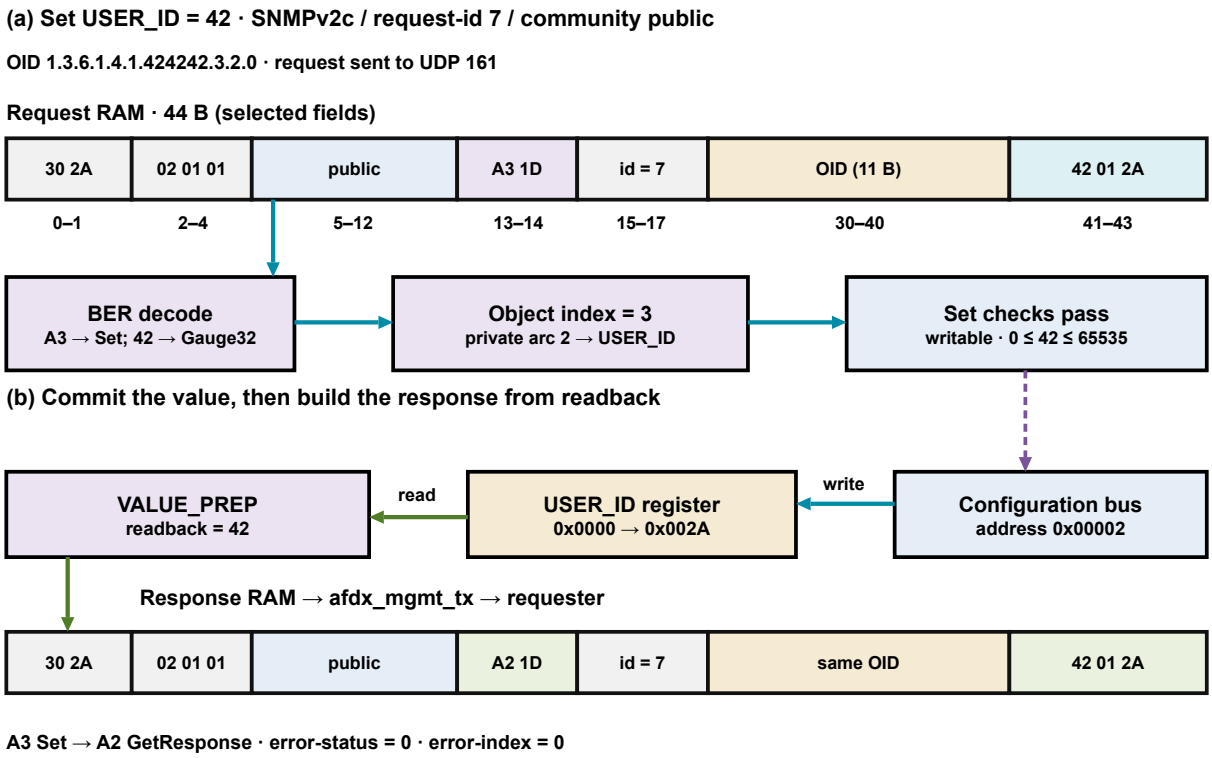


图 7-4 SNMP Set 示例：将 USER_ID 写为 42，从请求 RAM 解码，经对象查表与配置读回构造响应

7.3.1 协议对象与操作

MIB 包含 35 个只读对象与 1 个读写对象（USER_ID），共 36 个。对象分为 sysDescr、sysUpTime 两个系统对象，以及演示用私有子树 1.3.6.1.4.1.424242.3 下的 34 个对象，覆盖版本号、USER_ID、RTC 秒和同步次数、全部 27 个收发统计计数器、SNMP 请求数、Community（community）错误数和 Trap 发送数。对象值直接取自硬件寄存器。

代理支持以下协议版本和操作：

- 版本：SNMPv1（version=0）和 v2c（version=1）。
- **Get / GetNext**：Get 精确匹配标量 OID；GetNext 按子标识符逐个比较，返回严格大于请求 OID 的第一个对象，可从 1.3 遍历全部对象。对象或实例未命中时，v1 响应 noSuchName，v2c Get 统一响应 noSuchObject；v2c GetNext 遍历至末尾时响应 endOfMibView。
- **Set**：USER_ID 为读写配置对象，类型为 Gauge32，取值 0–65535。代理先检查 OID、类型和取值范围，再经配置总线写入并读回，以读回值组装响应。只读对象、类型不符和越界分别按 v1/v2c 的错误码响应，配置不变。
- **Trap**：向 UDP 端口 162 发送 SNMPv2-Trap（PDU 0xA7），携带 sysUpTime.0 与 snmpTrapOID.0 两个 VarBind。当 TRAP_ENABLE 已置位时，ENABLE 由 0 变 1 触发 coldStart，Community 校验不符触发 authenticationFailure。

每个请求只处理一个 VarBind。若请求的 Community 不符，代理静默丢弃而不发送响应，并递增 BAD_COMMUNITY 计数器。

7.3.2 请求解析与响应组装

代理的主状态机有 47 个状态，按功能分为 7 组。图 7-5 以 Set USER_ID 为 42 的同一请求，展示 OID 解码与设置状态的具体数据变化。请求先完整写入 256×8 的请求 RAM，解析与查找通过公共读取状态序列访问 RAM：原状态分支将返回位置写入 return_state，读取序列用 4 拍依次检查越界、发出读请求、等待同步读出并锁存数据，随后转回原状态分支。Fetch 状态组由解析、Lookup、Value 和 Encode 共用；每个解析状态只处理一个已就绪的字节，单拍逻辑只有一级比较。Set 未命中时经 Value 组的 SET_ERROR 编码错误响应，异常处理完成后返回接收状态。

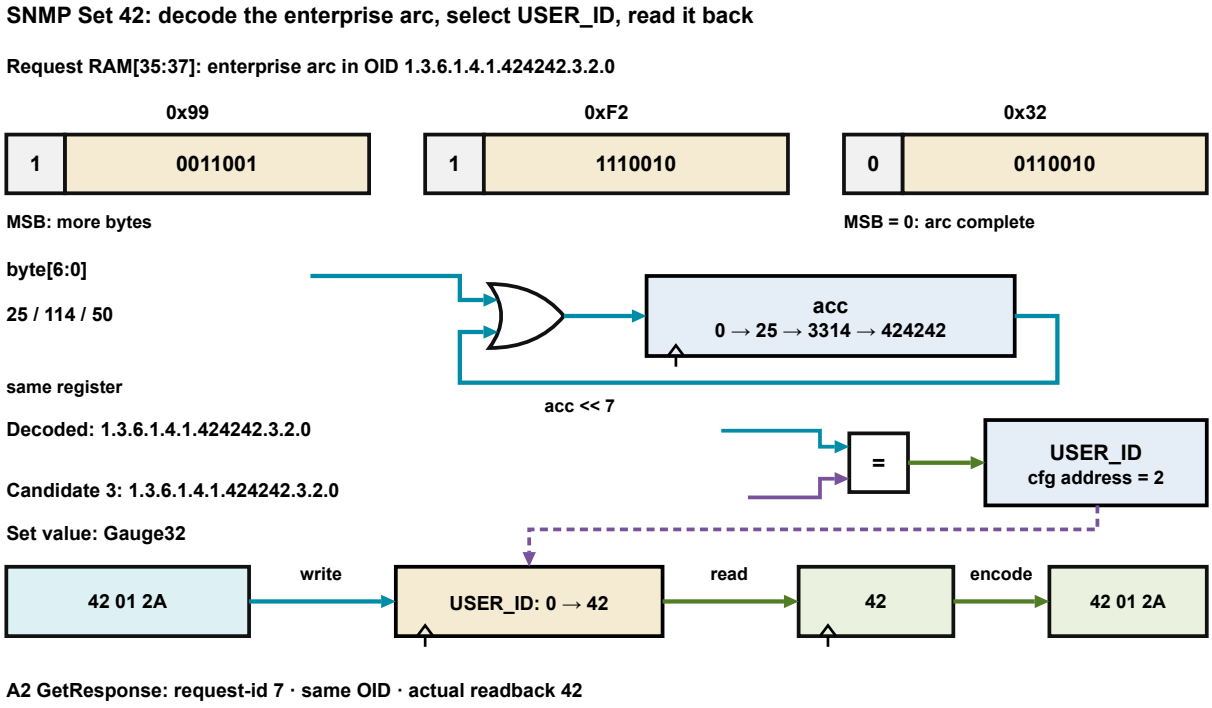


图 7-5 SNMP OID 的续字节位、7 位移位累加、对象比较及 Set 写后读回数据通路

代理按 BER 格式依次解析 SEQUENCE、version、community 和 PDU；PDU 内依次是 request-id、error-status、error-index 和 VarBindList。长度字段支持短形式（Short-form Length）与长形式（Long-form Length，0x81、0x82）。响应在 256×8 的响应 RAM 中逐字节组装，原样保留 request-id 和 community，再经服务发送口输出。

查找对象时，代理按对象表顺序依次检查 36 个候选，对每个候选按子标识符逐段比较：多字节子标识符按 BER 的 7 位分组累加还原成整数后比较数值，使 GetNext 按 OID 的数值顺序选择对象。

在 LEN1–LEN4 四拍中，代理自内向外计算值、VarBind、VarBindList、PDU 和整条消息的长度，随后在 BUILD 状态按“类型、长度、内容”逐字段写入响应 RAM。其中，community、OID 和回显值在 COPY 状态下从请求 RAM 逐字节复制。响应组装完成后，发送状态机逐字节读出数据，每字节用 4 拍送入服务发送口。

代理一次处理一个请求，只在 IDLE 状态接收新请求。处理期间到达的请求仍按端口匹配结果分流到代理，并计入请求数，但不产生响应，由管理端超时重发。超过 256 字节的请求整条丢弃。

7.3.3 配置访问与寄存器

代理通过与主机共享的配置读通路访问寄存器。主机访问不同地址时，优先处理主机请求；主机与代理读取同一地址时，将两者在同一拍合并。只有主机无读写请求且 `cfg_ready` 为 1 时，代理才获得写访问权。

SNMP 寄存器位于 `0x90–0xA3`，包括启用、响应 VL、community 长度和内容、请求计数、Trap 开关与计数及 sysDescr，见附录表 B-1。

7.4 ARINC 615A 软件数据加载

ARINC 615A 目标端采用严谨的三层微架构：顶层封装 `afdx_615a`、协议状态机核心 `afdx_615a_engine` 与共享发送输出缓冲 `afdx_mgmt_tx`。外部机载数据加载机（Data Loader）经 TFTP 发起单文件上传请求（LUR），目标端自动拉取头文件（LUH）和目标数据文件，完成硬件 CRC 校验后将数据存入片上 8 KiB 高速加载区，并通过状态文件（LUS）实时报告传输进度与结果；目标端同时支持介质定义下载（Media-Defined Download, MDD），将加载区内容作为文件 `MEMORY.BIN` 回传加载机。协议文件版本默认采用 `A4`，向下兼容 `A3`；目标端硬件标识为 `DEMC3_1`。

`afdx_615a` 顶层例化了专用时钟使能树（`CE_DIV=2`），将包含约 70 个状态的核心状态机 `afdx_615a_engine` 隔离在 16 ns 多周期路径中，使复杂的协议会话与校验逻辑完全免于 8 ns 单周期关键路径约束；对外报文捕获与发送输出缓冲保持 125 MHz 全速运行。

目标端重点实现单文件上传与介质定义下载（Media-Defined Download, MDD）两项核心操作；其余操作（如信息查询操作 LCI 与操作员定义卸载 LNO）在初始化阶段直接返回拒绝状态码。底层传输遵循 RFC 1350 标准 TFTP 协议，采用固定 512 字节块长：请求报文按操作码及以零字节定界的文件名、模式字段解析；会话期间的数据与 ERROR 报文均按目的 TID 关联对应传输通道。

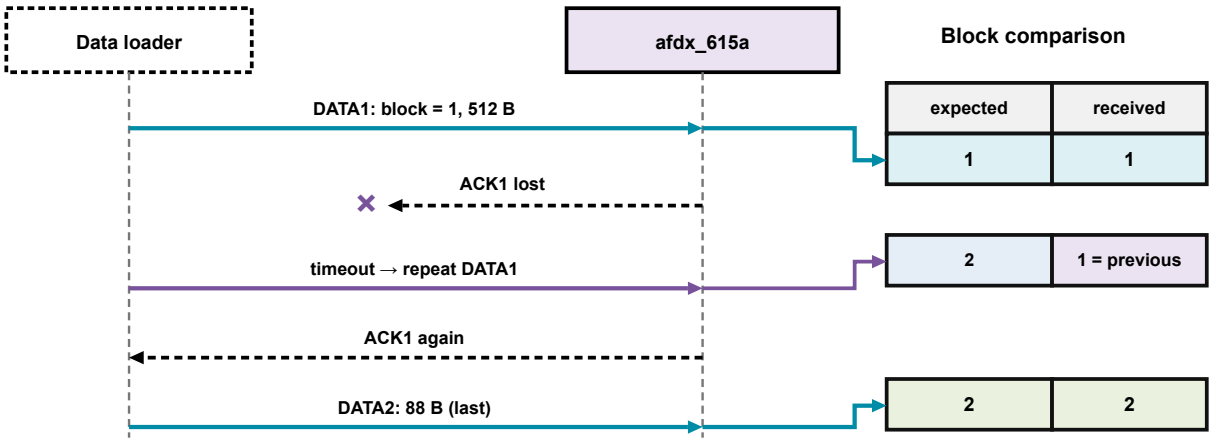
7.4.1 报文分派与异常处理

`afdx_615a` 的主状态机包含报文分派、文件处理与状态报告等状态组，图 7-6 展示重复数据块的关键分派路径。收到的报文先整条存入 1 KiB 的接收缓冲，由分派状态 DISPATCH 按操作码、端口（TID）和块号分派给六类处理过程。文件处理包含 List、Header、Data、Download 和 Status 过程；结束处理完成后返回等待状态。

各处理过程共用字节读取、整数拼接与报文输出状态序列。读取字节时，用 `return_state` 保存返回位置；读取整数时，另用 `num_return` 保存字段处理的返回位置。输出报文时，先在发送 RAM 中逐字节组包，发送后根据返回状态寄存器 `after_tx` 转移，通常返回等待报文的 `WAIT_PACKET` 状态。这些公共序列使各处理分支共用相同的同步读与发送时序。

数据或文件检查失败时，状态机锁存错误码 reason 并转入 FAILURE 状态，依次发送 TFTP ERROR 和失败状态文件；状态文件传输失败（含重试耗尽）时，结束本次操作。请求名称或模式错误时只返回 ERROR；LCI/LNO 在初始化阶段拒绝。

(a) Lost ACK1: resend the block, never append it twice



(b) DATA_DISPATCH selects copying or acknowledgment only

Arrival	FSM action	RAM write	loaded	CRC (hex)
DATA1	DATA_COPY	[0:511] ← 512 B	0 → 512	56EE
repeat DATA1	ACK → WAIT_PACKET	none	512 → 512	56EE
DATA2	COPY → file check	[512:599] ← 88 B	512 → 600	B166

DATA1 progress status completes before ACK1; final length / CRC checks precede ACK2.

图 7-6　615A 的 ACK1 丢失示例：重复 DATA1 仅重发确认，加载长度与 CRC 均不再次更新

接收缓冲一次保存一个报文。当前报文未取走时，新到报文仍按端口匹配结果分流到服务，但不写入缓冲；对端未收到响应时，按超时机制重发。

7.4.2　TFTP 传输与重传

TFTP 使用 RRQ、WRQ、DATA、ACK、ERROR 五种报文，采用 octet 模式，块长为 512 字节，块号从 1 开始；WRQ 以 ACK0 建立传输。请求目的端口为 REQUEST_PORT（默认 59），建立传输后双方使用各自的传输标识符（TID），目标 TID 从 0x8100 递增。主文件与状态文件分别保存 TID 和块号。

发送报文保存在 RAM 中，直至收到响应；超时（默认 2 s）后，原样重发上一报文。重传上限默认为 1 次；重传次数耗尽后，判定传输失败，错误原因码为 5。收到重复 DATA 时，只重发 ACK，不重复写入 RAM。

7.4.3　上传与文件校验

上传流程见图 7-7(a)：

1. 加载端读取 DEMC3_1.LUI，目标端返回接受状态 0001。
2. 目标端写入 DEMC3_1.LUS 报告初始状态。

3. 加载端写入上传请求文件 `DEMC3_1.LUR`（含头文件名和件号）；协议版本不支持时拒绝请求，错误原因码为 1。
4. 目标端发送 LUS `0002` 后读取请求文件中的 LUH 头文件；头文件校验通过后读取其指定的数据文件。
5. 数据块按序写入加载区并流式累计 CRC-16。首个满块写入后暂缓 ACK，先经独立会话发送一次进度 LUS，再确认该数据块。
6. 末块（短块或空块）到达后，仅当长度和 CRC-16 均正确时才予以确认，并发送 LUS `0003`（进度 100%）；否则发送 TFTP ERROR 和失败状态 LUS `1003`。

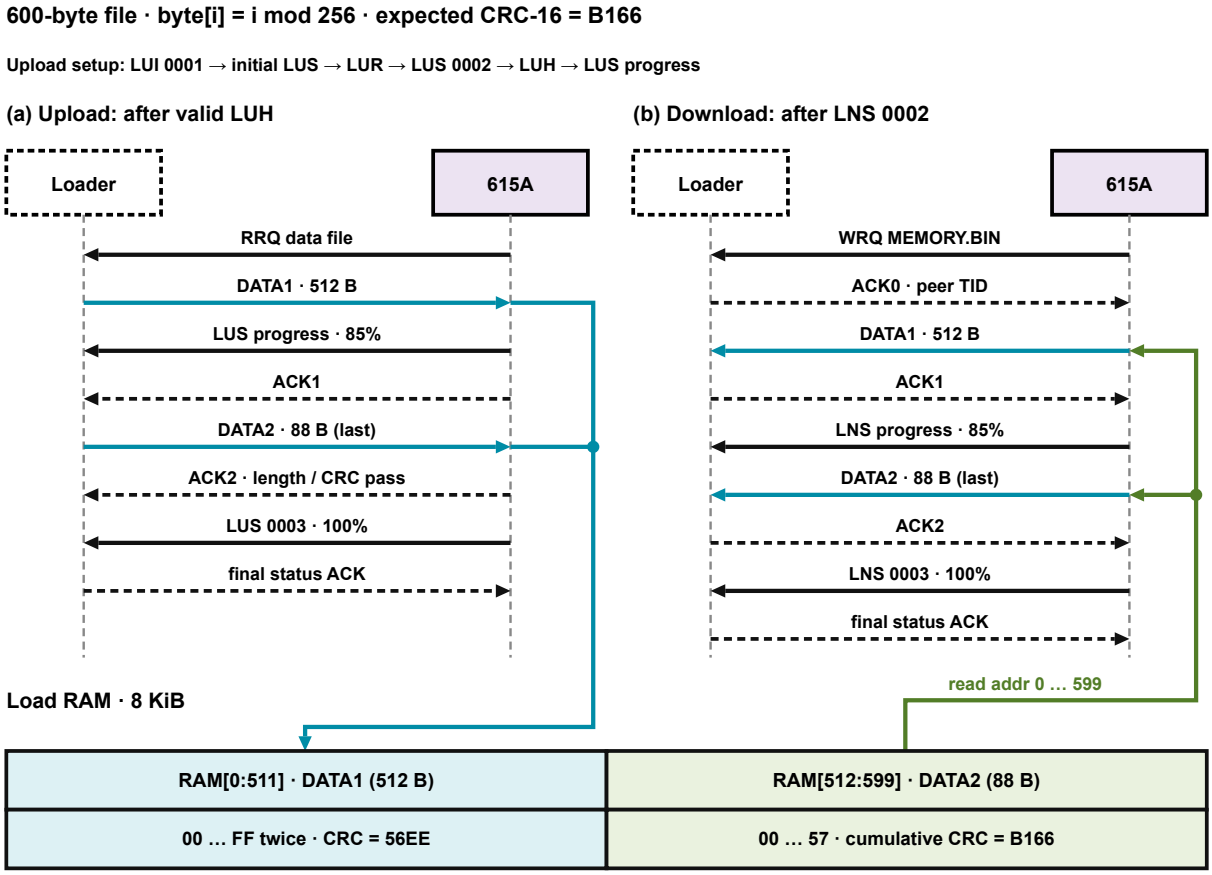


图 7-7　615A 的 600 字节文件示例：上传写入加载 RAM，下载从同一 RAM 读出；图示两块 DATA、ACK 与进度及最终状态文件的先后，省略状态文件内部的常规握手

两种传输的末块均少于 512 字节；文件长度为 512 字节的整倍数时，补发零长度 DATA 块。上传在 LUH 校验完成后先报告进度，再请求数据文件。

LUH 解析支持 ARINC 665-1 的 `8002` 格式版本：目标端依次提取头文件字数、数据文件列表指针、数据文件数、文件名、数据文件字数和两级 CRC-16 字段，对头文件实际长度、格式版本、单文件列表数量、头文件 CRC-16 以及目标数据文件的字节长度和流式 CRC-16 执行完整校验。

CRC-16 的多项式为 `0x1021`，最高位优先、初值 `0xFFFF`、结果不取反，以 8 位并行的组合函数每拍处理一字节。

每生成一份新 LUS，计数加 1，重传时不增加计数。进度按已写入字节数占预期总字节数的整数百分比计算，成功前上限为 99。计算时先由移位相加完成乘 100，再以预

期总字节数为减数逐拍迭代求商，最后通过减 10 迭代分离十位与个位，转换为十进制 ASCII 字符。

7.4.4 介质定义下载（Media-Defined Download, MDD）

介质定义下载流程见图 7-7(b)。加载机读取 `DEMC3_1.LND`，目标端响应后发送初始状态文件 LNS。随后，加载机发送下载请求文件 LNR，目标端校验并仅接纳其中的 `MEMORY.BIN` 请求。目标端发送 LNS `0002` 后，按 512 字节块采用停等机制主动回传最近一次成功上传的数据内容；首个满块获确认后上报传输进度，末块获确认后发送最终状态文件 LNS `0003`。下载操作仅以只读方式访问加载区，不影响既有加载数据状态。

文件上传与下载操作严格互斥，会话在最终状态文件（LUS/LNS）获得 ACK 确认后方判定为成功；新一轮上传启动时自动撤销前次的有效长度。片内规划 8 KiB 高速 RAM 作为加载数据暂存区，既支持主机经由配置总线按字节读回，亦可用于介质定义下载（MDD）文件回传。

7.4.5 存储与寄存器

协议 RAM 共有 6 块，均以 `afdx_ram_sdp` 例化：接收报文（1 KiB）、发送与重传报文（1 KiB）、LUR（512 B）、LUH（2 KiB）、加载数据（8 KiB），以及第 6 块专门用于存储拒绝原因文本的 `denial_ram`（32 项 $\times$ 8 bit = 256 bit）。在生成拒绝响应时，状态机直接从 `denial_ram` 逐字节流式读出预置的拒绝说明字符串，替代了原有的宽位组合多路选择逻辑，大幅削减了物理级数。

寄存器 `0xB0`–`0xBB` 见表 B-1。`STATUS` 报告空闲、进行中、成功、失败及原因码：0 无错误；1 元数据不支持或无效（版本、文件名、数量）；2 容量或长度超限；3 CRC 错；4 对端 TFTP ERROR 或模式错误；5 重传耗尽。主机通过 `MEMORY_ADDRESS` 选择加载区字节，再从 `MEMORY_DATA` 读回。

第 8 章　配置、复位与统计

`afdx_cfg` 负责配置总线的地址译码、配置寄存器和 27 个统计计数器的维护、读数据多路选择，并在配置表写入时与收发核心执行许可握手。模块输出全部来自寄存器，结构见图 8-1。

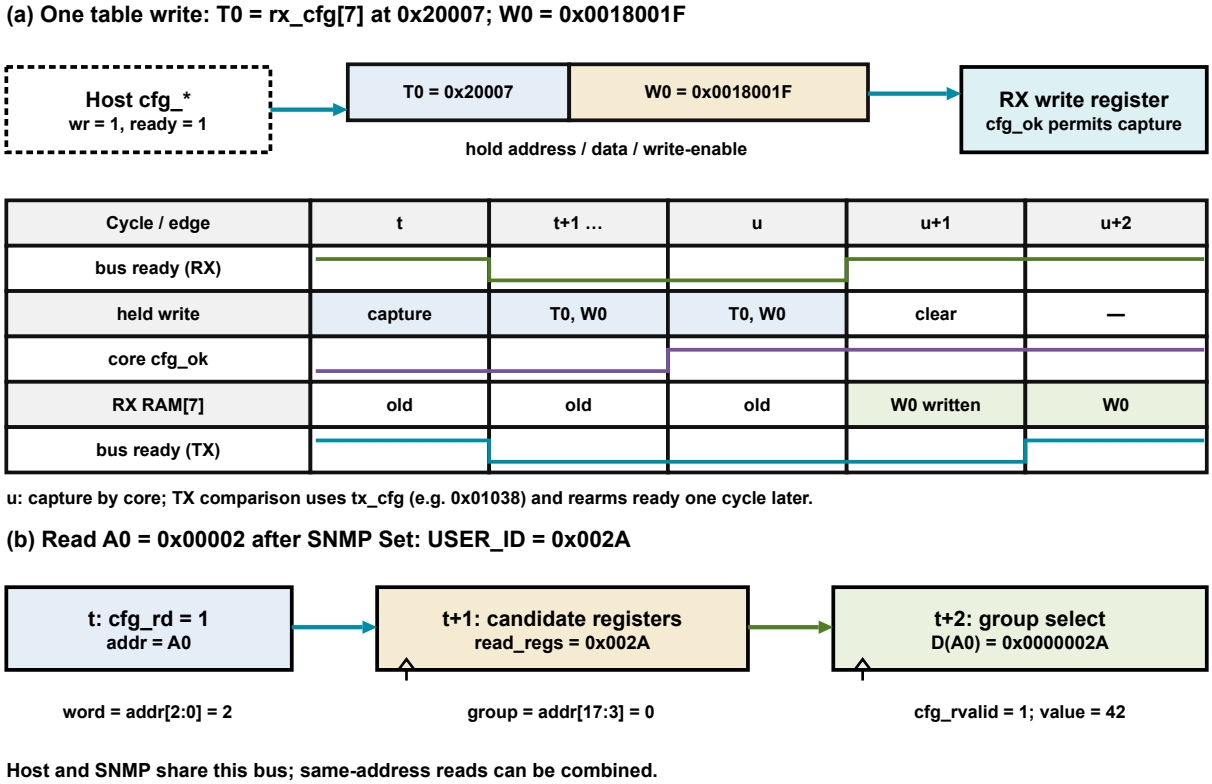


图 8-1　配置事务示例：(a) 暂存对 `rx_cfg[7]` 的写入，等待许可后写 RAM，并对比 RX/TX 的就绪重武装；(b) 读回 `USER_ID`=42 的两拍候选锁存与选择

8.1　地址映射

配置地址为 18 位字地址，映射见表 8-1。各条 VL 通过自己的 `en` 位控制启用。对表外地址的写请求予以忽略，读请求返回 0。

三张表按地址高位译码：地址位 17:10 为 `0000_0100` 时选中 `tx_cfg`，位 17:16 为 `01` 时选中 `rx_map`，位 17:12 为 `10_0000` 时选中 `rx_cfg`。

表 8-1　配置地址映射

地址	读写	内容
0x00000	只读	VERSION，0xC3110000
0x00001	只读	STATUS，位 0 为 tx_ready
0x00002	读写	USER_ID[15:0]，填入源 MAC 第 9–10 字节
0x00040–0x00046	只读	发送计数器：accept、drop_dis、drop_full、drop_long、drop_nopage、tx_a、tx_b
0x00048–0x0004F	只读	A 网接收计数器，对应 ev_a 位 0–7
0x00050–0x00057	只读	B 网接收计数器，对应 ev_b 位 0–7
0x00058、0x00059	只读	rm_drop、deliver
0x0005A、0x0005B	只读	a_gdrop、b_gdrop（保留，恒为 0）
0x00080–0x000BB	—	RTC、SNMP、615A 寄存器（附录表 B-1）
0x01000 + VL×8 + 字号	只写	tx_cfg
0x10000 + VLID	只写	rx_map，数据取位 12:0
0x20000 + idx	只写	rx_cfg，idx < 4096

8.2　配置表格式

三张配置表的字段见表 8-2。复位后三张表全部清零，即全部 VL 未启用、全部 VLID 未映射。

表 8-2　配置表字段

表与字	位	字段	含义
tx_cfg 字 0	0	en	VL 启用
	1、2	net_a、net_b	从 A、B 网发送
	8:3	qmax	队列最多排队的消息数，0–63
	19:9	max_payload	最大消息长度，1–1471 字节
tx_cfg 字 1	31:0	bag	BAG，单位为拍，1 ms = 125000，不超过 2^{30}
tx_cfg 字 2	15:0	VLID	目的 MAC 低 16 位
tx_cfg 字 3、4	31:0	源 IP、目的 IP	IPv4 地址
tx_cfg 字 5	31:16、15:0	源端口、目的端口	消息不带端口时使用的默认 UDP 端口
tx_cfg 字 6、7	—	保留	写 0
rx_map[VLID]	12	valid	该 VLID 已映射
	11:0	idx	接收 VL 索引，0–4095
rx_cfg[idx]	0	en	VL 启用
	1、2	net_a、net_b	接收 A、B 网的帧
	3、4	ic_en、rm_en	启用完整性检查、冗余管理
	16:5	skew_max	两网最大到达时间差，单位 32.768 µs
	27:17	max_payload	最大载荷，1–1471 字节

8.3　配置读写与仲裁

读数据的多路选择分两拍完成，读延迟固定为 2 拍：第 1 拍按地址低 3 位从四组数据源（全局寄存器、发送计数器、A 网计数器、B 网计数器）及三项服务寄存器中并行预选出候选字并锁存，同时按地址位 17:3 锁存组号；第 2 拍按组号从各路寄存值中选出目标字送至 `cfg_rdata`，并拉高 `cfg_rvalid`，每拍至多经过一级 8 选 1 逻辑。

`cfg_ready` 为 1 时，总线在当拍接受 `cfg_wr` 写请求：普通寄存器和服务寄存器直接写入；配置表写请求则先装入暂存寄存器并拉低 `cfg_ready`，等待对应核心给出 `cfg_ok`（第 4.7 节）。`tx_ready` 为 0 时，配置表写暂存寄存器保持清零，`cfg_ready` 为 0，不接受表初始化期间的写请求。

SNMP 代理读写管理对象使用同一条总线。顶层的仲裁规则是：主机在本拍写入，或主机与代理读取不同地址时，优先处理主机请求；主机读与代理读同一地址时合并为一次读，两者各自在 2 拍后取得数据；代理写只在主机无读写且 `cfg_ready` 为 1 时授予总线访问权。主机与代理的读数据分别由各自的 2 拍有效标记对齐。

8.4　配置写与查表的冲突仲裁

配置表的读口归收发引擎，写口归配置总线。配置写通过 `cfg_ok` 与引擎读请求互斥，避免同步 RAM 同拍同址读写冲突。各引擎的许可条件为：

$$\text{发送侧：}\quad \text{cfg_ok} = \text{enable} \wedge (\text{op} = \text{IDLE}) \wedge \neg\text{want_enq} \wedge \neg\text{want_grant} \tag{8-1}$$

$$\begin{aligned}\text{接收侧：}\quad \text{cfg_ok} = \text{enable} &\wedge (\text{op} \in \{\text{IDLE}, \text{AGE}\}) \\ &\wedge \neg\text{desc_a_valid} \wedge \neg\text{desc_b_valid}\end{aligned} \tag{8-2}$$

`cfg_ok` 由引擎自身的状态寄存器与同级模块的寄存输出（入队请求、帧描述有效、`grant_ready`）组合生成。`afdx_cfg` 接收一次写表请求后，保持写使能（`tx_cfg_we`、`rx_map_we` 或 `rx_cfg_we`）及地址、数据；当对应核心的 `cfg_ok` 为 1 时，引擎本拍不置位对应配置表的读使能寄存器，核心模块在该拍把写请求锁存到写口寄存器并于下一拍写入 RAM，`afdx_cfg` 撤销写请求。接收侧在许可后的下一拍恢复 `cfg_ready`；发送侧通过 `tx_cfg_ok_q` 与 `tx_cfg_rearm` 再延后一拍恢复就绪，保证本地应答与真实写请求完成对齐。时序为：

$$\begin{aligned}&\text{cfg_ready}(t+1) = 0, \qquad u = \min\{\,\tau \ge t+1 : \text{cfg_ok}(\tau) = 1\,\},\\ &\text{cfg_ready}_{\text{RX}}(u+1) = 1, \qquad \text{cfg_ready}_{\text{TX}}(u+2) = 1.\end{aligned} \tag{8-3}$$

其中 t 为总线接受写请求的时刻，$u - t$ 为写表等待时间，上界为 64 拍（第 3.7 节）。

8.5　复位与初始化

复位与表初始化过程见第 3.4 节。配置模块复位后，各寄存器恢复默认值，27 个计数器清零。`tx_ready` 为 0 期间，配置表写暂存寄存器保持清零，`cfg_ready` 保持为 0；此时不接受配置写请求，读访问照常进行。

8.6 统计计数器

27 个计数器均为 32 位，计满后回绕至 0，仅在复位时清零。为隔离发送核心与接收核心长距离跨模块引入的时序关键路径，`afdx_cfg` 对来自 TX/RX 通路的 5 路事件统计脉冲（`ev_tx`、`ev_a`、`ev_b`、`ev_rm_drop`、`ev_deliver`）插入了两级流水寄存（`_p` 与 `_q`）后再驱动 32 位加法器，降低了跨芯片远距离脉冲直接驱动大位宽累加器的布线延迟。

计入事件输入两级流水打拍（`_p` 与 `_q`）及读数据通路的两级流水结构，在 t 拍发出配置总线读请求，$t+2$ 拍返回对应事件脉冲截至 $t-3$ 拍在核心源头产生的累计次数（对应配置模块内部计数值在 $t-1$ 拍及以前的累加状态）。计数器分为：

- 发送（**7** 个）：接纳的消息数；因 VL 未启用、队列满、超长、无空闲页而丢弃的消息数；A、B 网发出帧数。
- 接收，每网 **8** 个：正确帧、FCS 错误、帧长错误、帧头错误、未知 VL、配置不允许、完整性检查丢弃、溢出。
- 接收，两网合计 **2** 个：冗余管理丢弃、交付。
- 保留 **2** 个：`a_gdrop`、`b_gdrop`，地址保留，恒为 0。

8.7 异常处理汇总

表 8-3 汇总全设计对各类异常的检测位置、处理方式和统计。发送侧拒收消息后，先回收已分配的页，再继续接收并丢弃剩余字节。

表 8-3 异常处理汇总

异常	检测位置	处理	统计
发送			
VL 或两网均未启用	`tx_engine`	拒绝入队，回收已用缓存页	`drop_dis`
队列已满	`tx_engine`	同上	`drop_full`
超过 `max_payload` 或 1471 字节	`tx_engine`、`tx_buf`	同上；继续接收并丢弃剩余字节	`drop_long`
无空闲页	`tx_buf`	同上	`drop_nopage`
MAC 发送下溢（TX Underrun）	`mac_tx`	发送未取反的 FCS，使对端判为错误帧，继续接收并丢弃本帧剩余字节	不计 `tx_a`/`tx_b`
接收（每网独立）			
前导码错误、帧长越界	`mac_rx`	整帧丢弃	帧长错误
FCS 错误	`mac_rx`	整帧丢弃	FCS 错误
固定字段、长度、分片或校验和错误	`rx_parse`	回退预写入指针	帧头错误
环形缓冲剩余空间不足	`rx_parse`	不写入本帧	溢出
VLID 未映射	`rx_engine`	回退预写入指针	未知 VL

接下页

表 8-3 （续）

异常	检测位置	处理	统计
VL 或本网未启用、超过 max_payload	rx_engine	回退预写入指针	配置不允许
SN 不连续	rx_engine	回退预写入指针，PSN 仍更新为本次 SN	完整性检查丢弃
另一网已交付的副本	rx_engine	回退预写入指针	rm_drop
交付队列已满	rx_engine	回退预写入指针	溢出
表初始化完成前开始的帧	rx_parse	整帧忽略	不计
服务与配置			
SNMP community 不符	afdx_snmp	不响应；Trap 启用时发 authenticationFailure	BAD_COMMUNITY
SNMP 请求超过 256 字节或代理忙	afdx_snmp	端口匹配后丢弃，由管理端超时重发	忙时计入 REQUESTS
615A 数据/文件检查错、状态传输失败、请求错误	afdx_615a	数据/文件错：ERROR 及失败状态文件；状态传输失败（含重试耗尽）：直接结束；名称/模式错：仅 ERROR；LCI/LNO：初始化拒绝	STATUS 原因码 1–5（第 7.4 节）
SNTP 响应版本、模式或 Originate 不符	afdx_rtc	丢弃，下一周期重发请求	—
表初始化期间写配置	afdx_cfg	cfg_ready= 0，不接受	—
未映射的地址	afdx_cfg	忽略写请求，读出 0	—

8.8 主机编程流程

8.8.1 初始化与启用

主机经配置总线按下列顺序使端系统进入工作状态。每次写表都要等 cfg_ready 为 1（第 4 章）。

1. 初始化：释放复位后轮询 STATUS，等待位 0（tx_ready）变为 1，约 524 µs。
2. 端系统标识：写 USER_ID。
3. 发送 **VL**：对每条发送 VL 先写字 1–5（BAG、VLID、IP、默认端口），最后写字 0，同时设置 en、网络选择、qmax 和 max_payload。
4. 接收 **VL**：对每条接收 VL 先写 rx_cfg[idx]，再写 rx_map[VLID]= {1, idx}，使映射生效时配置已经就绪。
5. 服务：为每项服务配置一条发送 VL（目的 IP、端口与服务端口一致），写服务的 TX_VL、端口和 community 等寄存器，最后写 ENABLE。
6. 开始收发：应用开始提交消息。VL 较多时按第 3.7.3 节错开各 VL 首条消息的提交时刻。

8.8.2 运行中更新与服务配置

tx_cfg 的 en 仅在入队时检查；已排队的消息继续参与授权，授权阶段逐拍读取字 0–5。修改一条发送 VL 时，主机应先停止向它提交消息，再把字 0 的 en 写 0，并保持该字其余字段不变；待该 VL 已提交的消息全部发出后，修改其余字，最后重新启用。

通过 cfg_ok 保证配置写与表读互斥，多字配置按字依次生效。接收侧的 rx_cfg 和 rx_map 各自通过单拍整字写入更新；两表的写入分别生效，历史状态在配置更新后保留。

应用与服务的配置还须满足以下条件：

- 应用须等待表示收发核心均已就绪的 tx_ready 有效后，再提交消息。app_tx_ready 由应用仲裁选中条件与发送核心的 m_ready 相与生成。
- 各服务的 UDP 端口须互不冲突；RTC 的模式与时间，以及 SNMP 的 ENABLE、community 和 TX_VL，仅在对应服务没有在途消息时修改。
- 615A 忙时，忽略对启用、发送 VL、请求端口、超时、重传上限和文件版本的控制写请求；写 0xB4 后至少等待 4 拍，且只在非忙状态下读 0xB5。

8.9 配置约束

主机须检查各项配置的组合是否满足带宽、抖动、缓存和交付要求。具体约束及超限处理如下。

- 带宽：A、B 两网由同一个 MAC 状态机同拍驱动，只发 A 网或只发 B 网的帧同样占用一个发送时隙，所以约束针对全部发送 VL，不分网络：$\sum_v C(n_v)/\mathrm{BAG}_v \le 1$，其中 n_v 为 max_payload，$C(n_v)$ 为第 3.7 节式 (3-1) 的线路占用拍数，服务 VL 计入求和。超出该约束时，各帧只能串行发送出，实际发送间隔将大于 BAG。
- 抖动：BAG 取 2^k ms；错开各 VL 首条消息的提交时刻，使任一 VL 之前排队发送的帧的线路占用与等待之和按式 (3-3) 不超过 5828 拍（50 µs）。
- 缓存：缓存页在组帧器读完后才回收，而 q_count 在授权时已减 1；正在接收的消息也会在入队判定前分配缓存页。因此，除各队列中的消息外，还须计入组帧器持有的两帧（下一帧与正在发送的帧），以及 tx_buf 正在接收的一条消息，每帧或每条消息至多占 3 页。满足 $\sum_v \mathrm{qmax}_v \times \lceil n_v/512 \rceil + 9 \le 256$ 时，256 页即可容纳上述消息同时驻留时的最大需求；超出时，因无空闲页而丢弃的消息计入 drop_nopage。256 页按每条最长消息占 3 页计算，最多驻留 85 条最长消息。
- 交付：交付口每拍一字节。若某条 VL 关闭冗余管理，且同时接收两网数据，则每条消息交付两次。所有接收 VL 的载荷交付总速率须低于 1 字节每拍；两网独立满速且都要交付时，所需交付速率超过该上限，超出部分按溢出计数。
- 顺序与冗余窗口：跨 VL 的发送顺序由各 VL 的 BAG 与候选选择共同决定；提前选中的候选可能使后到的小帧等待。接收副本的到达间隔超过 skew_max 时，冗余记录按过期处理，该副本可能再次交付。
- 取值范围：bag $\le 2^{30}$ 拍；qmax ≤ 63；max_payload 为 1–1471；skew_max 小于过期清理门限 0x4000 个计时单位（12 位字段自然满足）。

第 9 章　时序与存储设计

9.1　时延预算与流水账本

端到端时延从发送端应用接口成功提交消息末字节时（m_{last} 与 `m_valid && m_ready` 握手完成）起算，至接收端应用接口完成末字节交付时（d_{last} 有效）结束。按照官方 1022 答疑 R97 口径，端到端时延严格分解为发送处理段 L_{tx}、网络链路传输段 L_{gmii} 与接收处理段 L_{rx}。

在收发通路无其他消息争用、当前 VL 已满足 BAG 最小发送间隔的基准条件下，表 9-1 详细列出 1471 字节满载帧（对应最大以太网帧 1518 字节）在各子模块流水中的逐拍开销与时间预算：

1. 发送处理流水：`afdx_tx_buf` 载荷 RAM 写口增加 1 拍流水寄存，写描述符并向调度引擎发起入队请求需 3 拍；`afdx_tx_engine` 入队判定需 6 拍、帧授权需 9 拍；`afdx_tx_builder` 帧头组装与 IP 校验和准备需 11 拍；核心输出端插入的 1 级流缓冲与双网 MAC 启动共 2 拍。从 m_{last} 到 GMII 首个前导码字节 `tx_sof` 固定为 32 拍（0.256 μs，原基线为 31 拍 / 0.248 μs）；若按 R95 规则计至以太网帧起始定界符（SFD，`8'hD5`，前导码第 8 字节）结束，发送处理时延 L_{tx} 典型为 39 拍（0.312 μs）。
2. 物理链路传输：前导码首字节至 FCS 末字节共 $8 + 1518 = 1526$ 字节，在线路上占 1525 拍（扣除与发送首拍重叠的 1 拍），折合 12.200 μs。直连仿真模型下 SFD 间的链路延迟 $L_{\text{gmii}} = 0.000$ μs。
3. 接收处理流水：接收 MAC 检测帧尾需 3 拍；`afdx_rx_parse` 帧尾分类需 3 拍；多 Bank `rx_map` 增加 2 拍读回流水，接收引擎执行 VL 映射、配置/状态查表、完整性检查、冗余管理与入队共需 14 拍；交付模块读取块 RAM 队头并自环形缓冲读出首字节需 6 拍。从 FCS 结束到首个交付字节 `d_valid` 由 24 拍更新为 26 拍（0.208 μs）。接收段首字节时延 L_{rx} 为 12.360 μs，末字节时延 $L_{\text{rx,last}}$ 为 24.120 μs。
4. 逐字节交付：交付模块每拍交付一个有效字节，1471 字节载荷从首字节到末字节需 $n - 1 = 1470$ 拍（11.760 μs）。

无争用基准下端到端总时延为 3053 拍（24.424 μs）；当接收端引擎在收到帧描述的当拍恰好处于 AGE 状态而让出 1 拍时，时延为 3054 拍（24.432 μs），这与性能仿真实测的最大端到端时延 24.432 μs 完全一致。

表 9-1　端到端时延流水账本（1471 字节满载消息）

阶段与区段	典型拍数	典型时间 (µs)	上界时间 (µs)	详细流水构成与硬件依据
发送写缓存与入队	4	0.032	0.080	载荷写寄存 1 拍，写 `tx_desc` 并置 `enq_valid` 3 拍
调度引擎入队与授权	15	0.120	0.240	ENQ 状态 6 拍，GRANT 状态 9 拍
组帧准备与流缓冲	12	0.096	0.160	IP 校验和准备 11 拍，流缓冲双缓冲锁存 1 拍
MAC 起始响应	1	0.008	0.016	检测起始并在第 32 拍输出首前导码 `tx_sof`（0.256 µs）
前导码发送至 SFD	7	0.056	0.056	7 字节 `0x55` 占 7 拍，第 39 拍输出 SFD（$L_{\text{tx}} = 0.312$ µs）
以太网帧传输	1518	12.144	12.144	帧头 42 字节、载荷 1471 字节、SN 1 字节、FCS 4 字节
MAC 帧尾与解析分类	6	0.048	0.064	MAC 帧尾判定 3 拍，解析器 FIN 状态机两级分类 3 拍
分体映射与引擎判定	14	0.112	0.160	32 子库查表 3 拍，读 cfg/state 2 拍，IC/RM 判定与提交 9 拍
交付队头读取与流水	6	0.048	0.064	块 RAM 队头读出 2 拍，读控制与环形缓冲读出 4 拍
逐字节流式交付	1470	11.760	11.760	接收应用口每拍交付 1 字节，首字节至末字节（$n-1$ 拍）
无冲突硬件流水上界	3053	24.424	24.744	m_{last} 至 d_{last} 纯硬件流水上界代数和
叠加 AGE 老化扫描 1 拍	3054	24.432	24.752	接收端恰遇 4096 拍 AGE 扫描打断 1 拍，与实测吻合
计入最坏在途争用与调度	—	—	26.360	计入最坏 202 拍调度扫描与在途争用等待（$24.744 + 1.616$ µs）

9.2　片上 RAM 分配与物理存储规划

全系统共规划 21 张逻辑存储表，在物理实现中封装为 52 个 `afdx_ram_sdp` 简单双端口（SDP）同步 RAM 模块实例，综合后占用 366 个 EBR 原语（逻辑存储总容量 2,444,544 bit，约 298.4 KiB），见表 9-2。所有 RAM 均采用严格的一写一读 SDP 同步时序，消除异步读写风险。

表 9-2　片上存储器分配与物理映射

存储表名称	逻辑规模（深 × 宽）	逻辑容量（bit）	物理例化	架构分工与物理优化说明
发送通路（共 7 张逻辑表，7 个物理模块）				
`tx_payload`	131072×8	1,048,576	1	128 KiB 载荷缓冲（256 页 × 512 字节），写口增加流水寄存
`tx_cfg`	1024×32	32,768	1	128 条发送 VL 配置表（每条 VL 占 8 个 32 位字）

接下页

表 9-2　（续）

存储表名称	逻辑规模（深 × 宽）	逻辑容量（bit）	物理例化	架构分工与物理优化说明
tx_desc	256 × 44	11,264	1	消息元数据描述符表（长度、端口使能、源/目的 UDP 端口）
tx_state	128 × 63	8,064	1	发送调度状态（next_time、sn、q_count、队头/队尾）
tx_free	256 × 8	2,048	1	空闲页池 FIFO，支持双页面回收端口并发归还
tx_mnext	256 × 8	2,048	1	发送消息链表（同 VL 下一条消息首页号）
tx_pnext	256 × 8	2,048	1	消息内分页链表（同一消息下一页号）
发送小计	—	*1,106,816*	*7*	发送侧总存储容量约 135.1 KiB
接收通路（共 6 张逻辑表，37 个物理模块）				
rx_map	65536 × 13	851,968	32	物理拆分为 32 个 2048 × 13 子库，两级读流水，16 路配置总线复制
rx_state	4096 × 44	180,224	1	4096 条接收 VL 状态（双网 PSN、冗余时间戳与有效位）
rx_cfg	4096 × 32	131,072	1	4096 条接收 VL 配置字（使能、网络选择、skew_max、最大长度）
rx_ring_a	4096 × 8	32,768	1	A 网环形载荷缓冲（4 KiB），支持预写入与指针回退
rx_ring_b	4096 × 8	32,768	1	B 网环形载荷缓冲（4 KiB），支持预写入与指针回退
rx_dq	32 × 68	2,176	1	接收交付队列（迁入块 RAM，消除 32 项寄存器多路选择树）
接收小计	—	*1,230,976*	*37*	接收侧总存储容量约 150.3 KiB
网络服务（共 8 张逻辑表，8 个物理模块）				
snmp/req_ram	256 × 8	2,048	1	SNMP 请求报文缓冲（256 字节，支持单 VarBind 请求）
snmp/resp_ram	256 × 8	2,048	1	SNMP 应答与 Trap 组装缓冲（256 字节）
615a/load_ram	8192 × 8	65,536	1	615A 数据加载暂存区（8 KiB），支持主机读回与介质卸载
615a/header_ram	2048 × 8	16,384	1	615A LUH 头文件缓存区（2 KiB）
615a/pkt_rx	1024 × 8	8,192	1	615A TFTP 接收报文缓冲（1 KiB）
615a/pkt_tx	1024 × 8	8,192	1	615A TFTP 发送与超时重传报文缓冲（1 KiB）
615a/lur_ram	512 × 8	4,096	1	615A 上传请求文件（LUR）暂存区（512 字节）
615a/denial_ram	32 × 8	256	1	615A 拒绝原因文本表（消除状态机大位宽组合选择）
服务小计	—	*106,752*	*8*	网络服务总存储容量约 13.0 KiB

接下页

表 9-2 （续）

存储表名称	逻辑规模（深 × 宽）	逻辑容量（bit）	物理例化	架构分工与物理优化说明
全设计合计	—	**2,444,544**	**52**	逻辑总容量约 298.4 KiB，综合映射为 366 个 EBR（占比 30.50%）

全系统逻辑 RAM 存储容量为 2,444,544 bit（约 298.4 KiB）。在 EQ6HL130 目标器件中，经 eLinx 官方综合映射，共消耗 366 个嵌入式块 RAM（EBR），占器件全部 1,200 个存储块资源的 30.50%，片上存储资源裕量达 69.5%。

9.3 静态时序收敛与关键路径工程优化

在 125 MHz（8 ns 周期）工作频率下，端系统不仅要在单周期内完成每拍一字节的 GMII 吞吐，还要支持 4096 条接收 VL 的查表与深层协议解析。为确保端系统静态时序闭环（Timing Closure），系统在 RTL 微架构与物理实现协同层面确立了六项优化措施：

1. 管理面协议引擎 **CE_DIV** $= 2$ 多周期解耦：ARINC 615A 与 SNMP 协议包含深层状态转移分支。设计将高层协议核心状态机（`afdx_snmp_engine.v` 与 `afdx_615a_engine.v`）独立封装，通过 2 分频低扇出时钟使能树驱动，在时序约束中为内部使能路径施加 2 周期建立时间约束（16 ns 预算）；同时在顶层设计 `afdx_mgmt_tx` 流式预取适配模块，平滑桥接 16 ns 多周期路径与 125 MHz 单周期全速发送仲裁域。
2. **`rx_map` 32** 子库多 **Bank** 分块存储与配置总线复制：`rx_map` 逻辑容量达 65536×13 bit。若直接例化单体大模块，其地址译码、写使能树与宽位数据选择器将产生严重的布线拥塞。物理设计将其均匀划分为 32 个独立的 2048×13 bit 块 RAM 子库；配置总线引入 16 路低扇出驱动树（`RX_CONFIG_BANKS=16`），读出侧插入两级流水 MUX，有效化解了芯片长跨度连线的延迟瓶颈。
3. **`rx_dq`** 迁入块 **RAM** 消除寄存器阵列选择树：原交付队列采用 32 项 $\times$ 68 bit 寄存器数组，队头读取需经过 32 选 1 宽位组合逻辑。重构为 `afdx_ram_sdp` 块 RAM 后，交付模块通过微状态机（`HEAD_IDLE -> HEAD_SELECT -> HEAD_RELEASE`）按地址流式读出，消除了宽位选择树的逻辑级数。
4. **`afdx_tx_buf`** 写口寄存与输出流缓冲解耦：在分页写载荷端口插入 `payload_we_r` 流水寄存，切断应用输入到 RAM 写口的路径；在组帧器与双网 MAC 间插入包含乒乓双缓冲结构的 1 级流缓冲，并更新提前启动参数 `START_LEAD_REGISTERED = -6`，兼顾路径隔离与到期准点发送。
5. 服务仲裁与统计事件两级流水寄存：`afdx_svc` 对 4 路发送仲裁授权向量进行寄存输出（`served_r`）；`afdx_cfg` 对跨越核心长距离布线的 5 路统计脉冲插入两级寄存（`_p` 与 `_q`），消除了远距离信号直驱 32 位累加器的建立时间压力。
6. **`S2d`** 综合策略与 **`PRES1.50`** 物理布局布线协同：综合脚本启用 `S2d` 模式，保持模块层级边界与局部时钟使能树拓扑；布局布线工具配置物理优化强度为 1.50（`PRES1.50`），提升全局时钟网络与关键信号连线的布线收敛度。

表 9-3　综合与布局布线后全量静态时序签核指标

时序分析指标	官方综合估算值	布局布线后 STA 结果	签核标准与工程结论
分析主时钟周期	8.000 ns（125 MHz）	8.000 ns（125 MHz）	125 MHz 单一主时钟驱动
最差建立时间裕量（Worst Setup）	+4.459 ns	+0.013 ns	全量 45,745 个时序端点 0 违例
最差保持时间裕量（Worst Hold）	−0.068 ns	+0.229 ns	布局布线后保持时间完全收敛
最高工作频率（Fmax）	282.406 MHz	125.203 MHz	满足 125 MHz 线速运行要求
8 ns 单周期路径数 / 最差裕量	—	153,342 条 / +0.013 ns	最差路径位于收发核心内部控制
16 ns 多周期路径数 / 最差裕量	—	50,296 条 / +0.940 ns	覆盖 2,755 个管理面引擎寄存器
同沿保持时间路径数	—	203,638 条	全部正裕量，无保持时间违例
多周期约束合法性审计	—	0 非法 / 0 漏约束	经 `tools/check_multicycle.py` 审计

表 9-3 汇总了官方综合估算与布局布线后静态时序的闭环签核结果。在 S2d 综合策略与 PRES1.50 物理实现下，全芯片 45,745 个时序端点建立时间与保持时间违例数均为 0：153,342 条 8 ns 单周期路径的最差建立裕量为 +0.013 ns，50,296 条 16 ns 多周期路径的最差建立裕量为 +0.940 ns，保持裕量为 +0.229 ns，布线后等效最高工作频率为 125.203 MHz，满足 125 MHz 稳定运行的指标要求。

9.4　时序设计规则与流水划分

为使 4096 条 VL 的查表通路和逐字节数据通路满足严谨的工程规范，全部 RTL 严格遵守表 9-4 的十条时序组织规则。

表 9-4　时序设计规则

编号	规则内容与工程实现
1	全设计使用单一 125 MHz 全局物理时钟树，消除异步 CDC 亚稳态风险。
2	虚拟链路状态全部集中于块 RAM，通过地址顺序访问；各 VL 共用数据通路与算术逻辑。
3	所有块 RAM 统一通过 `afdx_ram_sdp` 例化，写使能、写地址和写数据均直接来自寄存器输出。
4	RAM 读出数据必须寄存后再做算术或比较；两者之间最多允许一级 4 选 1 选择逻辑。
5	复杂算术拆解到不同拍执行：RTC 时间戳换算每轮两拍迭代，615A 乘 100 采用移位相加分步求商。
6	大于 8 选 1 的多路选择逻辑必须分两拍两级流水完成。
7	字节流水线连续推进；RAM 读取与数据组装执行超前预读（例如组帧器提前 6 拍补偿启动流水）。
8	同一 RAM 的同拍同址读写严格互斥，写操作必须获得引擎的 `cfg_ok` 许可。
9	模块间通信以寄存器输出为主；跨模块长距离脉冲（如统计事件脉冲）必须打两拍流水。
10	管理面复杂协议状态机通过 `CE_DIV=2` 局部使能树解耦为 16 ns 多周期路径，实现全局时序闭环。

第 10 章　可验证性设计与实测摘要

10.1　验证体系与自动化回归策略

设计构建了涵盖 RTL 行为级仿真、自动化全量回归测试、性能与变长压力验证、综合后门级网表验证的多层次验证体系。验证平台采用多重断言判据：不仅核对测试平台控制台无差错打印，还通过独立的 Python/Tcl 参考模型对顶层端口的字节流记录与配置寄存器读回值进行端到端全量比对。

针对全设计 24 个 RTL 模块，回归测试套件共包含 39 项自动化测试用例（涵盖单模块存储/MAC/调度、系统双端对发、故障异常注入、4096 容量抽测、SNMP/615A/RTC 专项服务，以及变长多 VL 并发测试 `tb_sys_perf_var`），在 ModelSim SE-64 2020.4 环境下累计通过 2585512887 次自检断言与 3879565 项独立参考核对，全部用例通过。

10.2　性能与多虚拟链路变长压力实测

性能仿真基于理想 125 MHz 时钟（每拍 8 ns，GMII 每拍传输 1 字节）。吞吐率按接收端去重后的有效载荷计数（不计头部、填充及 A/B 冗余副本）；端到端时延按 $m_{\text{last}} \rightarrow \text{SFD} \rightarrow d_{\text{valid}}/d_{\text{last}}$ 分段测量；传输抖动严格按照官方 1022 答疑 R97-2 口径统计同一 VL 多帧端到端时延极差。

测试包含定长满载（`gbe77`）与四组多 VL 变长并发场景（`var_fixed`、`var_cycle`、`var_high`、`var_burst`），覆盖 1、17、256、1024、1471 字节等典型载荷，测试结果汇总于表 10-1。

表 10-1　多工况性能与分段时延实测结果（直连 0 拍链路）

测试工况	激活 VL 数	单向吞吐 (Mbps)	L_{tx} 最大 (µs)	L_{rx} 首/末 (µs)	L_{app} 首/末 (µs)	R97 抖动 (µs)
`gbe77`	77	906.136	0.312	12.360 / 24.120	12.672 / 24.432	0.008
`var_fixed`	32	57.794	0.360	12.360 / 24.120	12.672 / 24.432	0.008
`var_cycle`	32	70.403	12.216	12.352 / 24.112	12.936 / 24.424	9.992
`var_high`	80	904.752	12.016	12.360 / 24.120	23.872 / 24.432	0.008
`var_burst`	32	57.794	21.128	12.352 / 24.112	32.824 / 32.888	0.000

实测表明：

- 吞吐率：定长满载每方向吞吐为 906.136 Mbps（81 VL 极限容量为 953.208 Mbps），变长高负载（`var_high`）为 904.752 Mbps，满足赛题 100 Mbps 指标要求。
- 端到端时延：全部工况下端到端时延最大值不超过 34.840 µs（`gbe77` 稳态最大为 24.432 µs），满足 1–2 ms 时延上限要求。

- 抖动：定长稳态发送间隔偏差为 0 μs；在 R97 同 VL 时延极差口径下，定长满载为 0.008 μs，变长循环多帧轮转下极差为 9.992 μs。

10.3　代码覆盖率签核

对全工程 24 个 RTL 模块统一开启 ModelSim 代码覆盖率插桩（`-coverage` 参数，插桩类型包括语句 `s`、分支 `b`、条件 `c`、表达式 `e`、状态机 `f` 和翻转 `t`），回归测试平台不计入统计范围。39 项全量回归累计覆盖率如表 10-2 所示。

表 10-2　24 个 RTL 模块代码覆盖率汇总与未覆盖项分类分析

覆盖率统计类别	覆盖项数 / 总项数	覆盖率	统计口径与未覆盖项分类说明
语句覆盖率（Statement）	6981 / 7083	98.55%	未覆盖项主要为硬件异常超时与死锁保护熔断分支
分支覆盖率（Branch）	3844 / 3985	96.46%	覆盖正常收发、协议边界与 14 类故障注入处理分支
条件覆盖率（Condition）	610 / 717	85.07%	覆盖各核心多信号组合输入判定；未覆盖为极端故障并发组合
表达式覆盖率（Expression）	1760 / 1991	88.39%	覆盖关键数据通路表达式；未覆盖项为互斥使能判定分支
状态机状态覆盖率（FSM State）	110 / 110	100.00%	工具识别纳入统计的 110 个状态机状态全部覆盖
状态机转换覆盖率（FSM Transition）	138 / 308	44.80%	已覆盖 138 条主干与异常转移；未覆盖 170 条转移统计项主要包含防御性非法跳转与互斥分支
翻转覆盖率（Toggle）	41314 / 53632	77.03%	未翻转主要为静态配置高位、未用地址高位及测试专用引脚

全设计状态机状态覆盖率达 100%，语句覆盖率为 98.55%，分支覆盖率为 96.46%，满足设计验证要求。

10.4　综合后门级网表功能对照仿真

为了验证综合与布局布线工具未改变 RTL 的行为语义，工程基于官方 eLinx 导出的真实门级网表（VQM 格式，包含芯片厂商专用原语网表）开展了综合后门级功能回归仿真。

在五功能综合测试平台 `tb_net_services` 激励下，门级网表仿真通过 **1124** 项自检断言（0 错误），仿真耗时 **8793.406** 秒。测试平台导出的七份外部记录（包含用户消息交付内容、SNMP 读写数据、615A 加载与卸载字节、RTC 同步偏差等）与同激励 RTL 行为仿真记录逐字节完全一致，验证了综合网表在联合测试场景下的功能正确性。

10.5 硬件可观察性与可控性机制

设计在硬件架构层提供了全方位的调试与验证支持：

- 确定性事件脉冲端口：`ev_tx`、`ev_a`、`ev_b`、`ev_rm_drop`、`ev_deliver` 在对应事件发生后的确定拍数内输出单拍指示，支持外部逻辑分析仪抓取。
- 配置总线全透明读回：27 个硬件统计计数器与三项高层服务的所有内部寄存器均可通过 32 位配置总线在线读回。
- **RAM** 碰撞实时断言：在仿真环境下通过宏内建了 SDP RAM 同拍同址读写碰撞断言，一旦发生未授权冲突即刻中断报警。

第 11 章　工程交付与复现指南

11.1　统一的设计方法与架构收敛

本设计以纯 RTL 将 AFDX 线速收发、完整性检查、冗余管理以及 RTC、SNMP、615A 三项高层服务收敛到同一套硬件架构中，核心工程方法归纳如下：

1. 单时钟同步数据通路与多周期使能域解耦。端系统逻辑统一运行于 125 MHz（8 ns 周期）主时钟，字节流传输、BAG 调度计时和 SNTP 时间戳采样共享同一时钟基准；对于复杂的管理协议状态机，通过局部使能两分频树（`CE_DIV=2`）构建隔拍更新的 16 ns 多周期路径，在消除异步跨时钟域亚稳态风险的同时实现了物理时序的稳固收敛。
2. 状态入表与硬件逻辑分时复用。52 个 `afdx_ram_sdp` 模块封装实例（综合后映射为 366 个 EBR 原语）统一采用简单双端口（SDP）同步读写接口承载载荷、配置、各 VL 独立状态与协议报文。128 条发送 VL 和 4096 条接收 VL 的状态全部按地址检索，字节流水与判定逻辑在全链路间分时复用，使组合逻辑深度与 VL 数量完全解耦。
3. 读写端口独占与无冲突时序仲裁。每张表的读写端口归属严格唯一：收发引擎独占状态表读写；配置总线对配置表的写入通过许可信号 `cfg_ok` 插入引擎空闲拍，从结构上消除同拍同址读写冲突；交付队列迁入块 RAM，由微状态机流式读出，消除宽位多路选择树。
4. 全流水寄存切分与精细时延补偿。所有 RAM 读出数据先打拍再参与算术或比较；发送侧组帧器采用 6 拍提前量精确补偿内部读取流水与流缓冲延迟，保证到期首字节准点送出；RTC 对 5 拍写回流水精确补偿 40 ns，保障时钟同步精度。

11.2　工程交付清单与 24 个 RTL 模块

工程完整源码交付于仓库 `rtl/` 目录，共包含 24 个 Verilog 文件，职责划分见表 11-1。

表 11-1 交付的 24 个 Verilog RTL 源码清单

源文件名	所属子系统	模块核心功能与职责
AFDX_top.v	顶层模块	顶层封装，整合双网收发、高层服务、配置总线与时间基准
afdx_timebase.v	时间基准	维护 32 位 now 时基与 16 位 now_c 冗余粗计时
afdx_cfg.v	配置与统计	32 位配置总线译码、27 个计数器两级流水维护与写握手
afdx_svc.v	服务通道	4 路整消息轮询发送仲裁（寄存输出）与 UDP 端口接收分发
afdx_tx_core.v	发送核心	发送顶层包装，集成分页缓存、引擎、组帧器、MAC 与输出流缓冲
afdx_tx_buf.v	发送核心	消息分页写入、写口流水寄存、描述符登记与失败页面释放
afdx_tx_free.v	发送核心	256 页空闲页池 FIFO，支持双页面回收端口并发回收
afdx_tx_engine.v	发送核心	128 条 VL 的 BAG 轮询调度、入队许可、帧授权与到期发送确认（TX Start Confirm）
afdx_tx_builder.v	发送核心	帧头组装、IP 校验和准备、载荷流预读与 -6 拍提前补偿
afdx_mac_tx.v	发送核心	纯 RTL 自研双网 MAC，单状态机两网同拍起始与发送下溢（TX Underrun）检测
afdx_crc32_d8.v	通用算术	8 位并行 CRC-32 组合逻辑生成与校验器
afdx_rx_core.v	接收核心	接收顶层包装，集成双 MAC、解析器、多 Bank 分块映射表与交付模块
afdx_mac_rx.v	接收核心	接收 MAC，SFD 定界、CRC32 流式累加校验与 FCS 剥离
afdx_rx_parse.v	接收核心	帧头检查、UDP 校验和验证、载荷预写入与帧尾分类
afdx_rx_engine.v	接收核心	VLID 映射、完整性检查（IC）、冗余管理（RM）与老化扫描
afdx_rx_deliver.v	接收核心	读取块 RAM 交付队列 rx_dq，从环形缓冲逐字节流式交付
afdx_ram_sdp.v	存储原语	简单双端口（Simple Dual-Port, SDP）同步 RAM 原语封装
afdx_rtc.v	高层服务	实时时钟、SNTPv4 纳秒级时钟同步与时间戳收发状态机
afdx_ntp_fraction.v	高层服务	本地纳秒时基与 NTP 32 位二进制小数逐位迭代换算器
afdx_snmp.v	高层服务	SNMP 顶层包装、UDP 请求捕获与 CE_DIV=2 使能复制树
afdx_snmp_engine.v	高层服务	SNMPv1/v2c ASN.1/BER 解码、MIB 读写与响应生成（16 ns 域）
afdx_615a.v	高层服务	615A 顶层包装、TFTP 捕获、6 块协议 RAM 接口与使能树
afdx_615a_engine.v	高层服务	ARINC 615A 数据加载协议状态机与文件 CRC-16 硬件校验（16 ns 域）
afdx_mgmt_tx.v	高层服务	管理面流式预取发送适配器（8 字节预取队列，125 MHz 主时钟域解耦）

11.3 仿真测试复现指南

全套验证用例均可由纯脚本自动化一键复现，环境要求为 ModelSim SE-64 2020.4 或兼容版本：

1. **8** 项核心评审用例一键运行：在 ModelSim 命令行窗口或脚本中进入工程根目录，直接执行：

   ```
   do sim/run_judge.do
   ```

   该入口无需配置 Python 环境，自动连续执行 8 项核心评审用例（services、capacity、snmp、burst77、burst81、rtc、performance、perf_var），实测总耗时为 8787.198 秒，判定全部为 PASS。
2. 全量 **39** 项自动化回归测试：在 PowerShell 下执行全量回归脚本（可附加 -Coverage 统计覆盖率）：

```
powershell -NoProfile -ExecutionPolicy Bypass -File sim/regress.ps1
```

自动完成全套 39 项测试平台的编译、仿真与参考模型交叉断言校验。

11.4 eLinx 物理实现与时序签核四步闭环

工程提供基于中科亿海微官方 eLinx 3.0.7 工具链的完整命令行批处理脚本，支持全流程一键复现与物理签核闭环：

1. 步骤一：官方逻辑综合与 **VQM** 导出

   ```
   powershell -ExecutionPolicy Bypass -File elinx/synth.ps1
   ```

   执行 `S2d` 综合策略，保持模块层级与时钟使能拓扑，生成门级网表与逻辑打包（Pack）报告。

2. 步骤二：物理布局布线（**Place & Route**）

   ```
   powershell -ExecutionPolicy Bypass -File elinx/impl.ps1
   ```

   执行 `PRES1.50` 物理优化，完成管脚布局布线并生成物理时序数据库。

3. 步骤三：多周期路径约束合法性审计

   ```
   python3 tools/check_multicycle.py <VQM_PATH> --constraints elinx/constraints
       .edc --output out/multicycle_audit
   ```

   对全芯片时序约束进行严格审计，核对 2,755 个引擎寄存器的使能引脚与多周期声明，确认 0 非法路径、0 漏约束。

4. 步骤四：布线后静态时序全量导出与签核

   ```
   python3 tools/report_routed_timing.py
   ```

   自动解析布局布线后的真实时序数据库，核验 8 ns 单周期路径（最差建立裕量 $+0.013$ ns）、16 ns 多周期路径（最差建立裕量 $+0.940$ ns）以及同沿保持时间裕量（$+0.229$ ns），输出 45,745 个端点 0 违例的权威签核报告。

附录 A　源文件与默认参数

A.1　源文件

RTL 共 24 个 Verilog 文件，均在仓库 `rtl/` 目录。

表 A-1　RTL 源文件清单（24 个模块）

文件	作用
`AFDX_top.v`	顶层：双网收发核心、服务通道、三项服务、配置与时间基准
`afdx_timebase.v`	周期计数 `now` 与粗粒度计时 `now_c`
`afdx_cfg.v`	地址译码、寄存器、27 个计数器两级流水维护、配置表写握手
`afdx_svc.v`	应用与服务的消息轮询发送（授权寄存）、按端口分发接收
`afdx_tx_core.v`	发送核心集成模块，例化发送侧各模块、输出流缓冲和 7 张表
`afdx_tx_buf.v`	消息分页写入（写口流水寄存）、入队请求、失败后回收缓存页
`afdx_tx_free.v`	256 页空闲页 FIFO，支持双页面回收端口并发回收
`afdx_tx_engine.v`	BAG 调度：扫描、入队、授权、发送起始确认（TX Start Confirm）
`afdx_tx_builder.v`	组帧、IP 头校验和、载荷预读、-6 拍提前补偿
`afdx_mac_tx.v`	双网发送 MAC：前导码、FCS、帧间隔、发送下溢（TX Underrun）检测与异常帧终止
`afdx_crc32_d8.v`	逐字节 CRC32 组合逻辑
`afdx_rx_core.v`	接收核心集成模块，例化接收侧各模块、多 Bank 分块映射表和交付模块
`afdx_mac_rx.v`	接收 MAC：SFD 检测、CRC 校验、剥离 FCS
`afdx_rx_parse.v`	帧头检查、载荷预写入环形缓冲、帧尾分类
`afdx_rx_engine.v`	32 子库查表、完整性检查、冗余管理、过期清理
`afdx_rx_deliver.v`	读取块 RAM 交付队列 `rx_dq`，从环形缓冲逐字节交付
`afdx_ram_sdp.v`	简单双端口（Simple Dual-Port, SDP）同步 RAM，全设计唯一的 RAM 例化原语
`afdx_rtc.v`	实时时钟与 SNTPv4 纳秒级时钟同步
`afdx_ntp_fraction.v`	纳秒与 NTP 小数的逐位换算
`afdx_snmp.v`	SNMP 顶层包装：请求捕获、MIB/Trap RAM 接口与 `CE_DIV=2` 使能树
`afdx_snmp_engine.v`	SNMP 协议核心：ASN.1/BER 编解码、MIB 读写与响应生成（16 ns 域）
`afdx_615a.v`	615A 顶层包装：TFTP 捕获、6 块协议 RAM 接口与使能树
`afdx_615a_engine.v`	615A 协议核心：数据加载状态机与文件 CRC-16 硬件校验（16 ns 域）
`afdx_mgmt_tx.v`	管理面流式预取发送适配器：8 字节预取队列，供 SNMP 与 615A 复用解耦

A.2 默认参数

表 A-2 顶层参数与默认容量

参数	默认值	含义
TX_PAGE_AW	8	发送缓存 256 页
TX_OFF_AW	9	每页 512 字节，共 128 KiB
LOOKAHEAD	512	提前选择窗口，512 拍（4.096 µs）
RX_RING_AW	12	每网接收环形缓冲 4 KiB
RX_DQ_AW	5	交付描述队列 32 项
发送 / 接收 VL	128 / 4096	接收映射表 65536 项
帧间隔	12 拍	96 ns
初始化	65538 拍	解除复位到 tx_ready，524.304 µs
SNMP 请求缓存	256 B	单 VarBind
615A 加载区	8 KiB	头文件区 2 KiB，LUR 区 512 B
615A 超时 / 重传	2 s / 1 次	可经寄存器修改
RTC 同步周期	125000 拍	1 ms，可经寄存器修改

附录 B　服务寄存器

基本寄存器、计数器和配置表地址见表 8-1，服务寄存器见表 B-1。RW 为可读写，RO 为只读。

表 B-1　服务寄存器

地址	名称	读写	含义
RTC			
0x80	CTRL	RW	位 0 启用，位 1 Server 模式；写位 2 装入暂存时间
0x81	PERIOD	RW	Client 端请求间隔（拍），复位 125000
0x82	TX_VL	RW	发送 VL，低 7 位
0x83	UDP_PORT	RW	UDP 端口，复位 123
0x84	SEC	RW	写秒暂存值；读当前秒并锁存纳秒
0x85	NS	RW	写纳秒暂存值；读锁存的纳秒值
0x86	OFFSET_SEC	RO	最近偏差的有符号秒部分
0x87	OFFSET_NS	RO	最近偏差的纳秒余数
0x88	SYNC_COUNT	RO	Client 时钟同步次数
SNMP			
0x90	ENABLE	RW	位 0 启用代理
0x91	TX_VL	RW	响应与 Trap 的发送 VL
0x92	COMMUNITY_LENGTH	RW	1–16，复位 6
0x93–0x96	COMMUNITY_0–3	RW	community 内容，复位为 public
0x97	SAMPLE_CYCLE	RO	上次 MIB 读取时的 now
0x98	UPTIME	RO	sysUpTime，单位 10 ms
0x99	REQUESTS	RO	端口匹配的请求数
0x9A	BAD_COMMUNITY	RO	community 不符次数
0x9B	TRAP_ENABLE	RW	位 0 启用 Trap
0x9C	TRAPS_SENT	RO	已发送 Trap 数
0xA0–0xA3	DESCRIPTION_0–3	RO	sysDescr 字符串
615A			
0xB0	ENABLE	RW	位 0 启用
0xB1	TX_VL	RW	发送 VL
0xB2	STATUS	RO	位 1:0 空闲/进行中/成功/失败；位 15:8 原因码
0xB3	LOADED_BYTES	RO	本次已写字节数
0xB4	MEMORY_ADDRESS	RW	加载区读回地址，低 13 位
0xB5	MEMORY_DATA	RO	加载区读回字节
0xB6	LUS_COUNTER	RO	本次状态文件数
0xB7	RETRY_TOTAL	RO	累计超时重传次数
0xB8	REQUEST_PORT	RW	TFTP 请求端口，复位 59
0xB9	TIMEOUT_CYCLES	RW	超时拍数，复位 250000000
0xBA	RETRY_LIMIT	RW	重传上限，复位 1
0xBB	PROTOCOL_VERSION	RW	A4 或 A3

附录 C　内部接口信号

本附录按信号说明第 4.7 节汇总的内部接口。

C.1　消息接口

服务通道 afdx_svc 把选中的消息以字节流交给发送核心，见表 C-1。没有服务报文时，这组信号与应用口组合直通。

表 C-1　消息接口（afdx_svc → afdx_tx_buf）

信号	位宽	含义
m_valid、m_ready	1、1	字节握手；m_ready 只在 tx_buf 处于 IDLE、RECV、DRAIN 时为 1
m_data、m_last	8、1	消息字节、末字节标记
m_vl	7	发送 VL 索引，首字节锁存
m_port_en、m_sport、m_dport	1、16、16	m_port_en 为 1 时使用消息自带的 UDP 端口；首字节锁存

C.2　入队与授权

afdx_tx_buf 写完消息后，将描述符写入 tx_desc，再向引擎发出入队请求。引擎选中一条消息后，将整帧字段一次性交给组帧器；组帧器在帧开始发送时报告起始时刻，见表 C-2。

表 C-2　入队、授权和发送起始指示接口

信号	位宽	含义
入队（*tx_buf* → *tx_engine*，请求应答）		
enq_valid	1	保持有效，直至收到 enq_ack
enq_vl、enq_head、enq_len	7、8、11	发送 VL 索引、消息首页号、字节数
enq_ack、enq_result	1、3	单拍应答；0 接纳，1 VL 未启用，2 队列满，3 超长（4 无空闲页由 tx_buf 自行判定）
tx_desc[43:0]	44	{len[10:0], port_en, sport[15:0], dport[15:0]}，以首页号为地址

接下页

表 C-2 （续）

信号	位宽	含义
授权（*tx_engine* → *tx_builder*，单深缓冲握手（*Single-Slot Handshake*）		
grant_valid、grant_ready	1、1	grant_ready = ¬pending，即单帧待发槽位空闲
grant_a、grant_b	1、1	两网启用位（配置字 0 的 [1]、[2]）
grant_timed、grant_time	1、32	grant_timed 为 1 时，帧不得早于 grant_time 起始；为 0 时可立即发送
grant_vlid	16	配置字 2
grant_src、grant_dst	32、32	配置字 3、4
grant_sport、grant_dport	16、16	port_en 为 1 取消息自带端口，否则取配置字 5
grant_len、grant_sn、grant_page	11、8、8	载荷字节数、帧序号、首页号
发送起始指示（*TX Start Indication*）（*tx_builder* → *tx_engine*）		
start_valid、t_start	1、32	单拍脉冲；t_start 为输出首个前导字节时的 now

C.3 组帧器与 MAC

组帧器将目的 MAC 地址至 SN 字段的帧内容以字节流送至 MAC，由 MAC 控制字节接收时刻，见表 C-3。

表 C-3 帧字节接口（tx_builder ↔ mac_tx）

信号	位宽	含义
s_valid、s_ready	1、1	字节握手；s_ready 在 byte_en= 1 且满足以下任一条件时为 1：前导码末拍；DATA 且 last_byte= 0；DRAIN
s_data_a、s_data_b	8、8	两网字节，仅第 11 字节不同
s_last	1	与 SN 字节同拍
s_en_a、s_en_b	1、1	起始拍锁存，选择本帧发往哪一网
tx_sof、tx_eof	1、1	首个前导字节、帧尾（FCS 发完）的单拍脉冲
tx_underrun	1	数据阶段上游未备好字节的下溢（Underrun）指示

C.4 帧描述与交付队列

每网解析器为正确帧生成一份 95 位帧描述。引擎读取描述并完成判定后，向解析器发出提交或回退指示；对于通过判定的帧，引擎还向交付队列写入一项，见表 C-4。

表 C-4 帧描述与交付队列项

字段	位宽	含义
帧描述（*rx_parse* → *rx_engine*，单深缓冲握手（*Single-Slot Handshake*），高位在前）		
vlid	16	目的 MAC 第 4–5 字节
sn	8	帧末 SN
sport、dport	16、16	UDP 端口
n	11	载荷字节数
start	12	本帧载荷在环形缓冲中的起点
arrival	16	帧尾时刻的 now_c
desc_valid、desc_take	1、1	交接握手
resolve_valid、resolve_commit	1、1	引擎判定：提交则 wr_commit 前进到本帧末，否则 wr_trial 回退
交付队列项（*rx_engine* → *rx_deliver*，*32* 项 × *68 bit* 块 *RAM rx_dq*，高位在前）		
net[67]	1	0 为 A 网，1 为 B 网
start[66:55]、n[54:44]	12、11	环形缓冲起点和字节数
idx[43:32]	12	接收 VL 索引
sport[31:16]、dport[15:0]	16、16	UDP 源端口与目的端口（共 68 位）

交付口 d_* 的字段与应用接收口对应（目的端口在内部记为 d_dport，在顶层记为 app_rx_port）。服务通道将这些信号同时广播给应用和三个服务。各服务在首字节到达当拍，根据 UDP 目的端口产生组合匹配信号 svc_rx_claim；匹配结果寄存并保持至末字节。端口匹配的消息由对应服务接收，不向应用口输出。

C.5 服务发送总线

三个服务的发送口打包成一组总线接到 afdx_svc，按 RTC、SNMP、615A 从低位到高位拼接，见表 C-5。各服务按相同的字节流握手约定驱动对应位段。

表 C-5 服务发送总线 svc_tx_*

信号	位宽	拼接
svc_tx_valid、_ready、_last、_port_en	各 3×1	[0] RTC，[1] SNMP，[2] 615A
svc_tx_data	3×8	[7:0] RTC，[15:8] SNMP，[23:16] 615A
svc_tx_vl	3×7	[6:0]、[13:7]、[20:14]
svc_tx_sport、svc_tx_dport	各 3×16	[15:0]、[31:16]、[47:32]

附录 D　术语与参考文献

D.1　术语

表 D-1　本文专用术语

术语	含义
拍	一个 125 MHz 时钟周期，8 ns
PSN	同一网上同一 VL 上一帧的序列号
预写入 / 提交 / 回退	载荷先写入环形缓冲；通过检查后更新提交写指针，拒绝时回退预写入指针
候选	调度引擎选出、等待授权给组帧器的 VL
`pend_vl`	已授权但尚未开始发送的 VL
`cfg_ok`	引擎在本拍不发出配置表读的指示
有效副本	帧格式、FCS 与协议头检查通过，且满足接收配置和已启用的完整性检查的单网帧副本
页面释放	将页号送回空闲页池（经页面回收口 `ret1_*` 或 `ret2_*`）
端口匹配	服务根据 UDP 目的端口确定是否接收当前消息（信号 `svc_rx_claim`）
表初始化	复位后逐项清零收发配置表与状态表（`rx_map`、`rx_cfg`、`rx_state`、`tx_cfg`、`tx_state`）的过程
线路占用	一帧在线路上的占用时长 $C(n)$，单位为拍
Server / Client	SNTP 角色：Server 端应答请求并回显时间戳；Client 端周期发送请求并执行时钟同步校准
应答	请求应答式握手的确认脉冲（`ack`），用于确认接口请求，与协议响应报文不同
计时单位	`now_c` 的单位，32.768 µs
发送抖动	帧起始相对 BAG 允许时刻的偏差
起始指示 / 起始确认	组帧器在 `tx_sof` 有效拍经 `start_valid/t_start` 发送起始指示（TX Start Indication）；发送引擎据此完成发送起始确认（TX Start Confirm）并写回更新后的 `next_time`

D.2　参考文献

[1] ARINC Specification 664 Part 7: Avionics Full Duplex Switched Ethernet Network, 2005-06-27.

[2] ARINC Report 615A-2: Software Data Loader Using Ethernet Interface, 2002-05-10.

[3] ARINC Report 665-1: Loadable Software Standards, 2001-01-12.

[4] RFC 1350: The TFTP Protocol (Revision 2), 1992.

[5] RFC 2347: TFTP Option Extension, 1998.

[6] RFC 1157: A Simple Network Management Protocol (SNMP), 1990.

[7] RFC 3416: Version 2 of the Protocol Operations for the Simple Network Management Protocol (SNMP), 2002.

[8] RFC 3418: Management Information Base (MIB) for the Simple Network Management Protocol (SNMP), 2002.

[9] RFC 4330: Simple Network Time Protocol (SNTP) Version 4 for IPv4, IPv6 and OSI, 2006.

[10] IEEE Std 802.3: IEEE Standard for Ethernet, 2018.

[11] 中科亿海微. 全国大学生嵌入式芯片与系统设计竞赛 FPGA 创新设计赛道选题指南.